

Critical Rounds in Multi-Round Proofs: Proof of Partial Knowledge and Trapdoor Commitments

Masayuki Abe¹[0009–0006–1614–0587], David Balbás²[0000–0003–4864–1125],
Dung Bui³[0000–0001–7049–733X], Miyako Ohkubo⁴[0009–0006–8656–035X],
Zehua Shang⁵[0009–0007–9115–0117], Akira Takahashi⁶[0000–0001–8556–3053], and
Mehdi Tibouchi¹[0000–0002–2736–2963]

¹ NTT Social Informatics Laboratories, Japan

abe.masayuki@iecl.ntt.co.jp, mehdi.tibouchi@ntt.com

² ETH Zurich, Switzerland david.balbas@imdea.org

³ Sorbonne Université, CNRS, LIP6, Paris, France dung.bui@lip6.fr

⁴ NICT, Japan m.ohkubo@nict.go.jp

⁵ Kyoto University, Japan shang.zehua.23m@st.kyoto-u.ac.jp

⁶ J.P. Morgan AI Research & AlgoCRYPT Center of Excellence, USA

takahashi.akira.58s@gmail.com

Abstract. Zero-knowledge simulators, initially developed for proving the security of proof systems, turned out to be also useful in constructing advanced protocols from simple three-move interactive proofs. However, in the context of multi-round public-coin protocols, the interfaces of these auxiliary algorithms become more complex, introducing a range of technical challenges that hinder the generalization of these constructions.

We introduce a framework to enhance the usability of zero-knowledge simulators in multi-round argument systems for protocol designs. Our main conceptual contribution is *critical-round zero-knowledge*, which refers to the ability to perform complete zero-knowledge simulations by knowing the challenge of just one specific round in advance. We show that this notion is satisfied by diverse protocols based on MPC-in-the-Head, interactive oracle proofs, and split-and-fold arguments.

We demonstrate the usefulness of the critical round zero-knowledge notion by constructing proofs of partial knowledge (Cramer, Damgård, and Schoenmakers, CRYPTO’94) and trapdoor commitments (Damgård, CRYPTO’89) from critical-round multi-round proofs.

Keywords: Multi-Round Proofs, Critical Round, Composition, Proofs of Partial Knowledge, Trapdoor Commitment

Table of Contents

Critical Rounds in Multi-Round Proofs: Proof of Partial Knowledge and Trapdoor Commitments	1
<i>Masayuki Abe, David Balbás, Dung Bui, Miyako Ohkubo, Zehua Shang, Akira Takahashi, and Mehdi Tibouchi</i>	
1 Introduction	3
1.1 Our Contributions	4
1.2 Related Work	5
2 Technical Overview	6
2.1 The Critical Round Framework	6
2.2 Multi-Round Proof of Partial Knowledge	7
2.3 Trapdoor Commitment	8
3 Preliminaries	10
3.1 Notation	10
3.2 Public-Coin Proof System	10
3.3 Trapdoor Commitments	11
4 Critical-Round Zero-Knowledge	12
4.1 Definitions	12
4.2 Instantiations of Critical Round Proofs	13
5 Multi-Round Proof of Partial Knowledge	15
5.1 Construction	15
5.2 Security	15
5.3 Suppressing Exponential Blow-up via Multi-Round Share-then-Hash	19
6 Trapdoor Commitment from Multi-Round Protocol	20
7 Conclusion	24
A Composition of Multi-Round Protocols via CDS	28
A.1 Monotone Access Structure	28
A.2 Secret Sharing Scheme	28
A.3 Insecurity of Naïve Multi-Round CDS	29
B Composition of Multi-Round Protocols via Share-then-Hash	29
B.1 Extending Predicate Special Soundness	30
B.2 Analysis of Share-then-Hash in the Predicate Special Soundness Framework	31
C Other Definitions	34
C.1 Relations to Other Notions	35
D Critical Rounds in the KKW Framework	36
D.1 KKW Framework	36
D.2 Critical Round in KKW	37
E More Applications	40
E.1 Accumulators	40
E.2 Trapdoor Commitments with Flexible Trapdoor Allocation	43

1 Introduction

Public-coin *three-move proofs* form a foundational building block in cryptographic protocol design. In such a protocol, a prover aims to convince a verifier of a statement x by first sending a commitment a , receiving a random challenge c , and responding with a message z . The canonical notions of security for these protocols are *special soundness* (SS), which enables witness extraction from two accepting transcripts (a, c, z) and (a, c', z') for $c \neq c'$, and *special honest-verifier zero-knowledge* (SHVZK), which ensures that a valid-looking transcript can be simulated when the verifier’s challenger c is fixed in advance. Three-move protocols are ubiquitous in cryptography, underpinning applications such as identification schemes, signature schemes, and zero-knowledge proofs.

Recent developments have extended this design paradigm to multi-round public-coin protocols, where transcripts are of the form $(x, a, c_1, z_1, \dots, c_\mu, z_\mu)$ and where each challenge c_i is sampled in round i . The classical notion of special soundness [Cra96] generalizes naturally to the $(k_1, \dots, k_\mu)$ -special soundness framework [ACK21, AFR23], in which knowledge extraction is guaranteed given k_i distinct accepting responses to different challenges at each round i , or in other words, given a so-called accepting *tree of transcripts*. For arguments based on computational hardness assumptions, (computational) special soundness [AFR23] is defined with respect to an *extended tree of transcripts*: given such a tree of accepting transcripts, the extended extractor either outputs a witness for the original relation or solves a related hard problem. A corresponding extension of SHVZK also holds in this setting, where *knowledge of the full challenge sequence* $(c_1, \dots, c_\mu)$ enables simulation of an accepting transcript.

These properties are sufficient to instantiate non-interactive zero-knowledge (NIZK) proofs and signature schemes via the Fiat-Shamir paradigm [FS87], where challenges are deterministically derived from prior transcript data through a random oracle. The multi-round analogue remains secure in the Fiat-Shamir setting: one can rewind the protocol at each round by programming the random oracle (to construct a tree required by special soundness), and simulate all challenges in advance (to rely on SHVZK). However, there are other natural applications where three-move protocols are used but their multi-round analogues either fail to apply or incur fundamental limitations. We examine two examples below.

Proof of Partial Knowledge. Composing proof protocols is a powerful technique for demonstrating partial knowledge of witnesses for compound statements. Consider, for example, a disjunctive statement $A \vee B$, where the prover knows a witness for A but not for B . The prover wishes to convince the verifier that the disjunction holds, without revealing which statement it can actually prove. The Cramer-Shoenmakers-Damgård (CDS) composition [CDS94] achieves this with three-move protocols. The core idea is to treat the challenge c sent from the verifier as a secret to be shared between the two statements. The prover can then answer any challenge c_A for A using its witness, while for B , it can simulate the proof *by running the zero-knowledge simulator* with a fixed challenge share c_B in advance. When the verifier provides the actual challenge c , the prover *completes* the shares by setting $c_A := c \oplus c_B$. This allows the prover to compute the response z_A for challenge c_A using its witness for A , while also using the pre-computed simulation (a_B, z_B) for the fixed challenge c_B . This observation naturally extends to composing statements following any monotone functions efficiently computable with *monotone span programs* (MSP) [CDM00].

One could naïvely extend the CDS approach to multi-round protocols $(a, c_1, z_1, \dots, c_\mu, z_\mu)$ as follows. Standard SHVZK simulation for multi-round protocols typically requires the simulator to know *all* future challenges $(c_1, \dots, c_\mu)$ in advance. If a prover tries to simulate the proof for B in a multi-round composition, it would need to fix the entire sequence of challenges $(c_1^B, \dots, c_\mu^B)$ beforehand. The challenges for the A part $(c_1^A, \dots, c_\mu^A)$ would then be derived from the verifier’s actual challenges $(c_1, \dots, c_\mu)$ and the fixed B -challenges using the share completion mechanism in each round. However, it is well known in the literature that such a straightforward generalization of CDS gives rise to an insecure protocol due to the following “cross attack” on the (knowledge) soundness of the composition [KLP22, FGQ⁺23, GHAKS23, ABO⁺24]. Essentially, the naïve multi-round version of CDS allows a cheating prover to fix the challenges for the A -part in some of the rounds and for the B -part in the others, thereby preventing a knowledge extractor from building a full tree of transcripts through rewinding (see also Appendix A.3 for details).

While previous works [KLP22, FGQ⁺23, GHAKS23, ABO⁺24] have addressed this issue in clever ways, these methods still fall short of matching the original CDS composition: they support only

less expressive access structures and rely on additional assumptions (see Table 1 and Section 1.2 for details). As such, composing multi-round protocols while preserving the full potential of the original CDS composition remains an open question.

Trapdoor Commitments. Another simple yet surprising example are trapdoor commitment (TDC) schemes [Dam90, Fis01], which allow a committer to commit to a message m and open it to any different message m' (i.e. equivocate it) only if it knows a trapdoor td . A trapdoor commitment can be constructed from a public-coin three-move protocol Π as follows. The commitment key is an instance x , and the trapdoor is a witness w such that $R(x, w) = 1$ for some hard relation R . To commit to a message m , the committer runs the zero-knowledge simulator of Π using m as a fixed challenge, obtaining a and z . The committer sends $com := a$ as a commitment, which is hiding due to SHVZK of Π , and sends $open = (m, z)$ to open it, which can be verified by running the verification algorithm of Π . Finally, to equivocate a on m' with the knowledge of a witness w , the committer simply runs the honest prover of Π to obtain z' such that (x, a, m', z') is a valid transcript. Binding follows from the special soundness of Π . Given two openings (m, z) and (m', z') such that $m \neq m'$, one can simply run the extractor on the two transcripts to obtain a witness w such that $R(x, w) = 1$. Note that this TDC construction does not require programming random oracles during the equivocation phase, unlike the naïve instantiation from programmable random oracles, i.e., $com := H(m||r)$ for a random oracle H and randomness r . This feature allows TDC to be equivocated during a real-world protocol execution, which is crucial for applications such as threshold ring signatures (see Section 7).

A central question in extending this construction to the multi-round case is: “Where and how should the committed message m be embedded?” If we embed m as the i -th round challenge, the committer still needs to select all the remaining challenges, as otherwise it would not be able to run the zero-knowledge simulator. But in this case, it does not seem possible to rewind the prover to obtain a full tree of transcripts.

The Challenge. The failures in extending these constructions to the multi-round case seem to stem from the following issues. The standard notion of SHVZK requires the zero-knowledge simulator to fix all challenges in advance, which significantly restricts flexibility in protocol design. This limitation becomes particularly evident when the simulator must be invoked as a component within other protocols, rather than being used solely in the security proof.

These observations raise the following questions: *Can we identify a more flexible characterization of multi-round public-coin protocols that enables applications beyond the current scope of SHVZK? And in particular, can such a framework capture efficient protocols already in use, thereby extending their utility to applications traditionally limited to the three-move setting?*

1.1 Our Contributions

In this paper, we advance the study of multi-round public-coin proofs by introducing *critical rounds*, and by analyzing their implications. We expand on these below and introduce further details, intuition, and (informal) theorem statements in the technical overview in Section 2.

1. *Study of Critical Round Protocols.* We introduce and formalize *critical round zero-knowledge (CRZK)*, where there exists a (critical) round i_{zk}^* such that the transcript can be simulated only by knowing the challenge at round i_{zk}^* . We then show that the critical-round framework captures important design paradigms, such as those based on MPCitH and IOPs. We prove that several popular protocols satisfy these notions, including the KKW MPCitH protocol [KKW18], PlonK [GWC19], Bulletproofs [BBB⁺18] and compressed Σ -protocols [AF22].
2. *Multi-Round Proof of Partial Knowledge.* We show that multi-round proofs with the CRZK property are amenable to secure construction of multi-round proofs of partial knowledge via a generalization of the CDS composition, essentially by limiting the application of the CDS technique to the critical round. Compared to existing approaches to multi-round composition, our result supports more expressive access structures than simple disjunctions and thresholds, while introducing no or mild extra assumptions. We give a comparison to other multi-round compositions in Table 1 to show that our composition (Section 5) is as powerful as the original CDS while preserving the zero-knowledge property. While our resulting protocols are relatively simple, the

Scheme	Underlying Protocol			Composed Protocol			Composition	Extra Assumptions
	# of Rounds	Soundness	ZK	# of Rounds	Soundness	ZK		
CDS94 [CDS94, CDM00]	3	2-SS	(S)HVZK	3	2-SS	(S)HVZK	MSP	-
FGQRW23 [FGQ ⁺ 23]	$2\mu + 1$	RBRs	HVZK	1	CS	HVZK	1-out-of- n	NPROM, CRS
Speed-Stacking [GHAks23]	$2\mu + 1$	CS	EHVZK	$2\mu + 1$	CS	EHVZK	t -out-of- n	NIPBC(CRS)
ABORST24 [ABO ⁺ 24]	$2\mu + 1$	$\tilde{k}$ -SS	HVZK	$2\mu + 1$	$\tilde{k}$ -SS	SHVCZK	mNC ¹	DualCom
MR-CDS (Sec.5)	$2\mu + 1$	$\tilde{k}$ -SS	CR(S)HVZK	$2\mu + 1$	$\tilde{k}'$ -SS	CR(S)HVZK	MSP(const. n)	-
MR-StH (App.B)	$2\mu + 1$	$\tilde{k}$ -SS	CR(S)HVZK	$2\mu + 1$	$(\tilde{k}', \Phi)$ -PSS	CR(S)HVZK	MSP	CRH

Table 1. Multi-round composition methods. Legend: x-(C)SS: (Computational) x-Special Soundness (Def. 3), RBRs: Round-by-Round Soundness (Def. 18), CS: Computational Soundness, (S/E)HV(C)ZK: (Special/Enhanced) Honest Verifier (Computational) ZK (Def. 4), PSS: Predicate Special Soundness (Def. 19), CRH: Collision-Resistant Hash Function.

analysis of their knowledge soundness turns out to be surprisingly delicate, due to parameter degradation unique to multi-round protocols. To address this challenge, we provide two flavors of multi-round composition: (1) *Multi-Round CDS*, supporting a constant number of statements with no extra assumptions, and (2) *Multi-Round Share-then-Hash*, enabling the composition of a polynomial number of statements assuming the existence of a collision-resistant hash. The latter can be viewed as the first multi-round generalization of the Share-then-Hash (StH) technique by Abe et al. [AAB⁺20] that we manage to realize without random oracles, improving on the original scheme.

3. *Trapdoor Commitments from Critical-Round Protocols.* We present a transformation from critical-round proofs to trapdoor commitment schemes which also relies on the properties of critical rounds. These results enable new recipes for instantiating trapdoor commitments, such as within the compressed Σ -protocols [AC20, ACK21]. We also construct an *accumulator* that binds multiple messages into a single string.

1.2 Related Work

Critical Round Zero-Knowledge. There exist multiple notions in the literature that are closely related to our notions of CRZK. One of the closest relations to CRZK is k -zero-knowledge in [GKK⁺22, DG23], which is a similar concept that is formulated for compiled non-interactive protocols via the Fiat-Shamir transformation. Therefore, it is formalized in the programmable random oracle model. This notion was originally developed to compensate for the absence of rewinding and challenge programmability in simulation-extractable SNARKs [CHM⁺20, GWC19, Set20, BBB⁺18]. One can see CRZK as a generalization and refinement of k -zero knowledge for interactive protocols.

Notions of Special Soundness. Since its introduction for Σ -protocols [Cra96], special soundness has been extended to capture a variety of public-coin proof protocols. To handle parallel soundness amplification of k -special sound protocols, [AAB⁺21] relaxes the extractor so it works only for transcripts that satisfy a predicate. The works in [AF22, AFR23] formalize a multi-round generalization of (possibly computational) special soundness, known as $(k_1, \dots, k_\mu)$ -special soundness. This concept is then extended to manage erroneous transcripts in several ways: almost special soundness in [BF23] provides a framework to analyze protocols based on deterministic commitments, whose opening gets checked probabilistically through a random challenge. Predicate special soundness in [AAB⁺24] uses predicates over challenges in each round; adaptive special soundness [AKLY24] conceptualizes a useful challenge subspace with respect to a set of transcripts available in each state of the extractor; statistical special soundness in [ABO⁺24] allows a statistical error over all challenges in a tree of transcripts; and $\mathfrak{G}$ -soundness [DFMS22] broadly describes conditions under which the extractor functions.

Compositions of Proofs. There are many composition methods for three-move proofs in the literature, e.g., [CDS94, CPS⁺16a, CPS⁺16b, AAB⁺20, AAB⁺21, ACF21, ZLH⁺22, GGHAK22, ABFV22, ABF⁺25]. The CDS composition is the most powerful in terms of the expressiveness of the composition and the generality of the underlying protocols, as it enables arbitrary access structures represented by monotone span programs [CDM00]. [AAB⁺20] extends the underlying protocol to k -special sound ones for $k > 2$ using the so-called Share-then-Hash technique, which is employed with a refined security

analysis in this paper. Other methods focus on reducing communication complexity or providing additional properties, such as delayed input.

In Table 1, we overview compositions for multi-round protocols. While most of these three-move compositions do not extend to multi-round protocols, Speed-stacking in [GHAKS23] is a multi-round extension of Stacking- Σ [GGHAK22] for three-move compositions. It employs a non-interactive partially binding commitment scheme (NIPBC) that is available in the common reference string (CRS) model making the composed protocol computationally sound. The composition in [FGQ⁺23] transforms round-by-round sound multi-round protocols into a non-interactive argument system in the non-programmable random oracle model (NPROM) and the CRS model. The composition in [ABO⁺24] supports compositions in monotone NC^1 (mNC^1) and preserves the soundness of the underlying protocols. It however uses a dual-mode commitment scheme (DualCom) and results in computational zero-knowledge protocols.

Trapdoor commitments. The idea of constructing a commitment scheme from a three-move public-coin poofs dates back to [Dam90]. There are three-move honest verifier zero-knowledge protocols for NP-complete languages, e.g., [Blu86, GMW91, HV20], that fit the generic construction of TDC. On the other hand, there is currently no known method to construct TDCs from multi-round proof systems.

2 Technical Overview

We start by introducing the intuition behind our notions of critical round zero-knowledge and critical round special soundness. Then, we present the main insights of our two primary use-cases for these notions: multi-round protocol composition for proofs of partial knowledge and trapdoor commitments.

2.1 The Critical Round Framework

Critical Round Zero-Knowledge. Traditional formulations of honest-verifier zero-knowledge in the multi-round setting require the simulator to fix all verifier challenges in advance. In our notion of *critical round special honest-verifier zero-knowledge* (CRSHVZK, Definition 7), simulation is enabled by fixing in advance only a single challenge in a designated *critical round* i_{zk}^* . That is, once the simulator fixes $c_{i_{\text{zk}}^*}$ in advance, it can generate a transcript $(a, c_1, z_1, \dots, c_{i_{\text{zk}}^*}, \dots, c_\mu, z_\mu)$ while reading the remaining challenges in real-time. Precisely, our notion enforces that the simulator can be decoupled into two (stateful) subroutines: SimA and SimZ. SimA generates the initial message: on input the critical round challenge $c_{i_{\text{zk}}^*}$, it outputs a . SimZ generates the remaining prover messages sequentially: on input an arbitrary challenge c_i , it outputs the i -th round message z_i .

Extended Special Soundness. As a generalization of special soundness for Σ -protocols, μ -round multi-round proofs often satisfy $(k_1, \dots, k_\mu)$ -special soundness, requiring the existence of a successful knowledge extractor taking a tree of transcripts as input. However, when the $(k_1, \dots, k_\mu)$ -special soundness is only computational, the guarantee is that the knowledge extractor outputs either a solution to some computationally hard relation R_{hd} or a valid witness for NP relation R . Following the framework in [AFR23], we define an extended extractor ExExt for extended special soundness (Definition 3) that outputs either a valid witness for relation R or a solution to external hardness assumption R_{hd} . Given a valid extended tree of transcripts T , ExExt succeeds with probability 1.⁷

Instantiations of Critical Rounds in Real-World Protocols. Our definitions are useful insofar as they align with how simulators and extractors are defined for some practical proof systems. In Section 4, we show that many existing proof paradigms including MPCitH and IOP-based proofs can be captured by our framework. For instance, the KKW MPCitH protocol [KKW18] is a 5-move interactive honest-verifier zero-knowledge proof that can be shown to be (2,2)-ExSS and CRSHVZK at either round $i_{\text{zk}}^* = 1$ or at round $i_{\text{zk}}^* = 2$.

For IOP-based proofs, a prominent example is the Plonk protocol [GWC19], where the verifier ultimately evaluates all the polynomial commitments sent by the prover on a single evaluation point

⁷ Witness extraction under computational assumptions has been discussed in several works, e.g., [DG23, LPS25, GOP⁺25]. They typically involve computationally bounded adversaries, while our definition ensures that the extended extractor always succeeds.

3, which is the fourth-round challenge in the protocol. As observed in prior work [GKK⁺22], the knowledge of this challenge in advance suffices to simulate the remainder of the transcript, without requiring knowledge of other challenges. As we argue in Proposition 1, Plonk admits a CRSHVZK simulator at $i_{\text{zk}}^* = 4$. Following a similar analysis, we capture Bulletproofs [BBB⁺18] and compressed Σ -protocols [AC20, ACK21] in our framework.

2.2 Multi-Round Proof of Partial Knowledge

Composition from Critical Round Protocols. Our work introduces a general framework for securely composing multi-round interactive proofs *in the plain model* for proving partial knowledge. This framework is presented in Section 5, with an extension via the Share-then-Hash technique analyzed in Appendix B. As recalled in the introduction, a naïve CDS composition quickly runs into soundness problems. Our critical round framework provides a path forward. The crucial insight lies in the properties of CRZK. Recall that a CRZK protocol has a designated critical round i_{zk}^* and a simulator split into (SimA, SimZ). SimA generates the first message a needing only the critical challenge $c_{i_{\text{zk}}^*}$ in advance. Importantly, the stateful SimZ generates subsequent responses z_i sequentially, taking the actual verifier challenge c_i for round i as input *at that round*. This structure allows us to adapt the CDS strategy effectively:

1. **Non-Critical Rounds** ($i \neq i_{\text{zk}}^*$): For rounds other than the critical one, the prover receives the verifier’s challenge c_i . For the statement it holds a witness for (say A), it computes the response z_i^A honestly using the real prover algorithm Z^A . For the statement it is simulating (say B), it uses the stateful simulator SimZ^B with input c_i to compute the response z_i^B . This is possible because SimZ operates round-by-round *after* seeing the challenge.
2. **Critical Round** ($i = i_{\text{zk}}^*$): This round mirrors the logic of the original 3-move CDS. To simulate the B part, the prover must run SimA^B , which requires fixing the critical challenge $c_{i_{\text{zk}}^*}^B$ in advance. The prover samples this value. When the verifier sends the actual challenge $c_{i_{\text{zk}}^*}$ for this round, the prover uses **Complete** algorithm of the secret sharing scheme, along with the pre-sampled $c_{i_{\text{zk}}^*}^B$, to determine the necessary challenge $c_{i_{\text{zk}}^*}^A$ for the A part. It then computes $z_{i_{\text{zk}}^*}^A$ honestly using Z^A and the witness for A , and use SimZ^B (with the state from SimA^B) to compute $z_{i_{\text{zk}}^*}^B$ for the fixed challenge $c_{i_{\text{zk}}^*}^B$.

This approach circumvents the soundness issue previously described. The prover is no longer required to fix all challenges for the simulated part upfront. The dynamic nature of SimZ handles the non-critical rounds, while the secret sharing scheme applied *only* at the critical round i_{zk}^* ensures the binding property, analogous to the 3-move case. Therefore, the prover cannot mix-and-match fixed challenges across rounds to defeat the extractor. Our full construction, Π^{cds} (Figure 2), which we call MR-CDS, formalizes this approach for general access structures represented by monotone span programs.

Security of Composed Critical Round Protocols. While MR-CDS is conceptually as simple as the original CDS, we encounter several issues when analyzing the special soundness. In the case of disjunctive statement $A \vee B$, the high-level goal of the analysis is to determine a $(k'_1, \dots, k'_\mu)$ -tree of transcripts T^{cds} for Π^{cds} in such a way that T^{cds} contains *at least* one $(k_1, \dots, k_\mu)$ -tree T , enabling extraction of witness for either A or B . Since secret-sharing of challenge happens in the critical round i_{zk}^* , Π^{cds} can preserve the parameters for rounds $i > i_{\text{zk}}^*$. In the critical round i_{zk}^* , however, distinct challenges (viewed as a reconstructed secret) only guarantee distinct challenges in either of the shares; for $c_{i_{\text{zk}}^*} = c_{i_{\text{zk}}^*}^A \oplus c_{i_{\text{zk}}^*}^B$ and $\hat{c}_{i_{\text{zk}}^*} = \hat{c}_{i_{\text{zk}}^*}^A \oplus \hat{c}_{i_{\text{zk}}^*}^B$, we have that $c_{i_{\text{zk}}^*}^A \neq \hat{c}_{i_{\text{zk}}^*}^A$ or $c_{i_{\text{zk}}^*}^B \neq \hat{c}_{i_{\text{zk}}^*}^B$. While MR-CDS preserves the special soundness parameter $k'_{i_{\text{zk}}^*} = k_{i_{\text{zk}}^*}$ when $k_{i_{\text{zk}}^*} = 2$, it generally grows exponentially with the number of composed protocols n at the critical round i_{zk}^* if $k_{i_{\text{zk}}^*} > 2$.⁸ This is easy to see by considering the case of 3-way composition, where $c_{i_{\text{zk}}^*}$ is shared as $c_{i_{\text{zk}}^*}^A \oplus c_{i_{\text{zk}}^*}^B \oplus c_{i_{\text{zk}}^*}^C$ and $k_{i_{\text{zk}}^*} = 3$. If a cheating prover

⁸ This may appear similar to the fact that n parallel repetitions of a k -special sound protocol make a $k' = (k-1)^n + 1$ -special sound protocol [AF22]. However, the critical difference here is that the challenge space also grows exponentially due to parallel repetitions, allowing a refined analysis to prove soundness of the resulting protocol. In contrast, the challenge space remains unchanged after CDS composition is applied; therefore, the exponential special soundness parameter in fact translates to an actual security loss.

without the knowledge of witness for any statement fixes $c_{i_{zk}^*}^A, \hat{c}_{i_{zk}^*}^A, c_{i_{zk}^*}^B, \hat{c}_{i_{zk}^*}^B, c_{i_{zk}^*}^C, \hat{c}_{i_{zk}^*}^C$ in advance, it can still respond to $(k_{i_{zk}^*} - 1)^3 = 8$ distinct $c_{i_{zk}^*}$ challenges, translating to $k'_{i_{zk}^*} = 9$. Indeed, the exponential growth of $k'_{i_{zk}^*}$ limits the practical composition of a polynomial number of protocols. To address this limitation, we generalize the Share-then-Hash method proposed in [AAB⁺20] for three-move protocols. Instead of using the secret shares directly as challenges in the critical round for simulated proofs, our multi-round Share-then-Hash (MR-StH) proof (Π^{sth} in Figure 2), computes the challenges as $c_{i_{zk}^*}^j \leftarrow H(\dots, s^j)$ for $j \in [n]$, where $H : \{0, 1\}^* \rightarrow \mathcal{C}_{i_{zk}^*}$ is a collision-resistant hash function, each s^j is a share of the i_{zk}^* -th round challenge $s \in S$ sent by the verifier. The intuition behind MR-StH is that it allows the secret domain S , viewed as a new challenge space, to be chosen exponentially larger than the original challenge space $\mathcal{C}_{i_{zk}^*}$. This suppresses the probability that a cheating prover guesses $s \in S$ such that it matches reconstruction of fixed shares s^j corresponding to fixed distinct challenges $c_{i_{zk}^*}^j$. Soundness analysis for MR-StH again introduces a non-trivial technical challenge. Indeed, the original StH for Σ -protocols [AAB⁺20] is only known to satisfy somewhat non-standard *statistical special soundness*, where witness extraction succeeds with overwhelming probability given a tree of transcripts produced by an adversary responding to uniformly sampled challenges. However, in the multi-round setting, the relationship between this notion and more standard soundness definitions – such as Fiat-Shamir knowledge soundness – remains unclear. To address this, we conduct the analysis of our multi-round StH within the *predicate special soundness* (PSS) framework recently introduced by Aardal et al. [AAB⁺24]. This framework enables a rigorous soundness analysis, bringing the dependence of $k'_{i_{zk}^*}$ down to linear in n , while pushing the exponential term into the numerator of the final knowledge error. The latter can be suppressed by appropriately choosing the size of the share domain S as a denominator. As such, our multi-round StH supports polynomial-size compositions. As [AAB⁺24] shows PSS implies Fiat-Shamir knowledge soundness in the random oracle model, we can then conclude that MR-StH is suitable for practical applications.

Finally, in both MR-CDS and MR-StH, the parameter k'_i for rounds $i < i_{zk}^*$ grows linearly with n , reflecting the possibility that the extractor might encounter different qualified sets in an access structure Γ along different branches of the execution tree of transcripts leading up to the critical round. We highlight that showing this linear loss for an arbitrary monotone access structure Γ is also non-trivial. A naïve analysis would require observing every possible qualified set in the access structure k_i times, leading to an exponential loss in k'_i depending on the size of minimal qualified sets.

We state a simplified version of our result below. For details, we refer the readers to Section 5 and to Appendix B.

Theorem 2 & 3 (simplified). *Let $\{\Pi^j\}_{j \in [n]}$ be $(2\mu + 1)$ -move public-coin proof protocols, each being $\vec{k}$ -SS and CRZK at the same critical round i_{zk}^* . Let Γ be a monotone access structure. MR-CDS composition $\Pi_{\Gamma}^{\text{cds}}$ of $\{\Pi^j\}_{j \in [n]}$ for Γ is $\vec{k}$ -SS and CRZK, where*

1. *For round $i > i_{zk}^*$, $k'_i = k_i$;*
2. *For round $i = i_{zk}^*$, $k'_i = (k_i - 1)^n + 1$;*
3. *For round $i < i_{zk}^*$, $k'_i = n(k_i - 1) + 1$.*

Additionally assuming $H : \{0, 1\}^ \rightarrow \mathcal{C}_{i_{zk}^*}$ is a collision-resistant hash function, MR-StH composition $\Pi_{\Gamma}^{\text{sth}}$ of $\{\Pi^j\}_{j \in [n]}$ for Γ is $(\vec{k}', \Phi)$ -predicate-special-sound where $k'_{i_{zk}^*} = n(k_{i_{zk}^*} - 2) + 2$ and k'_i for $i \neq i_{zk}^*$ are the same as k_i of $\Pi_{\Gamma}^{\text{cds}}$.*

The result on StH applies to composition of protocols with varying critical rounds, special sound parameters, and challenge spaces. If each composed protocol has critical round i_{zk}^j , then one can set $i_{zk}^* = \max_j i_{zk}^j$ and introduce dummy rounds before round i_{zk}^j for each $j \in [n]$; If each protocol has a special soundness parameter k_i^j , then one can set $k_i = \max_j k_i^j$ and consider every protocol $(k_1, \dots, k_\mu)$ -special sound; If each protocol has a challenge space $\mathcal{C}_{i_{zk}^*}^j$, then one can use CRH $H^j : \{0, 1\}^* \rightarrow \mathcal{C}_{i_{zk}^*}^j$ for each $j \in [n]$. Thus, our result allows composition of protocols with diverse structures, as long as each of them has a critical round.

2.3 Trapdoor Commitment

Our second main application is the construction of trapdoor commitment schemes from critical round multi-round protocols. Recall from the introduction the construction of a trapdoor commitment from

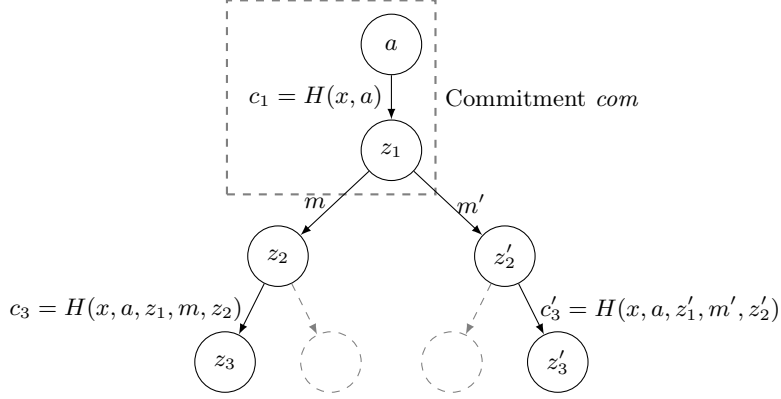


Fig. 1. Visualization of a trapdoor commitment with two openings to m, m' from a three-round protocol that is (2,2)-ExSS and CRZK at round $i_{zk}^* = 2$. Dashed arrows and circles can be obtained via rewinding by programming the random oracle H .

a three-move protocol Π . Now, consider an attempt to generalize this construction to the case when Π is a multi-round protocol which is extended $(k_1, \dots, k_\mu)$ -special sound. The committer must first find a (deterministic) encoding of m into challenges $c_1, \dots, c_\mu$. Then, it runs the zero-knowledge simulator to get a and $z_1, \dots, z_\mu$. In the binding game, the adversary will open the commitment a with distinct messages m and m' , producing two accepting transcripts $(a, c_1, z_1, \dots, c_\mu, z_\mu)$ and $(a, c'_1, z'_1, \dots, c'_\mu, z'_\mu)$ that branch at the first round i such that $c_i \neq c'_i$. To reduce such an adversary to extended $(k_1, \dots, k_\mu)$ -special soundness, we need to build a tree of transcripts that branches in every round. However, this does not seem possible as all challenges are under the control of the adversary. One can try to involve a random oracle in encoding m into $c_1, \dots, c_\mu$ in a Fiat-Shamir style, but this strategy does not seem to help either. The reason is that the random oracle must be programmed in the committing phase and it results in different commitment a .

Trapdoor Commitment from a Critical Round Protocol. Our solution essentially consists of leveraging the critical round framework to be able to use the Fiat-Shamir transformation in a more flexible way. Let Π be CRZK at round i_{zk}^* such that the simulator only needs to know the challenge $c_{i_{zk}^*}$ in advance. Then, we can construct our trapdoor commitment scheme by fixing $m = c_{i_{zk}^*}$ and using Fiat-Shamir to compute the remaining challenges. In more detail, given (x, m) as input, the committer:

- Fixes $c_{i_{zk}^*} = m$ and runs the zero-knowledge simulator of Π to obtain a .
- Computes $c_i = H_i(x, a, z_1, c_1, \dots, z_{i-1})$ for $i \neq i_{zk}^*$, and where z_i is also obtained by running the simulator.
- The commitment is the transcript up to round i_{zk}^* : $com := (a, z_1, \dots, z_{i_{zk}^*-1})$.
- The opening is the remaining part of the transcript, given by $open := (z_{i_{zk}^*}, \dots, z_\mu)$.

To equivocate the commitment, one simply computes the honest proof for a different message $m' \neq m$.

Hiding and equivocability follow from the zero-knowledge property of Π as in the three-move case. For binding, consider an adversary that outputs $com := (a, z_1, \dots, z_{i_{zk}^*-1})$ and openings $open, open'$ to two distinct messages m, m' . Thus, the outputs of the adversary can be seen as two full transcripts with the same prefix up to $z_{i_{zk}^*-1}$, this is,

$$\begin{aligned} & (x, a, c_1, z_1, \dots, z_{i_{zk}^*-1}, m, z_{i_{zk}^*}, \dots, c_\mu, z_\mu) \\ & (x, a, c_1, z_1, \dots, z_{i_{zk}^*-1}, m', z'_{i_{zk}^*}, \dots, c'_\mu, z'_\mu). \end{aligned}$$

In the special soundness language, if Π is $\vec{k}$ -ExSS and $k_{i_{zk}^*} = 2$, this means that the two transcripts form a subtree with a single branch up to round $i_{zk}^* - 1$ and two branches at round i_{zk}^* . We can simply rewind the adversary to obtain the remaining part of the tree of transcripts. To obtain the missing branches, one needs to use a programmable random oracle. We also introduce a simplified statement of our main result below. For details, we refer the readers to Section 6.

Theorem 4 (simplified). *Let Π be a $(2\mu + 1)$ -move public-coin proof protocol for a one-way relation R that is $\vec{k}$ -ExSS and CRZK at round i_{zk}^* where $k_{i_{zk}^*} = 2$. Then, our construction is a trapdoor*

commitment scheme that is hiding and equivocal, and binding in the programmable random oracle model.

In Appendix E, we generalize the above construction to work for $k_{i_{zk}^*} > 2$, although the opening size degrades linearly with $k_{i_{zk}^*}$.

3 Preliminaries

3.1 Notation

Throughout the paper, we use sans-serif for algorithms. Sets are denoted by calligraphic letters. Entities such as a prover, a verifier, and an adversary are denoted by capital letters. Some exceptions may be used for better readability.

We use notation $x \leftarrow \$ \mathcal{D}$ as sampling from a probability distribution $\mathcal{D}$. If $\mathcal{D}$ is a set, then we assume elements from the set are sampled uniformly. A sequence of values from 1 to n (inclusive) is denoted using $[n]$.

3.2 Public-Coin Proof System

We follow the definitions of [ACK21]. Let $R : \mathcal{X} \times \mathcal{W} \rightarrow \{0, 1\}$ be a binary relation defined over instances $\mathcal{X}$ and witnesses $\mathcal{W}$. Language $\mathcal{L}_R$ associated by R is $\mathcal{L}_R := \{x \in \mathcal{X} \mid \exists w \in \mathcal{W} : R(x, w) = 1\}$. By $\mathcal{L}_{RW}$, we denote set $\mathcal{L}_{RW} := \{(x, w) \mid R(x, w) = 1\}$. $\mathcal{L}_R(\lambda)$ ($\mathcal{L}_{RW}(\lambda)$, resp.) denotes a subset of $\mathcal{L}_R$ ($\mathcal{L}_{RW}$, resp.) whose instance x is limited to λ bits. An interactive proof system Π for relation R is a pair of interactive algorithms, prover P and verifier V , where a witness w is given to P as private input and instance x is given to both P and V as common input, and V outputs $b \in \{0, 1\}$ at the end of the execution. By $\langle P(w), V \rangle(x) \rightarrow b$ we denote an execution of the algorithms. A transcript of a protocol execution consists of x , b , and all content of input and output communication tapes of V . It is (perfectly) complete if, for any $(x, w) \in \mathcal{L}_{RW}$, $\langle P(w), V \rangle(x) \rightarrow 1$.

Definition 1 (Public-Coin Proof Protocol). A $(2\mu + 1)$ -move public-coin proof protocol for relation R is a set of probabilistic polynomial-time algorithms A , Z , V and efficiently and uniformly sampleable space $\{\mathcal{C}_i\}_{i \in [\mu]}$ that constitutes an interactive proof system (P, V) that:

- Step 1: P runs $(st_1, a) \leftarrow A(x, w)$ and sends a to V .
- Step $2i$: V uniformly chooses c_i from $\mathcal{C}_i$ and sends it to P .
- Step $2i + 1$: P runs $(st_{i+1}, z_i) \leftarrow Z(st_i, c_i)$ and sends z_i to V .
- Final: V runs $V(x, a, c_1, z_1, \dots, c_\mu, z_\mu) \rightarrow b$ and outputs b .

A transcript $(a, c_1, z_1, \dots, c_\mu, z_\mu)$ with respect to instance x is accepting if $V(x, a, c_1, z_1, \dots, c_\mu, z_\mu) = 1$. The protocol has μ rounds, each of them consisting of a prover message followed by a verifier challenge.

Definition 2 (Tree of Transcripts [ACK21]). A $(k_1, \dots, k_\mu)$ -tree is a directed tree where every node at depth i ($1 \leq i \leq \mu$) has exactly k_i outgoing edges. $(k_1, \dots, k_\mu)$ -tree of transcripts for a $(2\mu + 1)$ -move public-coin proof protocol is a set of $\prod_{i=1}^\mu k_i$ transcripts that can be represented by a $(k_1, \dots, k_\mu)$ -tree in the following manner. Every path of the tree corresponds to a transcript in a way that the i -th response from the prover and the i -th challenge from the verifier is assigned to the i -th node and the i -th edge from the top, respectively. Challenges assigned on a set of edges from a node must be pairwise distinct. A tree of transcripts is called accepting if every transcript in the tree is accepted.

We introduce some useful notations related to the tree of transcript. Let T be a $(k_1, \dots, k_\mu)$ -tree of transcripts and $\tau := (a, c_1, z_1, \dots, c_\mu, z_\mu)$ be a transcript. By $\tau \in T$, we mean that τ is on a path of T . Every node in depth i is indexed by $id := (1, j_1, \dots, j_{i-1}) \in 1 \times [k_1] \times \dots \times [k_{i-1}]$. The root node is at depth 1 and indexed by (1) . As every node except for the root node has only one incoming edge, every edge is represented by the same index as its destination node. By $\text{path}(T, id)$ we denote the path from the root to node id . A partial transcript assigned on $\text{path}(T, id)$ is denoted by $\text{trans}(T, id)$. By $\text{chal}(T, id)$, we denote a set of challenges assigned to outgoing edges from node id . The partial transcript of τ up to the i -th round is denoted by $\text{prefix}(\tau, i) := (a, c_1, z_1, \dots, c_i, z_i)$. Below, we present definitions of soundness and zero-knowledge. Additional basic notations can be found in Appendix C.

Building on $(k_1, \dots, k_\mu)$ -Special Soundness [ACK21], we introduce the notion of *extended special soundness* (ExSS) that captures a broader family of protocols. For an argument system based on hardness assumptions, special soundness is defined with respect to an extended relation. Namely, given a tree T of accepted transcripts, the extended extractor either outputs a witness for the original relation R or provides a solution to a hard problem. We thus have the following definition for the extended special soundness. Let R_{hd} be a relation for a hardness assumption where for instance y and solution s , it outputs $R_{hd}(y, s) = 1$. It is assumed that given y generated by an instance generator, finding s is hard. A particular example is that y is a hash function chosen uniformly from a family of hash functions and $s = (s_1, s_2)$ is a collision that $y(s_1) = y(s_2)$.

Definition 3 ((Extended) $\vec{k}$ -Special Soundness). Let $\mu, k_1, \dots, k_\mu \in \mathbb{N}$ be such that $\prod_{i=1}^\mu k_i$ can be upper bounded by a polynomial. A $(2\mu + 1)$ -move public-coin argument system for relation R is extended $\vec{k}$ -special sound ($\vec{k}$ -ExSS for short) for hardness assumption on R_{hd} if there exists a polynomial time algorithm, ExExt , that, given an instance y and x , an $\vec{k} = (k_1, \dots, k_\mu)$ -tree of accepting transcripts T , outputs (w, s) such that either $R_{hd}(y, s) = 1$ or $R(x, w) = 1$ with probability 1. If R_{hd} is empty, we say that the protocol is $\vec{k}$ -special sound ($\vec{k}$ -SS for short).

Remark 1. For the above definition to be practical, $K = \prod_{i=1}^\mu k_i$ must be of polynomial size, as an efficient extraction algorithm would otherwise be impossible. Accordingly, we assume a polynomial-sized K throughout our paper.

In the three-move case, the concept of special honest verifier zero-knowledge illustrates that having prior access to the challenge is sufficient to simulate the prover without requiring the witness. This idea extends naturally to the multi-round scenario, where having all challenges in advance enables simulation. Formally:

Definition 4 (Special Honest-Verifier Zero-Knowledge). A $(2\mu + 1)$ -round public-coin proof protocol is special honest-verifier zero-knowledge if there exists a polynomial-time algorithm Sim that, for any $(x, w) \in L_{RW}$ and $c_i \in \mathcal{C}_i$ for $i \in [\mu]$, distribution of $(a, c_1, z_1, \dots, c_\mu, z_\mu)$ generated as $(a, z_1, \dots, z_\mu) \leftarrow \text{Sim}(x, c_1, \dots, c_\mu)$ and that of $(a', c_1, z'_1, \dots, c_\mu, z'_\mu)$ generated as $(st_1, a') \leftarrow A(x, w)$ and $(st_{i+1}, z'_i) \leftarrow Z(st_i, c_i)$ for i from 1 to μ are indistinguishable.

3.3 Trapdoor Commitments

Trapdoor commitments are a type of commitment schemes where, if P knows a *trapdoor*, they can open a commitment to any message. Without the knowledge of the trapdoor, however, P can only open the commitment to reveal the originally committed message, preserving binding. In other words, the commitment is only binding if the trapdoor is not known to P . A trapdoor commitment scheme has four properties: *completeness*, *hiding*, *binding*, and *equivocability*. We formalize these properties below, following [CV05].

Definition 5 (Trapdoor Commitment Scheme). A trapdoor commitment scheme is a tuple of five algorithms $\mathcal{TD} = (\text{Gen}, \text{Com}, \text{TCom}, \text{Equiv}, \text{Ver})$ for a message space $\mathcal{M}$ such that:

$\text{Gen}(1^\lambda) \rightarrow (ck, td)$: On input the security parameter λ , $\text{Gen}(1^\lambda)$ returns a commitment key ck and a trapdoor td .

$\text{Com}(ck, m) \rightarrow (com, open)$: On input a commitment key ck and a message $m \in \mathcal{M}$, $\text{Com}(ck, m)$ outputs a commitment com and an opening $open$.

$\text{TCom}(ck, td) \rightarrow (com, st)$: On input a commitment key ck and a trapdoor td , $\text{TCom}(ck, td)$ outputs a commitment com and a state st .

$\text{Equiv}(m, st) \rightarrow open$: On input a message $m \in \mathcal{M}$ and a state st , $\text{Equiv}(m, st)$ outputs an opening $open$.

$\text{Ver}(ck, com, m, open) \rightarrow 0/1$: On input a commitment key ck , a commitment com , a message m , and an opening $open$, $\text{Ver}(ck, com, m, open)$ outputs 1 (accept) or 0 (reject).

Moreover, these algorithms must satisfy the following properties:

Completeness. For any $m \in \mathcal{M}$,

$$\Pr \left[\text{Ver}(ck, com, m, open) = 1 \mid \begin{array}{l} (ck, td) \leftarrow \text{Gen}(1^\lambda) \\ (com, open) \leftarrow \text{Com}(ck, m) \end{array} \right] = 1$$

Binding. A trapdoor commitment scheme is computationally binding if for any PPT adversary $\mathcal{A}$, the following advantage is $\leq \text{negl}(\lambda)$:

$$\text{Adv}_{\mathcal{T}\mathcal{D}, \mathcal{A}}^{\text{bin}}(\lambda) := \Pr \left[\begin{array}{l} \text{Ver}(ck, com, m, open) = 1 \\ \wedge \text{Ver}(ck, com, m', open') = 1 \\ \wedge m \neq m' \end{array} \mid \begin{array}{l} (ck, td) \leftarrow \text{Gen}(1^\lambda) \\ (com, m, open, m', \\ open') \leftarrow \mathcal{A}(ck) \end{array} \right]$$

If the property holds for any (even computationally unbounded) adversary $\mathcal{A}$, then the scheme is statistically binding.

Hiding. A trapdoor commitment is computationally (resp. statistically) hiding if for any PPT (resp. unbounded) adversary $\mathcal{A}$, and for every $m, m' \in \mathcal{M}$,

$$\left| \Pr \left[1 \leftarrow \mathcal{A}(ck, com) \mid \begin{array}{l} (ck, td) \leftarrow \text{Gen}(1^\lambda) \\ (com, open) \leftarrow \text{Com}(ck, m) \end{array} \right] \right. \\ \left. - \Pr \left[1 \leftarrow \mathcal{A}(ck, com) \mid \begin{array}{l} (ck, td) \leftarrow \text{Gen}(1^\lambda) \\ (com, open) \leftarrow \text{Com}(ck, m') \end{array} \right] \right| \leq \text{negl}(\lambda).$$

Equivocability. A trapdoor commitment is computationally (resp. statistically) equivocable if for any $m \in \mathcal{M}$ the following distributions are computationally (resp. statistically) indistinguishable:

$$\left\{ (ck, com, open, m) : \begin{array}{l} (ck, td) \leftarrow \text{Gen}(1^\lambda) \\ (com, open) \leftarrow \text{Com}(ck, m) \end{array} \right\} \quad \text{and} \\ \left\{ (ck, com, open, m) : \begin{array}{l} (ck, td) \leftarrow \text{Gen}(1^\lambda) \\ (com, st) \leftarrow \text{TCom}(ck, td) \\ open \leftarrow \text{Equiv}(st, m) \end{array} \right\}.$$

Next we introduce a property called trapdoor extractability, for which one can extract a valid trapdoor from any adversary that breaks binding. The extractor have oracle access to the adversary, which allows for extraction techniques such as rewinding. It is assumed that there exists a trapdoor-checking function f_t such that $f_t(ck, td) = 1$ if and only if td is a valid trapdoor for ck .

Definition 6 (Trapdoor extractability). A trapdoor commitment scheme is trapdoor extractable with extraction error δ_{tde} if there exists an (expected) polynomial-time extractor Ext_{tde} such that for any PPT adversary $\mathcal{A}$, any $(ck, td) \leftarrow \text{Gen}(1^\lambda)$,

$$\Pr[td' \leftarrow \text{Ext}_{\text{tde}}^{\mathcal{A}(\cdot)}(ck) : f_t(ck, td') = 1] \geq \frac{\text{Adv}_{\mathcal{T}\mathcal{D}, \mathcal{A}}^{\text{bin}}(\lambda) - \delta_{\text{tde}}(\lambda)}{\text{poly}(\lambda)},$$

where $\text{Adv}_{\mathcal{T}\mathcal{D}, \mathcal{A}}^{\text{bin}}(\lambda)$ is defined as in the binding property.

4 Critical-Round Zero-Knowledge

4.1 Definitions

Recall that the most general form of multi-round zero-knowledge simulator as shown in Definition 4 takes all challenges $(c_1, \dots, c_\mu)$ as input. Here, we formalize a stronger notion where the entire simulation is possible by knowing only the challenge in the critical round. In Appendix C.1, we discuss how our CRZK definition is related to other notions such as k -zero knowledge [GKK⁺22, DG23] and k -special unsoundness [AFK23, BGT25].

Definition 7 (Critical-Round (Special) Honest Verifier Zero-Knowledge). A $(2\mu + 1)$ -move public-coin proof protocol is critical-round special honest verifier zero-knowledge (CRSHVZK for short) at round i_{zk}^* if there exists a set of polynomial-time algorithm $(\text{SimA}, \text{SimZ})$ that:

- **SimA** takes x and $c_{i_{zk}^*} \in \mathcal{C}_{i_{zk}^*}$ as input, and outputs (st_1, a) where st_1 is a state information and a is a prover's message at step 1.
- **SimZ** takes st_i and $c_i \in \mathcal{C}_i$, and outputs (st_{i+1}, z_i) where st_{i+1} is an updated state, and z_i is a prover's response for step $2i + 1$.
- For any $(x, w) \in L_{RW}$ and $c_i \in \mathcal{C}_i$ for $i \in [\mu]$, distribution of $(a, c_1, z_1, \dots, c_\mu, z_\mu)$ generated as $(st_1, a) \leftarrow \text{SimA}(x, c_{i_{zk}^*})$ and $(st_{i+1}, z_i) \leftarrow \text{SimZ}(st_i, c_i)$ for i from 1 to μ , and that of $(a', c_1, z'_1, \dots, c_\mu, z'_\mu)$ generated as $(st_1, a') \leftarrow A(x, w)$ and $(st_{i+1}, z'_i) \leftarrow Z(st_i, c_i)$ for i from 1 to μ are indistinguishable.

As a weaker variant, if in the above definition **SimA** only takes x as input, and outputs $(st_1, a, c_{i_{zk}^*})$, we call it *critical-round honest verifier zero-knowledge* (**CRHVZK** for short) at round i_{zk}^* .

4.2 Instantiations of Critical Round Proofs

MPC-in-the-Head based Proofs. As an illustrative toy example, we take the honest-verifier zero-knowledge proof in [KKW18] (denoted by KKW) and formally prove that their 5-move protocol satisfies CRZK and ExSS. We note that many other multi-round protocols that follow the MPCitH with preprocessing paradigm also fall into this class, including (but not limited to) [KKW18, BN20, KZ22, FJR22].

In the KKW framework (detailed description is available in Appendix D), the prover runs an MPC protocol that evaluates a boolean circuit C on an input w (witness), commits to the views of all parties, and then opens all-but-one of these views to the verifier. This MPC protocol is: (1) secure against semi-honest all-but-one corruptions, hence there exists a simulator Sim_p that outputs simulated consistent views of $n - 1$ parties, and (2) executed in a *deterministic* manner by using a preprocessing material sampled *independently* of the witness w . To prevent the prover from cheating in the preprocessing phase, the prover follows the cut-and-choose paradigm, i.e., first generates and commits to m executions of the preprocessing stage, and later opens all of them except one (corresponding to a verifier's challenge). The unopened material is used for executing the MPC protocol later on.

In Theorem 1, we show that KKW is flexible and can be analyzed as CRZK at either $i_{zk}^* = 1$ or $i_{zk}^* = 2$. The formal proof is shown in Theorem 7.

Theorem 1. *Given the 5-move interactive honest-verifier zero-knowledge proof in [KKW18] (denoted as KKW), assuming that the hash function used is collision-resistant and the commitment scheme used is computationally binding and hiding, then KKW is flexible and can be:*

- *Critical-round special honest-verifier zero-knowledge at round $i_{zk}^* = 1$ or at round $i_{zk}^* = 2$.*
- *Extended $\vec{k}$ -special sound where $\vec{k} = (2, 2)$.*

Let (a, c_1, z_1, c_2, z_2) be a transcript of the KKW protocol. We provide the intuition behind the formal proof (in Theorem 7) of the flexibility property of KKW with respect to CRZK.

CRZK. We prove that KKW is CRZK at either $i_{zk}^* = 1$ or $i_{zk}^* = 2$ by constructing two simulators (**SimA**, **SimZ**) that simulate the real execution.

- For $i_{zk}^* = 1$, $\text{SimA}(x, c_1) \rightarrow (st_1, a)$ takes the first-round challenge c_1 as input. For $i_{zk}^* = 2$, $\text{SimA}(x, c_2) \rightarrow (st_1, a)$ instead takes the second-round challenge c_2 as input. In both cases, **SimZ** uses **SimA**'s output along with c_1 and c_2 to complete the simulation of the real transcripts.
- The construction of **SimA** also differs in each case. For $i_{zk}^* = 1$, since **SimA** knows the challenge c_1 , it simulates the c_1 -th preprocessing step by emulating the auxiliary values aux to ensure the correctness of the MPC output. On the other hand, for $i_{zk}^* = 2$, **SimA** knows the index of all-but-one party in the MPC protocol, allowing it to use the MPC simulator to simulate the views of the remaining party.

IOP-based Proofs. For SNARKs based on IOPs [RRR16, BCS16] and polynomial IOPs [BFS20], the proof of knowledge soundness does not involve building a tree of transcripts as in the KKW framework. Rather, it often relies on the extractability of a polynomial commitment scheme, which is usually proven in the algebraic group model or relies on knowledge assumptions where no rewinding of the adversary is required. In these cases, the knowledge extractor can be naturally adjusted to the algebraic group model (AGM) or to extra assumptions such that it is given additional auxiliary information (e.g., the algebraic representations of all group elements) extracted from the adversary. The shape of the extended tree of transcripts T for extended $\vec{k}$ -special soundness is related to the polynomial evaluation checks and batching operations which usually rely on the statistical soundness of the Schwartz-Zippel lemma. To avoid reliance on the AGM, in some cases, one can analyze the interactive proof systems underlying these SNARKs in the standard model.

Plonk. Plonk [GWC19] is a SNARK that is built by compiling a multi-round polynomial IOP via the Fiat-Shamir transform, where the polynomial oracles are instantiated with the KZG polynomial commitment scheme [KZG10]. At a high level, the prover first commits to several polynomials encoding the witness values, and then proves circuit satisfiability via multiple polynomial checks. In the standard model, a recent work by Lipmaa, Parisella, and Siim [LPS25] characterizes Plonk as a computationally (i.e., extended) $\vec{k}$ -special sound five-round protocol.⁹ Concretely, they obtain that for an upper bound of n circuit gates, Plonk is extended $\vec{k}$ -special sound for $\vec{k} = (3n+1, 3n+1, 3, 4n+21, 6)$. For CRZK, a recent result from Lipmaa [Lip25, Theorem 5] proves that Plonk is trapdoorless zero-knowledge using a simulator that only requires to program the third round of five-round Plonk. This implies that Plonk is CRZK at round $i_{zk}^* = 3$.

Proposition 1. *The 5-round Plonk protocol as described in [LPS25] satisfies $\vec{k}$ -ExSS in the standard model, where $\vec{k} = (3n+1, 3n+1, 3, 4n+21, 6)$. Moreover, Plonk satisfies critical-round zero knowledge at round $i_{zk}^* = 3$.*

Bulletproofs and Compressed Σ -protocols. Another prominent example of a multi-round proof is Bulletproofs [BCC⁺16, BBB⁺18], a widely used transparent proof system secure under the discrete logarithm assumption. In Bulletproofs, the prover commits to a length d witness and then reduces the proving statement to another one of length $d/2$. After $\log d$ rounds, the prover opens the commitments and the verifier can check the satisfiability of the relation.

For CRZK, [DG23] proves the 3-ZK notion [GKK⁺22] of the protocols¹⁰. As their simulator works by obtaining the third round challenge x as input and then the following rounds can be simulated sequentially, their analysis directly implies CRZK at round $i_{zk}^* = 3$. As such, the following statements hold:

Proposition 2. *Bulletproofs for aggregate range proof (BP-ARP) [BBB⁺18] is a $(2\mu+1)$ -move protocol where $\mu = 4 + \lceil \log(\ell d) \rceil$, ℓ is the number of aggregated commitments, and d is the maximum bit length of the committed values. It is $\vec{k}$ -special sound, where $\vec{k} = (\ell d, \ell+2, 3, 2, 7, \dots, 7)$. Moreover, BP-ARP is CRZK at round $i_{zk}^* = 3$ from [DG23, Lemma 6.4].*

Proposition 3. *Bulletproofs for arithmetic circuit satisfiability proof (BP-ACS) is a $(2\mu+1)$ -move protocol where $\mu = 4 + \lceil \log(\ell) \rceil$ and ℓ is the number of multiplication gates in the circuit. It is $\vec{k}$ -special sound, where $\vec{k} = (\ell, Q+1, 7, 2, 7, \dots, 7)$, and Q is the number of linear constraints. Moreover, BP-ACS is CRZK at round $i_{zk}^* = 3$ from [DG23, Lemma C.2].*

A close variant of Bulletproofs is compressed Σ -protocols [AF22]. As their special HVZK simulator only requires knowing the first round challenge in advance and then it can produce a transcript honestly while receiving the following challenges, we observe that $i_{zk}^* = 1$. We thus get the following statement:

⁹ We note that Plonk is usually regarded as a four-round protocol, but the first round is divided into two in their analysis.

¹⁰ In [DG23], they call it 2-ZK grouping the first two challenges y, z together as a single round. In this work, we view them as challenges from two separate rounds so that the number of rounds matches the dimension of the special soundness parameter $\vec{k}$.

Proposition 4. *The compressed Σ -protocols for discrete logarithm relation [AC20] are $(2\mu+1)$ -move protocols where $\mu = \lceil \log(n+1) \rceil + 1$ and n is the number of group generators. It is $\vec{k}$ -special sound, where $\vec{k} = (2, 2, 3, \dots, 3)$. Moreover, it is CRZK at round $i_{zk}^* = 1$ from [AC20, Theorem 3].*

5 Multi-Round Proof of Partial Knowledge

5.1 Construction

In this section, our goal is to build a proof system for a logically composed relation using proof systems for atomic relations in a black-box manner. We assume the reader is familiar with the concept of monotone access structures (Appendix A.1) and semi-smooth secret sharing schemes (Appendix A.2). These relations can be logically composed using a monotone access structure Γ . That is, given n public-coin protocols Π^j for relation R^j , we define a composed protocol Π_F^{cds} for relation $R_F((x^1, \dots, x^n), (w^1, \dots, w^n))$ that outputs 1 if and only if $\exists A \in \Gamma, \forall j \in A, R^j(x^j, w^j) = 1$. Thus, the prover will know valid witnesses for some of the relations (generating qualified transcripts) and will need to simulate the remaining non-qualified transcripts. In brief, given multi-round proofs that are special sound and CRSHVZK at critical round $i_{zk}^* = i^*$, the prover in our composition follows the 3-move CDS composition framework as below:

- For steps $2i+1$ ($i \neq i^*$), the prover simulates the non-qualified transcripts and honestly generates the qualified ones based on the challenge sent by the verifier at step $2i$.
- For step $2i^*+1$ at the critical round, the prover samples the critical-round challenges of non-qualified transcripts in advance. After receiving the challenge from the verifier, the prover completes the remaining challenges for the qualified transcripts via the share completion algorithm of SSS_{Γ^*} .

Our MR-CDS composition, Π_F^{cds} , is presented in Fig. 2. The private input to the prover algorithm is a set of witnesses $\{w^j\}_{j \in A}$ that $A \in \Gamma$. The ingredients of our protocol include a semi-smooth secret sharing scheme SSS_{Γ^*} over secret domain S and n CRSHVZK protocols $\{\Pi^j\}_{j \in [n]}$ at critical round i^* . Note that efficient SSS_{Γ^*} exists for any Γ^* (and thus Γ) that is recognized by monotone span program [CDM00]. For simplicity, we assume that the share domain of SSS_{Γ^*} matches the challenge space $\mathcal{C}_{i^*}$ for Π_F^{cds} , while this restriction becomes superfluous for our MR-StH composition, Π_F^{sth} , presented in Section 5.3. Following [CDS94], if challenges are longer than shares, one can simply take the first appropriate number of bits of the challenge to be the corresponding share. The completion process can be also postprocessed by simply copying the bits of the shares and padding with random bits to define the corresponding challenges.

5.2 Security

To illustrate the intuition, we first informally discuss the security of Π_F^{cds} when $i^* = i_{zk}^* = 1$ and $k_{i^*} = 2$. Completeness of Π_F^{cds} follows by inspection: since Π^j for $j \in [n]$ are complete and CRSHVZK, V^j always accepts. Moreover, the share completion property of SSS_{Γ^*} guarantees $\text{CheckShares}_{\Gamma^*}$ always outputs 1. CRSHVZK of Π_F^{cds} is also straightforward. One can construct a simulator, which, instead of running an honest prover algorithm, executes CRHVZK simulators for all $j \in [n]$ on input $\{c_{i^*}^j\}_{j \in [n]} = \{s^j\}_{j \in [n]} \leftarrow \text{Share}_{\Gamma^*}(s)$. The perfect hiding and share completion properties of SSS_{Γ^*} guarantee that $\{s^j\}_{j \in [n]}$ is identically distributed in real and simulated transcripts.

To prove $\vec{k}'$ -SS of Π_F^{cds} from $\vec{k}$ -SS of Π^j , we need to carefully set the parameters $\vec{k}'$. Since any $\vec{k}'$ -tree of transcripts for Π_F^{cds} can be decomposed into n individual trees T^j for Π^j , we can conclude the existence of a valid extractor for Π_F^{cds} by showing the existence of some $A \in \Gamma$ such that for all $j \in A$, T^j contains a $\vec{k}$ -tree of accepting transcripts. We split the analysis of k'_i into two cases: 1) $i > i^*$, and 2) $i = i^*$. In Case 1, as all T^j for $j \in [n]$ have identical challenges in the corresponding edges in depth i , we can simply set $k'_i = k_i$. Case 2 boils down to showing that two distinct critical-round challenges s and $\hat{s}$ (used as secrets of SSS_{Γ^*}) result in a subset of decomposed trees $\{T^j\}_{j \in A}$ for some $A \in \Gamma$, where each T^j has distinct challenges $c_{i^*}^j$ and $\hat{c}_{i^*}^j$ in the corresponding edges in depth i^* . This follows from the “consistency testing” property of SSS_{Γ^*} (Appendix A.2) and the analysis is essentially identical to [CDS94].

Composition Compiler Π_F^{sth} and Π_F^{cds} . Statement: $\vec{x} = \{x^j\}_{j \in [n]}$. Witness: $\{w^j\}_{j \in A}$ such that $A \in \Gamma$ and $R^j(x^j, w^j) = 1, \forall j \in A$.
Step 1 : Prover computes as follows: – Sample $\{s^j\}_{j \in \bar{A}} \leftarrow D_{\bar{A}}$. – for all $j \in \bar{A}$: $\text{Compute } c_{i^*}^j = H(\Gamma, \vec{x}, j, s^j);$ $\text{Set } c_{i^*}^j = s^j;$ $\text{Simulate } (st_1^j, a^j) \leftarrow \text{SimA}^j(x^j, c_{i^*}^j).$ – for all $j \in A$: $\text{Compute } (st_1^j, a^j) \leftarrow A^j(x^j, w^j; r^j).$ – Send $\{a^j\}_{j \in [n]}$ to the verifier. Repeat the following for $i = 1, \dots, \mu$: Step 2i ($i \neq i^*$) : Verifier samples $c_i \leftarrow \mathcal{C}_i$ and sends c_i to the prover. Step 2i* : Verifier samples $s \leftarrow \mathcal{S}$ and sends s to the prover. Step 2i + 1 ($i \neq i^*$) : Prover computes as follows: – for all $j \in \bar{A}$: $\text{Compute } (st_{i+1}^j, z_i^j) \leftarrow \text{SimZ}^j(st_i^j, c_i).$ – for all $j \in A$: $\text{Compute } (st_{i+1}^j, z_i^j) \leftarrow Z^j(st_i^j, c_i).$ – Send $\{z_i^j\}_{j \in [n]}$ to the verifier. Step 2i* + 1 : Prover computes as follows: – Compute $\{s^j\}_{j \in A} \leftarrow \text{Complete}_{\Gamma^*}(s, \{s^j\}_{j \in \bar{A}}).$ – for all $j \in \bar{A}$: $\text{Compute } (st_{i^*+1}^j, z_{i^*}^j) \leftarrow \text{SimZ}^j(st_{i^*}^j, c_{i^*}^j).$ – for all $j \in A$: $\text{Compute } c_{i^*}^j = H(\Gamma, \vec{x}, j, s^j);$ $\text{Set } c_{i^*}^j = s^j;$ $\text{Compute } (st_{i^*+1}^j, z_{i^*}^j) \leftarrow Z^j(st_{i^*}^j, c_{i^*}^j).$ – Send $\{z_{i^*}^j, s^j\}_{j \in [n]}$ to the verifier. Verification : Verifier computes as follows: – for all $j \in [n]$: $\text{Compute } c_{i^*}^j = H(\Gamma, \vec{x}, j, s^j);$ $\text{Set } c_{i^*}^j = s^j;$ – for all $j \in [n]$: assert $\forall^j(x^j, \pi^j = (a_1^j, c_1^j, z_1^j, \dots, c_\mu^j, z_\mu^j)) = 1,$ where $c_i^j := c_i$ for $i \neq i^*$; – assert $\text{CheckShares}_{\Gamma^*}(s, \{s^j\}_{j \in [n]}) = 1.$

Fig. 2. Our multi-round composition Π_F^{cds} (MR-CDS) and Π_F^{sth} (MR-StH) with critical round $i^* = i_{\text{zk}}^*$. The code highlighted in solid box is only executed by Π_F^{cds} , while the code in dashed box is only executed by Π_F^{sth} , respectively. Π_F^{cds} assumes $\mathcal{C}_{i^*}$ to be identical to the share space of the secret sharing scheme SSS_{Γ^*} . Π_F^{sth} relies on a collision-resistant hash $H : \{0, 1\}^* \rightarrow \mathcal{C}_{i^*}$.

Although the analysis of the above cases is rather simple, we will encounter some subtleties when it comes to more general cases.

Extension $k_{i^*} > 2$: First, we need to set $k'_{i^*} = (k_{i^*} - 1)^n + 1$ to retain the analysis of Case 2 when $k_{i^*} > 2$. This parameter adjustment is tight; with a set of $k_{i^*} - 1$ pairwise distinct challenges for each $j \in [n]$, there can be at most $(k_{i^*} - 1)^n$ pairwise distinct reconstructed secrets corresponding to the challenges of the composed protocol Π_F^{cds} . Consequently, to obtain k_{i^*} pairwise distinct challenges for *some* j , we must set $k'_{i^*} > (k_{i^*} - 1)^n$. As this translates to the knowledge error including an additive term $(k_{i^*} - 1)^n / |S|$, Π_F^{cds} supports composition of a constant number of instances if the secret domain S coincides with the share domain $\mathcal{C}_{i^*}$. We address this limitation in Section 5.3.

Extension $i^* = i_{\text{zk}}^* > 1$: In the composition illustrated in Fig. 2, the critical round i^* refers specifically to the zero-knowledge critical round i_{zk}^* relative to the CRSHVZK simulator. We show an arbitrary choice of i_{zk}^* is possible, while a careful analysis of SS parameters $\vec{k}'$ is necessary. If $i^* > 1$, the branches in round $i < i^*$ may obtain potentially different qualified sets $A_1, \dots, A_{k'_i} \in \Gamma$ defined by k'_i sub-trees. To collect k_i pairwise distinct challenges for *all* j in a fixed qualified set A , one might

naively set $k'_i = |\Gamma| \cdot k_i$ to account for every potential qualified set k_i times. However, through a more refined analysis, it can be shown that $k'_i = nk_i$ for all $i < i^*$ are sufficient.

In Theorem 2, we present the security of $\Pi_{\Gamma}^{\text{cds}}$ in its most general form, encompassing all the above extensions. We then introduce a series of remarks, followed by the theorem proof.

Theorem 2 (MR-CDS Composition: $\Pi_{\Gamma}^{\text{cds}}$). *For each $j \in [n]$, consider a $(2\mu + 1)$ -move public-coin proof protocol Π^j for relation R^j . If SSS_{Γ^*} is a semi-smooth perfect secret sharing scheme over domain S for access structure Γ^* , and every Π^j is $\vec{k}$ -SS and CRSHVZK at round $i^* = i_{\text{zk}}^*$, then, protocol $\Pi_{\Gamma}^{\text{cds}}$ is a $(2\mu + 1)$ -move proof system for relation $R_{\Gamma}((x^1, \dots, x^n), (w^1, \dots, w^n))$ that outputs 1 if and only if $\exists A \in \Gamma, \forall j \in A, R^j(x^j, w^j) = 1$. It is complete, $\vec{k}'$ -SS and CRSHVZK at round i^* , where*

1. For $i > i^*$, $k'_i = k_i$;
2. For $i = i^*$, $k'_i = (k_i - 1)^n + 1$;
3. For $i < i^*$, $k'_i = n(k_i - 1) + 1$.

Remark 2. This theorem gives us an interesting insight. When $k_i^* = 2$ and $i^* = 1$, we have $k'_i = k_i$ for all i , i.e., the shape of the tree preserves. Thus MR-CDS is closed for such protocols. This observation suggests that, for the purpose of composition, it is advantageous to consider the KKW protocol with $i^* = 1$ rather than $i^* = 2$ (see Theorem 1 for the critical-round flexibility of KKW). Likewise, the compressed Σ -protocols has $k_i^* = 2$ and $i^* = 1$ (Proposition 4), preserving the special soundness parameter if compiled with MR-CDS. By Propositions 2 and 3, MR-CDS applied to the BP-ARP and BP-ACS protocols has CRSHVZK at round $i_{\text{zk}}^* = 3$, and they thus obtain degraded parameters $\vec{k}' = (n\ell d, n(\ell + 2), 2^n + 1, 2, 7, \dots, 7)$ and $\vec{k} = (n\ell, n(Q + 1), 6^n + 1, 2, 7, \dots, 7)$, respectively. MR-StH can compensate for this loss with a suitable choice of the secret space S . As mentioned in Section 2.2, it is also possible to compose different protocols, e.g., KKW and BP-ACS, BP-ARP and compressed Σ -protocol, and so on.

Remark 3. Theorem 2 is also useful even if each protocol is Extended $\vec{k}$ -Special Sound. The protocol Π^j for relation R^j with $\vec{k}$ -ExSS on hardness assumption R_{hd}^j can be viewed as a $\vec{k}$ -SS protocol for relation $\bar{R}^j$ that outputs 1 if and only if $R^j(x, w) = 1 \vee R_{\text{hd}}^j(y, u) = 1$. Therefore, we obtain a composed protocol for relation $\bar{R}_{\Gamma}((x^1, y^1, \dots, x^n, y^n), (w^1, u^1, \dots, w^n, u^n))$ that outputs 1 if and only if $\exists A \in \Gamma, \forall j \in A, \bar{R}^j((x^j, y^j), (w^j, u^j)) = 1$. This essentially means that for every instance $j \in A$, a cheating prover must know either witness w^j or a solution u^j to the hard problem. Assuming that every R_{hd}^j is computationally hard relation, one can then conclude the argument of knowledge of $\Pi_{\Gamma}^{\text{cds}}$ for relation R_{Γ} . The same remark applies to MR-StH composition $\Pi_{\Gamma}^{\text{sth}}$.

We prove Theorem 2 below.

Proof. Completeness. It directly follows from the completeness and critical-round special honest verifier zero-knowledge of Π^j . We note that $\text{Complete}_{\Gamma^*}$ works for the set $\bar{A}$ of shares $\{s^j\}_{j \in \bar{A}}$ since $\bar{A} \notin \Gamma^*$ when $A \in \Gamma$.

Critical-Round Special Honest Verifier Zero-Knowledge. Consider the following simulator for $\Pi_{\Gamma}^{\text{cds}}$ that proceeds as follows, given as input $\{x^j\}_{j \in [n]} \in \mathcal{L}_{R_{\Gamma}}, \{c_i\}_{i \neq i^*}$ and s :

1. Compute $\{c_{i^*}^j\}_{j \in [n]} = \{s^j\}_{j \in [n]} \leftarrow \text{Share}_{\Gamma^*}(s)$
2. For all $j \in [n]$: $(st_1^j, a^j) \leftarrow \text{SimA}^j(x^j, c_{i^*}^j)$
3. For all $i \in [\mu - 1]$ and $j \in [n]$: $(st_{i+1}^j, z_i^j) \leftarrow \text{SimZ}^j(st_i^j, c_i^j)$, where $c_i^j = c_i$ if $i \neq i^*$
4. Output $(\{a^j\}_{j \in [n]}, \{c_i\}_{i \in [\mu] \setminus \{i^*\}}, \{z_i^j\}_{i \in [\mu], j \in [n]}, s, \{s^j\}_{j \in [n]})$

The perfect hiding and share completion properties of SSS_{Γ^*} imply that the distribution of $\{s^j\}_{j \in [n]}$ generated as above is identical to that of a real transcript. Since $\{x^j\}_{j \in [n]} \in \mathcal{L}_{R_{\Gamma}}$, for $j \in A$, the simulator generates a^j, z_i^j exactly as a real prover; for $j \in \bar{A}$, the distribution of a^j, z_i^j output by CRSHVZK simulators for Π^j is indistinguishable with the one generated by a real prover. Grouping Step 1-2 together as SimA and Step 3 as SimZ, respectively, the above algorithm is indeed a valid CRSHVZK simulator.

As in [CDS94], if SSS_{Γ^*} additionally has smoothness, one can obtain a variant of Theorem 2 in which CRSHVZK is replaced by non-special CRHVZK. To do so, we can modify **Step 1** of Fig. 2 so

that prover instead samples independent and uniformly random shares $s^j = c_{i^*}^j$ for $j \in \bar{A}$. Clearly, this modification allows the invocation of non-special simulators for Π^j as they output uniformly random challenges in the critical round i^* . Note that the resulting simulator for $\Pi_{\Gamma}^{\text{cds}}$ is also non-special because all challenges in round i^* must be sampled internally by the underlying non-special simulators for Π^j .

Special Soundness. The proof can be viewed as a generalization of [CDS94]. However, we must carefully choose the special soundness parameters for $\Pi_{\Gamma}^{\text{cds}}$ in such a way that the corresponding tree of accepting transcripts contains a well-structured tree for every $j \in A \in \Gamma$ from which the extractor Ext^j for Π^j can successfully extract witness for x^j . We construct an efficient extractor Ext^{cds} that determines some $A \in \Gamma$ and then internally runs sub-extractors $\{\text{Ext}^j\}_{j \in A}$ to output a valid set of witnesses $\mathbf{w} = \{w^j\}_{j \in A}$, given as input any $\bar{k}'$ -tree T^{cds} of accepting transcripts.

We first recall the structure of the tree T^{cds} for $\Pi_{\Gamma}^{\text{cds}}$. Each path in T^{cds} defines a transcript

$$(\{a^j\}_{j \in [n]}, \{c_i\}_{i \in [\mu] \setminus \{i^*\}}, \{z_i^j\}_{i \in [\mu], j \in [n]}, s, \{s^j\}_{j \in [n]})$$

which can be decomposed into n individual transcripts:

$$\{(a^j, \{c_i\}_{i \in [\mu] \setminus \{i^*\}}, c_{i^*}^j, \{z_i^j\}_{i \in [\mu]})\}_{j \in [n]}$$

where $c_{i^*}^j = s^j$ and $c_i^j = s^j$ for $j \in [n]$. Thus, T^{cds} can also be decomposed into n individual trees, $T^1, \dots, T^n$, where every path of T^j is derived from decomposition of the corresponding path in T^{cds} . We first observe the following properties:

1. In non-critical rounds $i \neq i^*$, the challenges in the decomposed trees T^j for $j \in [n]$ remain the same as the corresponding challenges in T^{cds} .
2. In round i^* , the shares s^j in the decomposed trees T^j for $j \in [n]$ pass the consistency check performed by $\text{CheckShares}_{\Gamma^*}$ with respect to the corresponding challenge s in T^{cds} .

Case 1: $i^* < i \leq \mu$. We first prove the following statement: if $i^* < i \leq \mu$, then for every $j \in [n]$, the corresponding decomposed tree T^j has k_i pairwise distinct challenges at round i . This is straightforward: as every set of edges in depth i of T^{cds} are labeled by k_i pairwise distinct challenges, the corresponding edges in T^j have the same property.

Case 2: $i = i^*$. Next, we prove the following statement: if $k_{i^*}' = (k_{i^*} - 1)^n + 1$, then for any $(1, \dots, 1, k_{i^*}', k_{i^*}'+1, \dots, k_{\mu})$ -subtree $T_{i^*}^{\text{cds}}$ for $\Pi_{\Gamma}^{\text{cds}}$, there exists $A \in \Gamma$ such that for every $j \in A$, the corresponding decomposed subtree $T_{i^*}^j$ contains a $(1, \dots, 1, k_{i^*}', \dots, k_{\mu})$ -subtree $\tilde{T}_{i^*}^j$ for statement x^j . We first denote by s_{ℓ} the ℓ th challenge for round i^* in $T_{i^*}^{\text{cds}}$, where s_{ℓ} for $\ell \in [k_{i^*}']$ are pairwise distinct. Moreover, $\{s_{\ell}^j\}_{j \in [n]}$ denotes a set of shares reconstructing to s_{ℓ} in $T_{i^*}^{\text{cds}}$. Then we have that $\text{CheckShares}_{\Gamma^*}(s_{\ell}, \{s_{\ell}^j\}_{j \in [n]}) = 1$ for every $\ell \in [k_{i^*}']$. We show that, for every $B \in \Gamma^*$, there exists $j \in B$ such that at least k_{i^*}' elements of $\{s_{\ell}^j\}_{\ell \in [k_{i^*}']}$ are pairwise distinct. Assume for contradiction that there exists $\tilde{B} \in \Gamma^*$ such that for every $j \in \tilde{B}$ at most $k_{i^*}' - 1$ elements of $\{s_{\ell}^j\}_{\ell \in [k_{i^*}]}$ are pairwise distinct. By the consistency testing property of SSS_{Γ^*} (see Appendix A.2), however, it must be that $\text{Rec}_{\Gamma^*}(\{s_{\ell}^j\}_{j \in \tilde{B}}) = s_{\ell}$ for every $\ell \in [k_{i^*}']$. As the output of Rec_{Γ^*} can take at most $(k_{i^*}' - 1)^{|\tilde{B}|}$ distinct values and $|\tilde{B}| \leq n$, it contradicts the fact that $\{s_{\ell}\}_{\ell \in [k_{i^*}]}$ are pairwise distinct when $k_{i^*}' = (k_{i^*} - 1)^n + 1$.

Finally, let $A = \bigcup_{B \in \Gamma^*} J_B$, where J_B denotes a set of $j \in B$ such that at least k_{i^*}' elements of $\{s_{\ell}^j\}_{\ell \in [k_{i^*}]}$ are pairwise distinct. As each J_B is guaranteed to be non-empty, we have that $A \cap B \neq \emptyset$ for every $B \in \Gamma^*$, which implies $A \in \Gamma$ by Proposition 5. Thus, for each $j \in A \in \Gamma$, one can derive from $T_{i^*}^j$ a $(1, \dots, 1, k_{i^*}', \dots, k_{\mu})$ -subtree $\tilde{T}_{i^*}^j$ of accepting transcripts for statement x^j .

Case 3: $i < i^*$. We prove the following statement: if $k_i' = n(k_i - 1) + 1$ for $i < i^*$, $k_{i^*}' = (k_{i^*} - 1)^n + 1$, then for any $(1, \dots, 1, k_i', \dots, k_{i^*}', k_{i^*}'+1, \dots, k_{\mu})$ -subtree T_i^{cds} for $\Pi_{\Gamma}^{\text{cds}}$, there exists $A \in \Gamma$ such that for every $j \in A$, the corresponding decomposed subtree T_i^j contains a $(1, \dots, 1, k_i, \dots, k_{i^*}, k_{i^*}'+1, \dots, k_{\mu})$ -subtree $\tilde{T}_i^j$ for statement x^j . Before delving into details, we first sketch the proof strategy for $i = i^* - 1$ (the proof for $i < i^* - 1$ can be made iteratively). By the analysis of Case 2, we have that, for each $\ell \in [k_{i^*}']$, the ℓ -th $(1, \dots, 1, k_{i^*}', k_{i^*}'+1, \dots, k_{\mu})$ -subtree determines some access structure $A_{\ell} \in \Gamma$. As A_{ℓ} 's might be potentially distinct from each other, to obtain a well-structured upper-level subtree, we need to determine some $\hat{\ell} \in [k_{i^*}']$ in such a way that for every $j \in A_{\hat{\ell}}$, there are at least $k_{i^*}-1$

subtrees that determine access structures having j as a member. We stress that the extracted subtrees $\{\tilde{T}_{i^*-1}^j\}_{j \in A_\ell}$ as a result may potentially contain different sets of level $i^* - 1$ challenges for each j . For instance, if $A_\ell = \{1, 2\}$ and $k_{i^*-1} = 2$, then it could be that $\tilde{T}_{i^*-1}^1$ for statement x^1 has its level $i^* - 1$ challenges labeled by c_1 and c_2 , while $\tilde{T}_{i^*-1}^2$ for statement x^2 has the challenges labeled by c_3 and c_4 , respectively.

More formally, we first denote by c_ℓ the ℓ th challenge for round $i = i^* - 1$ in T_i^{cds} for $\ell \in [k'_i]$.¹¹ Then for each $\ell \in [k'_i]$, T_i^{cds} defines the corresponding $(1, \dots, 1, k'_{i^*}, k_{i^*+1}, \dots, k_\mu)$ -subtree $T_{i^*, \ell}^{\text{cds}}$ of T_i^{cds} . By the analysis of Case 2, $T_{i^*, \ell}^{\text{cds}}$ defines some qualified set $A_\ell \in \Gamma$ such that for every $j \in A_\ell$, the corresponding decomposed subtree $T_{i^*, \ell}^j$ contains a $(1, \dots, 1, k_{i^*}, \dots, k_\mu)$ -subtree $\tilde{T}_{i^*, \ell}^j$ for statement x^j . Now we show that there exists $\hat{\ell} \in [k'_i]$ such that, for all $j \in A_{\hat{\ell}}$, the number of A_ℓ containing j is at least k_i . That is, let $L_j = \{\ell \in [k'_i] : j \in A_\ell\}$ for $j \in [n]$, we prove $\exists \hat{\ell} \in [k'_i]$ such that $\forall j \in A_{\hat{\ell}}, |L_j| \geq k_i$. Assume for contradiction that $\forall \hat{\ell} \in [k'_i], \exists j_{\hat{\ell}} \in A_{\hat{\ell}}$ such that $|L_{j_{\hat{\ell}}}| \leq k_i - 1$. Consider the union of all such sets $L = \bigcup_{\hat{\ell} \in [k'_i]} L_{j_{\hat{\ell}}}$. Observe that for each $\hat{\ell} \in [k'_i]$ and $j_{\hat{\ell}} \in A_{\hat{\ell}}$, we have $\hat{\ell} \in L_{j_{\hat{\ell}}}$. Thus, it must be that $|L| \geq k'_i = n(k_i - 1) + 1$. However, since each $L_{j_{\hat{\ell}}}$ has cardinality at most $k_i - 1$, we have $|L| = |\bigcup_{\hat{\ell} \in [k'_i]} L_{j_{\hat{\ell}}}| \leq |\bigcup_{j \in [n]} L_j| \leq n(k_i - 1)$, which leads to contradiction.

Since each A_ℓ is associated with pairwise distinct c_ℓ 's, we obtain a $(1, \dots, k_{i^*-1}, k_{i^*}, \dots, k_\mu)$ -subtree $\tilde{T}_i^j$ for every $j \in A_{\hat{\ell}}$. Applying the above argument iteratively for $i = i^* - 1, \dots, 1$, we prove the statement for Case 3.

Extractor. Combining Case 1-3 above, there exists $A \in \Gamma$ such that for every $j \in A$, the corresponding decomposed tree T^j contains a $\vec{k}$ -tree $\tilde{T}^j$ for Π^j . The extractor Ext^{cds} then simply runs sub-extractors Ext^j on $\tilde{T}^j$ to extract witnesses for all $j \in A$. As Π^j is $\vec{k}$ -SS, every Ext^j always succeeds in extracting witness. $\square$

5.3 Suppressing Exponential Blow-up via Multi-Round Share-then-Hash

To support polynomial-size compositions, we generalize the Share-then-Hash method with the help of a collision resistant hash function (CRH) $H : \{0, 1\}^* \mapsto \mathcal{C}_{i^*}$. We present MR-StH composition, Π_F^{sth} , formally in Fig. 2 alongside Π_F^{cds} . The only modification is that the prover is now required to obtain challenges $c_{i^*}^j \leftarrow H(\dots, s^j)$, instead of setting $c_{i^*}^j = s^j$. Unlike the CDS composition, MR-StH allows for a secret sharing scheme with larger domains than the challenge space $\mathcal{C}_{i^*}$. Thus, one can compensate for the exponential blowup in the numerator by setting $|S| \gg (k_{i^*} - 1)^n$. As sketched in Section 2.2, the knowledge soundness of Π_F^{sth} is analyzed within the predicate special soundness framework of [AAB⁺24] (recalled in Appendix B.1) instead of the standard special soundness notion. We state the following theorem whose formal analysis is deferred to Appendix B.

Theorem 3 (MR-StH Composition: Π_F^{sth}). *For each $j \in [n]$, consider a $(2\mu + 1)$ -move public-coin proof protocol Π^j for relation R^j . If SSS_{Γ^*} is a semi-smooth perfect secret sharing scheme over domain S for access structure Γ^* , $H : \{0, 1\}^* \mapsto \mathcal{C}_{i^*}$ is a collision-resistant hash function, and every Π^j is $\vec{k}$ -SS and CRSHVZK at round $i^* = i_{\text{zk}}^*$, then, protocol Π_F^{sth} is a $(2\mu + 1)$ -move proof system for relation $R_\Gamma((x^1, \dots, x^n), (w^1, \dots, w^n))$ that outputs 1 if and only if $\exists A \in \Gamma, \forall j \in A, R^j(x^j, w^j) = 1$. It is complete, $(\vec{k}', \Phi)$ -predicate-special-sound for $\vec{k}' = (k'_1, \dots, k'_\mu)$ and a predicate system Φ , and CRSHVZK at round i^* , where*

1. For $i > i^*$, $k'_i = k_i$ and $\Phi_{i, \ell}^{\text{chal}}$ for each $\ell \in [k'_i]$ always outputs 1;
2. For $i = i^*$, $k'_i = \tau \geq n(k_i - 2) + 2$ and $\Phi_{i, \ell}^{\text{chal}}(t_1, \dots, t_{\ell-1}, s_\ell)$ for each $\ell \in [k'_i]$ returns 1 if (2) is satisfied, and 0 otherwise;
3. For $i < i^*$, $k'_i = n(k_i - 1) + 1$ and $\Phi_{i, \ell}^{\text{chal}}$ for $\ell \in [k'_i]$ always outputs 1.

Plugging Theorem 3 and the analysis of *failure density* (Lemma 2) into the general theorem on Fiat-Shamir knowledge soundness established by [AAB⁺24] (Theorem 5) results in the following corollary. The last term in the knowledge error essentially quantifies the probability that a uniformly sampled $s \in S$ (i.e., the i_{zk}^* -th round challenge) falls into a “bad” set of secrets that could cause extraction to fail.

¹¹ Note that we are abusing the notation here, because the ℓ th challenge for round i should be technically denoted by $c_{i, \ell}$ following the protocol description. We drop the subscript i for readability since we focus on the i th round when proving Case 3.

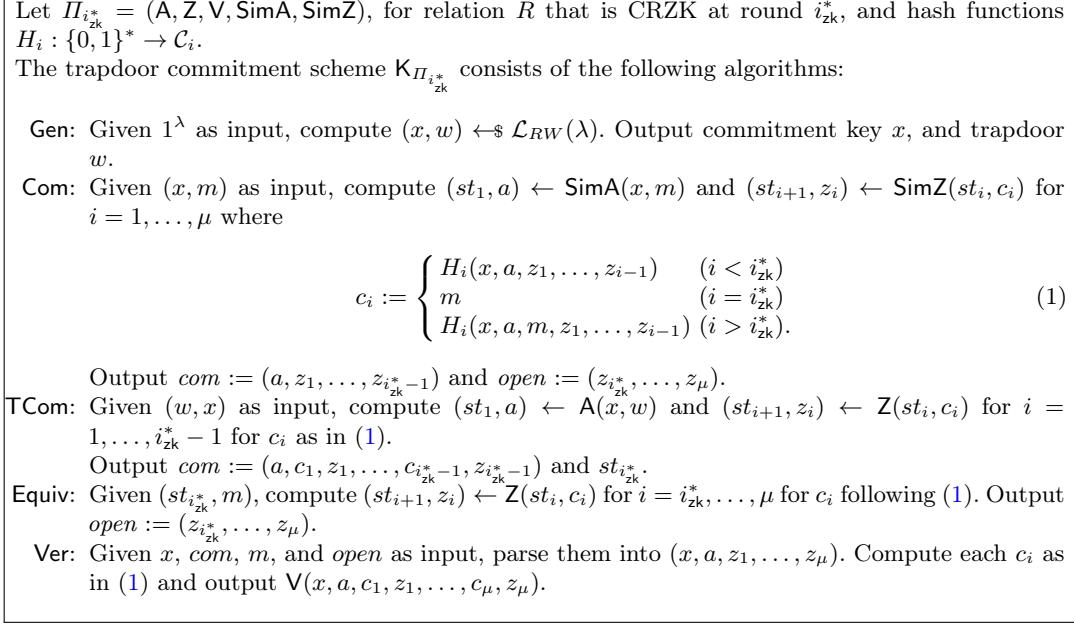


Fig. 3. Trapdoor commitment scheme $K_{\Pi_{i_{zk}^*}}$.

Corollary 1. *Let Q be the number of random oracle queries made by a cheating prover. The Fiat-Shamir-transformed version of $\Pi_{\Gamma}^{\text{sth}}$ is adaptively knowledge sound in the random oracle model with knowledge error*

$$2(Q+1) \left(\sum_{i \in [\mu] \setminus \{i_{zk}^*\}} \frac{k'_i - 1}{|\mathcal{C}_i|} + \frac{(k'_{i_{zk}^*} - 1) \cdot 2^n \cdot (k_{i_{zk}^*} - 1)^n}{|S|} \right).$$

6 Trapdoor Commitment from Multi-Round Protocol

Given a $(2\mu + 1)$ -move public-coin proof protocol, $\Pi_{i_{zk}^*} = (A, Z, V, \text{SimA}, \text{SimZ})$, for relation R that is CRZK at round i_{zk}^* , and hash functions $H_i : \{0, 1\}^* \rightarrow \mathcal{C}_i$, we construct a trapdoor commitment scheme $K = (\text{Gen}, \text{Com}, \text{TCom}, \text{Equiv}, \text{Ver})$ as illustrated in Figure 3. For simplicity, we assume that the message space $\mathcal{M} = \mathcal{C}_{i_{zk}^*}$.

We show that the hiding and equivocation properties follow from CRZK at round i_{zk}^* : there exist two simulators $(\text{SimA}, \text{SimZ})$ that generate indistinguishable transcripts, where SimA only takes as input a commitment key and a message (as the challenge at round i_{zk}^*). Furthermore, we prove that if $\Pi_{i_{zk}^*}$ is (extended) $\vec{k}$ -SS with $k_{i_{zk}^*} = 2$, then the construction is binding and trapdoor extractability. As an example, by Proposition 4, we can instantiate our TDC from the compressed Σ -protocols, since they have $i_{zk}^* = 1$ and $k_{i_{zk}^*} = 2$ and challenge spaces are exponentially large.

We briefly explain how ExSS is used in the security proofs for binding and trapdoor extractability. The proof for binding and trapdoor extractability is a standard proof by reduction. Given adversary $\mathcal{A}$ that breaks binding with non-negligible probability, We construct a new adversary $\mathcal{B}$ that uses $\mathcal{A}$ as a subroutine to either find a trapdoor for corresponding commitment key or break the hardness assumption. $\mathcal{B}$ first runs a recursive tree builder adapted from [AFK23] with rewindable access to $\mathcal{A}$ to build a full extended tree of accepting transcripts. We stress that this customization is non-trivial for the case of multi-round proofs: the binding adversary $\mathcal{A}$ outputs *two* different transcripts with a *single* call. Moreover, the messages are chosen by $\mathcal{A}$ rather than sampled by the tree builder. $\mathcal{B}$ then forwards the tree to the extended extractor ExExt. By definition of ExSS, it should either output a trapdoor or a solution to the hard problem.

We remark that in Appendix E.1 we construct an accumulator using similar ideas than for our trapdoor commitment, for CRZK protocols where $k_{i_{zk}^*} > 2$.

Theorem 4. $K_{\Pi_{i_{zk}^*}}$ in Figure 3 constitutes a trapdoor commitment scheme with the following properties:

- It is hiding and equivocal if $\Pi_{i_{zk}^*}$ is CRSHVZK at round i_{zk}^* .
- It is trapdoor extractable with negligible extraction error if
 - $\Pi_{i_{zk}^*}$ is perfectly complete,
 - $\Pi_{i_{zk}^*}$ is (extended) $\vec{k}$ -SS with $k_{i_{zk}^*} = 2$,
 - Relation R_{hd} is one-way,
 - H_i is a programmable random oracle,
 - For all $i \in [\mu] \setminus i_{zk}^*$, $\frac{k_i}{|\mathcal{C}_i|}$ is negligible.
- It is binding if, in addition to the conditions for trapdoor extractability, relation R is one-way.

Proof. (Correctness) It holds directly from the perfect completeness of $\Pi_{i_{zk}^*}$ and the critical-round zero-knowledge at round i_{zk}^* .

(Hiding Property) It is proved by a game transition argument. We begin with the left term of the hiding definition in Definition 5;

$$\begin{aligned} P_0 &:= \Pr [1 \leftarrow \mathcal{A}(ck, com) \mid (ck, td) \leftarrow \text{Gen}(1^\lambda), (com, open) \leftarrow \text{Com}(ck, m)] \\ &= \Pr \left[1 \leftarrow \mathcal{A}(x, (a, z_1, \dots, z_{i_{zk}^*-1})) \left| \begin{array}{l} (x, w) \leftarrow L_{RW}(1^\lambda), \\ (st_1, a) \leftarrow \text{SimA}(x, m), \\ \forall i \in [\mu], (st_{i+1}, z_i) \leftarrow \text{SimZ}(st_i, c_i) \end{array} \right. \right] \end{aligned}$$

where c_i is computed following (1). We then modify the game by replacing the zero-knowledge simulator with a real prover algorithm.

$$P_1 := \Pr \left[1 \leftarrow \mathcal{A}(x, (a, z_1, \dots, z_{i_{zk}^*-1})) \left| \begin{array}{l} (x, w) \leftarrow L_{RW}(1^\lambda), \\ (st_1, a) \leftarrow \text{A}(x, w), \\ \forall i \in [\mu], (st_{i+1}, z_i) \leftarrow \text{Z}(st_i, c_i) \end{array} \right. \right]$$

Then $|P_0 - P_1|$ is bound by the zero-knowledge error, say ϵ_{crzk} , as in Definition 7. Finally, we make it back to the simulation as follows.

$$P_2 = \Pr \left[1 \leftarrow \mathcal{A}(x, (a, z_1, \dots, z_{i_{zk}^*-1})) \left| \begin{array}{l} (x, w) \leftarrow L_{RW}(1^\lambda), \\ (st_1, a) \leftarrow \text{SimA}(x, m'), \\ \forall i \in [\mu], (st_{i+1}, z_i) \leftarrow \text{SimZ}(st_i, c_i) \end{array} \right. \right]$$

Again, $|P_1 - P_2|$ is upper bound by ϵ_{crzk} . Since P_2 is the same as the right term of the hiding definition, we obtain $P_0 - P_2 \leq 2\epsilon_{\text{crzk}}$ as a bound for the hiding property that is negligible as ϵ_{crzk} is negligible by assumption.

(Equivocability) It holds due to CRSHVZK of $\Pi_{i_{zk}^*}$ since the equivocability game in Definition 5 is exactly the same as the CRSHVZK game in Definition 7.

(Binding) We start from binding game first. Let $\mathcal{A}$ be a successful adversary that opens a commitment in two ways with probability $\epsilon_{\text{bin}} := \text{Adv}_{\mathcal{T}\mathcal{D}, \mathcal{A}}^{\text{bin}}(\lambda)$ and Q be the number of random oracle queries made by $\mathcal{A}$. Precisely, given the commitment key x , it outputs $(com, m_0, open_0, m_1, open_1)$ such that $m_0 \neq m_1$, $m_0, m_1 \in \mathcal{C}_{i_{zk}^*}$, and $\text{Ver}(com, m_j, open_j) = 1$ for $j \in \{0, 1\}$ with probability ϵ_{bin} over its random coins, choices of x and random oracles. For simplicity, we assume that all the challenge spaces $\mathcal{C}_i$ are identical and represent each H_i using a single random oracle H . The overall structure of the proof follows the blueprint of the Fiat-Shamir security proof from [AFK23].

Given $\mathcal{A}$ as a black-box, we construct an adversary $\mathcal{B}$ that breaks the one-wayness of R or R_{hd} . It constructs a $\vec{k}$ -tree of transcripts by running the tree builder $\mathcal{T}_0$, as illustrated in Figure 4. $\mathcal{B}$ simulates H by lazy sampling. Whenever H receives a fresh query, $\mathcal{B}$ returns a random value. By $(in, out) \rightarrow H$, we denote programming H as $out = H(in)$. Finally, $\mathcal{T}_0$ outputs $v = 1$ with a valid $\vec{k}$ -tree of transcripts. Otherwise it aborts.

The recursive tree builder $\mathcal{T}_i$ is inspired by the one in [AFK23] for the multi-round Fiat-Shamir security, but there are notable differences due to the adversary $\mathcal{A}$ playing the binding game rather than the standard Fiat-Shamir security game as in [AFK23].



Fig. 4. Tree builder $\mathcal{T}_i$ with binding adversary $\mathcal{A}$.

- Parallel Tree Building: $\mathcal{A}$ outputs two transcripts, $(com, m_0, open_0)$ and $(com, m_1, open_1)$, branching at round i_{zk}^* . Both of them must be included in the tree since the self-chosen values m_0 and m_1 could represent the only challenges at round i_{zk}^* for which $\mathcal{A}$ is successful. Therefore, $\mathcal{T}_{i > i_{zk}^*}$ must construct sub-trees for both branches (com, m_0) and (com, m_1) in parallel, as described in Step 2 of the case $i > i_{zk}^*$ in Figure 4. Within each sub-tree construction, at most one of the sub-trees returned from the descending builder $\mathcal{T}_{i+1}$ will match the intended branch.

$\mathcal{T}_{i_{zk}^*}$ only needs to combine the two sub-trees into a single structure. The ancestor builders $\mathcal{T}_{i < i_{zk}^* - 1}$ remain consistent with the original builders outlined in [AFK23].

- Self-Chosen Challenges: In round i_{zk}^* , the challenges m_0 and m_1 are selected by $\mathcal{A}$. But this is not essentially different from choosing them uniformly and consider the case that $\mathcal{A}$ succeeds. The process of constructing the sub-tree for $i > i_{zk}^*$ is analogous to allowing the adversary to choose the initial message a in the top tree. Consequently, the challenge m is included as a part of input to $H_{i > i_{zk}^*}$. If $\mathcal{A}$ selects m_0 and m_1 such that tree building fails, it subsequently reduces the success probability of $\mathcal{A}$, as demonstrated in Proposition 2 of [AFK23]. Furthermore, since the sub-extractor $\mathcal{T}_{i_{zk}^*}$ succeeds with the same probability as $\mathcal{T}_{i_{zk}^*-1}$, it can be considered merged into $\mathcal{T}_{i_{zk}^*-1}$ when analyzing the bounds.

Let y be the hard instance. From Proposition 2 in [AFK23], the probability that $\mathcal{T}_0$ returns $v = 1$ (and outputs an accepting tree T) is at least

$$\Pr \left[v = 1 \left| \begin{array}{l} (x, w) \leftarrow \text{Gen}(1^\lambda) \\ (y, s) \leftarrow L_{R_{hd}}(1^\lambda), \\ (T, v) \leftarrow \mathcal{T}_0^{\mathcal{A}(\cdot)}(x, y) \end{array} \right. \right] \geq \frac{\epsilon_{\text{bin}} - (Q + 1)\theta}{1 - \theta}$$

where $\theta = 1 - \prod_{i \in [\mu] \setminus i_{\text{zk}}^*} (1 - \frac{k_i - 1}{|\mathcal{C}_i|})$ and Q is the number of random oracle queries made by $\mathcal{A}$. The expected number of invocations to $\mathcal{A}$ is at most $\xi + Q \cdot (\xi - 1)$ where $\xi = \prod_{i \in [\mu] \setminus i_{\text{zk}}^*} k_i$.

Finally, after the tree builder $\mathcal{T}_0$ outputs a valid tree for extended $\vec{k}$ -SS, the adversary $\mathcal{B}$ forwards the tree to **ExExt** as input. **ExExt** then outputs (w', s') with probability 1 such that either $R_{hd}(y, s') = 1$ or $R(x, w') = 1$. Let the probability ϵ_R and ϵ_{hd} be defined as

$$\epsilon_R := \Pr \left[v = 1 \wedge R(x, w') = 1 \left| \begin{array}{l} (x, w) \leftarrow \text{Gen}(1^\lambda) \\ (y, s) \leftarrow L_{R_{hd}}(1^\lambda), \\ (T, v) \leftarrow \mathcal{T}_0^{\mathcal{A}(\cdot)}(x, y) \\ (w', s') \leftarrow \text{ExExt}(x, y, T) \end{array} \right. \right]$$

and

$$\epsilon_{hd} := \Pr \left[v = 1 \wedge R_{hd}(y, s') = 1 \left| \begin{array}{l} (x, w) \leftarrow \text{Gen}(1^\lambda) \\ (y, s) \leftarrow L_{R_{hd}}(1^\lambda), \\ (T, v) \leftarrow \mathcal{T}_0^{\mathcal{A}(\cdot)}(x, y) \\ (w', s') \leftarrow \text{ExExt}(x, y, T) \end{array} \right. \right].$$

The probability is taken over the random coins of Gen , $L_{R_{hd}}$, $\mathcal{T}_0$, and **ExExt**.

By the union bound, we thus have

$$\begin{aligned} \epsilon_{hd} + \epsilon_R &\geq \Pr \left[v = 1 \left| \begin{array}{l} (x, w) \leftarrow \text{Gen}(1^\lambda) \\ (y, s) \leftarrow L_{R_{hd}}(1^\lambda), \\ (T, v) \leftarrow \mathcal{T}_0^{\mathcal{A}(\cdot)}(x, y) \\ (w', s') \leftarrow \text{ExExt}(x, y, T) \end{array} \right. \right] \\ &= \Pr \left[v = 1 \left| \begin{array}{l} (x, w) \leftarrow \text{Gen}(1^\lambda) \\ (y, s) \leftarrow L_{R_{hd}}(1^\lambda), \\ (T, v) \leftarrow \mathcal{T}_0^{\mathcal{A}(\cdot)}(x, y) \end{array} \right. \right] \\ &\geq \frac{\epsilon_{\text{bin}} - (Q + 1)\theta}{1 - \theta}, \end{aligned}$$

that upper bounds ϵ_{bin} as

$$\epsilon_{\text{bin}} \leq (1 - \theta)(\epsilon_{hd} + \epsilon_R) + (Q + 1)\theta,$$

where $\theta = 1 - \prod_{i \in [\mu] \setminus i_{\text{zk}}^*} (1 - \frac{k_i - 1}{|\mathcal{C}_i|})$ and Q is the number of random oracle queries made by $\mathcal{A}$. θ is negligible if for all $i \in [\mu] \setminus i_{\text{zk}}^*$, $|\mathcal{C}_i|$ is exponentially large, which holds by assumption. By the one-wayness of the hard relation of R_{hd} and R , both ϵ_{hd} and ϵ_R are negligible. It turns out that ϵ_{bin} is negligible as well.

(*Trapdoor Extractability*) For the trapdoor extractability game, given oracle access to any binding adversary $\mathcal{A}$ with success probability $\epsilon_{\text{bin}} := \text{Adv}_{\mathcal{T}_D, \mathcal{A}}^{\text{bin}}(\lambda)$, we construct an extractor Ext_{tde} attempting to extract a trapdoor. The extractor succeeds if it outputs w' that satisfies $f_t(x, w') = 1$.

Ext_{tde} behaves essentially the same as the adversary $\mathcal{B}$ built in the binding game. It first runs the tree builder $\mathcal{T}_0$ to obtain a $\vec{k}$ -tree of accepting transcripts and then forwards the tree to the extended extractor **ExExt**. **ExExt** outputs (w', s') with probability 1 such that either $R_{hd}(y, s') = 1$ or $R(x, w') = 1$. Let the probability ϵ_R and ϵ_{hd} be defined as as in the binding game.

From the above binding game, we have

$$\epsilon_{hd} + \epsilon_R \geq \frac{\epsilon_{\text{bin}} - (Q + 1)\theta}{1 - \theta}.$$

From the construction of Ext_{tde} , it holds that

$$\Pr \left[f_t(x, w') = 1 \mid \begin{array}{l} (x, w) \leftarrow \text{Gen}(1^\lambda) \\ (y, s) \leftarrow L_{R_{hd}}(1^\lambda), \\ w' \leftarrow \text{Ext}_{\text{tde}}^{A(\cdot)}(x, y) \end{array} \right] = \epsilon_R \geq \frac{\epsilon_{\text{bin}} - (Q + 1)\theta - (1 - \theta)\epsilon_{hd}}{1 - \theta},$$

where $\theta = 1 - \prod_{i \in [\mu] \setminus i_{zk}^*} (1 - \frac{k_i - 1}{|C_i|})$ and Q is the number of random oracle queries made by $\mathcal{A}$. In conclusion, for any PPT adversary $\mathcal{A}$ breaking the binding property with probability $\text{Adv}_{\text{TDC}, \mathcal{A}}^{\text{bin}}(\lambda)$, there exists an extractor that outputs a valid trapdoor with extraction error at most $\delta_{\text{tde}} = (Q + 1)\theta + (1 - \theta)\epsilon_{hd}$. As we assume that ϵ_{hd} is negligible and that for all $i \in [\mu] \setminus i_{zk}^*$, $|C_i|$ is exponentially large, the extraction error is negligible. $\square$

7 Conclusion

Based on the observation that the roles of each round differ in multi-round protocols, we introduced the novel notion of CRZK and demonstrated its usefulness. We briefly describe a few further applications of our results and mention some open problems.

Further Applications of Proofs of Partial Knowledge. Our results on proofs-of-partial-knowledge (PoPK) can be used to build modular and efficient proof systems, compared to the naive alternative in which a monolithic proof for the entire relation is constructed by expressing the access structure at the circuit level (e.g., at the R1CS level in a SNARK). One advantage is overcoming CRS size limitations. For instance, in an SNARK, it is not possible to prove a disjunction on, e.g., 4 witnesses of size 2^{20} with a CRS of size 2^{21} . Our technique overcomes the limitation and avoids having to run a new CRS ceremony. Another advantage can be running times: if the prover time is not linear but, e.g., quasilinear, proving a disjunction of k statements using our method is faster than increasing the circuit size k times.

Separately, PoPK for MSP is useful in constructing advanced primitives, e.g., anonymous credentials (AC) and attribute-based signatures (ABS) with complex policy predicates. Recall that the central building block in these primitives is a proof system convincing the verifier that a prover or signer holds a set of secret attributes satisfying a predicate, while partially disclosing non-sensitive attributes. With our PoPK, for instance, one may design AC or ABS supporting a hierarchical policy such as $(\text{age} \geq 20 \text{ AND } (\text{country} \in \text{EU OR } \text{gender} = \text{Female})) \text{ OR } (\text{country} \in \text{Asia AND } \text{gender} = \text{Male})$, which cannot be expressed as a simple threshold access structure.

Further Applications of Trapdoor Commitments. With our trapdoor commitments, trapdoor equivocation can be performed during a protocol execution in the real world using only a trapdoor; only the proofs of hiding, binding and trapdoor extractability rely on programmability. This feature has important consequences when TDC is used as a building block. One concrete application benefiting from our TDCs is post-quantum threshold ring signatures in [HS20], where signers holding trapdoors as secret keys need to equivocate commitments. As their instantiation relies on inefficient ZKP for graph hamiltonicity, our TDC potentially enables more efficient instantiations from state-of-the-art multi-round proofs for NP. Another application are designated verifier proofs [JSI96], where a designated verifier knowing the trapdoor equivocates a commitment from a prover to divert the proof transcript to a third-party.

Other than these applications requiring equivocation in the real world, our TDC could also be used to improve efficiency of classical applications in the CRS model, such as adaptive ZK and concurrent ZK proofs [Dam00], which require straightline ZK simulation that can be achieved by TDC equivocation using the CRS trapdoor.

Open questions. Several open questions remain:

- We observed that some multi-round proof protocols that utilize the witness in only one round can admit a CRZK simulator. Could this observation be extended into a more general theorem?
- Are the blow-ups of k'_i for rounds $i \leq i^*$ inherent in MR-CDS and MR-StH compositions in Section 5?
- The black-box construction of non-malleable zero-knowledge from [BCO⁺24] relies on the ability to simulate a sigma-protocol from the knowledge of the (only) challenge. An interesting open question is to explore whether CRZK protocols can be used in place of such sigma-protocols efficiently.

Acknowledgements

This work was partially done while Dung Bui was an intern at NTT Social Informatics Laboratories, and while David Balbás was an intern at NTT Social Informatics Laboratories and a PhD student at IMDEA Software Institute.

David Balbás is supported by the PICOCRYPT project that has received funding from the European Research Council (ERC) under the European Union’s Horizon 2020 research and innovation programme (Grant agreement No. 101001283), partially funded by the grants PID2022-142290OB-I00 and CEX2024-001471-M funded by MCIN/AEI/10.13039/501100011033/ FEDER, UE, and partially funded by Ministerio de Universidades (FPU21/00600).

Dung Bui is supported by the funding from the European Unions Horizon Europe research and innovation programme under the project “Quantum Security Networks Partnership” (QSNP, grant agreement No 101114043).

Zhiyu Peng’s participation in the early stages of this work is appreciated. We also thank Miguel Ambrona for useful comments about Plonk, Marius Aardal and Sebastian Kolby for helpful discussions regarding the predicate special soundness framework of [AAB⁺24], and Luisa Siniscalchi for discussions about potential applications of our notions.

This paper was prepared in part for information purposes by the Artificial Intelligence Research group of JPMorgan Chase & Co and its affiliates (“JP Morgan”), and is not a product of the Research Department of JP Morgan. JP Morgan makes no representation and warranty whatsoever and disclaims all liability, for the completeness, accuracy or reliability of the information contained herein. This document is not intended as investment research or investment advice, or a recommendation, offer or solicitation for the purchase or sale of any security, financial instrument, financial product or service, or to be used in any way for evaluating the merits of participating in any transaction, and shall not constitute a solicitation under any jurisdiction or to any person, if such solicitation under such jurisdiction or to such person would be unlawful.

References

- AAB⁺20. M. Abe, M. Ambrona, A. Bogdanov, M. Ohkubo, and A. Rosen. Non-interactive composition of sigma-protocols via share-then-hash. In *ASIACRYPT 2020, Part III*, LNCS 12493, pages 749–773, December 2020.
- AAB⁺21. M. Abe, M. Ambrona, A. Bogdanov, M. Ohkubo, and A. Rosen. Acyclicity programming for sigma-protocols. In *TCC 2021, Part I*, LNCS 13042, pages 435–465, November 2021.
- AAB⁺24. M. A. Aardal, D. F. Aranha, K. Boudgoust, S. Kolby, and A. Takahashi. Aggregating falcon signatures with LaBRADOR. In *CRYPTO 2024, Part I*, LNCS, pages 71–106, August 2024.
- AABN02. M. Abdalla, J. H. An, M. Bellare, and C. Namprempe. From identification to signatures via the Fiat-Shamir transform: Minimizing assumptions for security and forward-security. In *EUROCRYPT 2002*, LNCS 2332, pages 418–433, April / May 2002.
- ABF⁺25. G. Avitabile, V. Botta, D. Friolo, D. Venturi, and I. Visconti. Compact proofs of partial knowledge for overlapping CNF formulae. *J. Cryptol.*, 38(1):7, 2025.
- ABFV22. G. Avitabile, V. Botta, D. Friolo, and I. Visconti. Efficient proofs of knowledge for threshold relations. In *ESORICS 2022, Part III*, LNCS 13556, pages 42–62, September 2022.
- ABO⁺24. M. Abe, A. Bogdanov, M. Ohkubo, A. Rosen, Z. Shang, and M. Tibouchi. CDS composition of multi-round protocols. In *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part IX*, Lecture Notes in Computer Science 14928, pages 391–423. Springer, 2024.
- AC20. T. Attema and R. Cramer. Compressed Σ -protocol theory and practical application to plug & play secure algorithmics. In *CRYPTO 2020, Part III*, LNCS 12172, pages 513–543, August 2020.

- ACF21. T. Attema, R. Cramer, and S. Fehr. Compressing proofs of k-out-of-n partial knowledge. In *CRYPTO 2021, Part IV, LNCS* 12828, pages 65–91, Virtual Event, August 2021.
- ACK21. T. Attema, R. Cramer, and L. Kohl. A compressed Σ -protocol theory for lattices. In *CRYPTO 2021, Part II, LNCS* 12826, pages 549–579, Virtual Event, August 2021.
- AF22. T. Attema and S. Fehr. Parallel repetition of $(k_1, \dots, k_\mu)$ -special-sound multi-round interactive proofs. In *CRYPTO 2022, Part I, LNCS* 13507, pages 415–443, August 2022.
- AFK22. T. Attema, S. Fehr, and M. Klooß. Fiat-shamir transformation of multi-round interactive proofs. In *TCC 2022, Part I, LNCS* 13747, pages 113–142, November 2022.
- AFK23. T. Attema, S. Fehr, and M. Klooß. Fiat-shamir transformation of multi-round interactive proofs (extended version). *Journal of Cryptology*, 36(4):36, October 2023.
- AFR23. T. Attema, S. Fehr, and N. Resch. Generalized special-sound interactive proofs and their knowledge soundness. In *TCC 2023, Part III, LNCS*, pages 424–454, November 2023.
- AKLY24. T. Attema, M. Klooß, R. W. F. Lai, and P. Yatsyna. Adaptive special soundness: Improved knowledge extraction by adaptive useful challenge sampling. *Cryptology ePrint Archive*, Paper 2024/2038, 2024.
- BBB⁺18. B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE Symposium on Security and Privacy*, pages 315–334. IEEE Computer Society Press, May 2018.
- BCC⁺16. J. Bootle, A. Cerulli, P. Chaidos, J. Groth, and C. Petit. Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. In *EUROCRYPT 2016, Part II, LNCS* 9666, pages 327–357, May 2016.
- BCO⁺24. V. Botta, M. Ciampi, E. Orsini, L. Siniscalchi, and I. Visconti. Black-box (and fast) non-malleable zero knowledge. *LNCS*, pages 458–490, August 2024.
- BCS16. E. Ben-Sasson, A. Chiesa, and N. Spooner. Interactive oracle proofs. In *TCC 2016-B, Part II, LNCS* 9986, pages 31–60, October / November 2016.
- Bd94. J. C. Benaloh and M. de Mare. One-way accumulators: A decentralized alternative to digital signatures (extended abstract). In *EUROCRYPT’93, LNCS* 765, pages 274–285, May 1994.
- BF23. B. Bünz and B. Fisch. Multilinear schwartz-zippel mod N and lattice-based succinct arguments. In *TCC 2023, Part III, LNCS*, pages 394–423, November 2023.
- BFS20. B. Bünz, B. Fisch, and A. Szeponiec. Transparent SNARKs from DARK compilers. In *EUROCRYPT 2020, Part I, LNCS* 12105, pages 677–706, May 2020.
- BGTZ25. A. R. Block, A. Garreta, P. R. Tiwari, and M. Zajac. On soundness notions for interactive oracle proofs. *J. Cryptol.*, 38(1):4, 2025.
- BL90. J. C. Benaloh and J. Leichter. Generalized secret sharing and monotone functions. In *CRYPTO’88, LNCS* 403, pages 27–35, August 1990.
- Blu86. M. Blum. How to prove a theorem so no one else can claim it. In *The International Congress of Mathematicians (ICM)*, 1986, 1986.
- BN20. C. Baum and A. Nof. Concretely-efficient zero-knowledge arguments for arithmetic circuits and their application to lattice-based cryptography. In *PKC 2020, Part I, LNCS* 12110, pages 495–526, May 2020.
- BP97. N. Bari and B. Pfitzmann. Collision-free accumulators and fail-stop signature schemes without trees. In *EUROCRYPT’97, LNCS* 1233, pages 480–494, May 1997.
- CCH⁺19. R. Canetti, Y. Chen, J. Holmgren, A. Lombardi, G. N. Rothblum, R. D. Rothblum, and D. Wichs. Fiat-Shamir: from practice to theory. In *51st ACM STOC*, pages 1082–1090. ACM Press, June 2019.
- CDM00. R. Cramer, I. Damgård, and P. D. MacKenzie. Efficient zero-knowledge proofs of knowledge without intractability assumptions. In *PKC 2000, LNCS* 1751, pages 354–372, January 2000.
- CDS94. R. Cramer, I. Damgård, and B. Schoenmakers. Proofs of partial knowledge and simplified design of witness hiding protocols. In *CRYPTO’94, LNCS* 839, pages 174–187, August 1994.
- CHM⁺20. A. Chiesa, Y. Hu, M. Maller, P. Mishra, P. Vesely, and N. P. Ward. Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In *EUROCRYPT 2020, Part I, LNCS* 12105, pages 738–768, May 2020.
- CPS⁺16a. M. Ciampi, G. Persiano, A. Scafuro, L. Siniscalchi, and I. Visconti. Improved OR-composition of sigma-protocols. In *TCC 2016-A, Part II, LNCS* 9563, pages 112–141, January 2016.
- CPS⁺16b. M. Ciampi, G. Persiano, A. Scafuro, L. Siniscalchi, and I. Visconti. Online/offline OR composition of sigma protocols. In *EUROCRYPT 2016, Part II, LNCS* 9666, pages 63–92, May 2016.
- Cra96. R. Cramer. *Modular Design of Secure yet Practical Cryptographic Protocols*. PhD thesis, University of Amsterdam, November 1996.
- CV05. D. Catalano and I. Visconti. Hybrid trapdoor commitments and their applications. In *ICALP 2005, LNCS* 3580, pages 298–310. Springer, Heidelberg, July 2005.
- Dam90. I. Damgård. On the existence of bit commitment schemes and zero-knowledge proofs. In *CRYPTO’89, LNCS* 435, pages 17–27, August 1990.

- Dam00. I. Damgård. Efficient concurrent zero-knowledge in the auxiliary string model. In *EUROCRYPT 2000, LNCS 1807*, pages 418–430, May 2000.
- DFMS22. J. Don, S. Fehr, C. Majenz, and C. Schaffner. Online-extractability in the quantum random-oracle model. In *EUROCRYPT 2022, Part III, LNCS 13277*, pages 677–706, May / June 2022.
- DG23. Q. Dao and P. Grubbs. Spartan and bulletproofs are simulation-extractable (for free!). In *EUROCRYPT 2023, Part II, LNCS 14005*, pages 531–562, April 2023.
- FGQ⁺23. P.-A. Fouque, A. Georgescu, C. Qian, A. Roux-Langlois, and W. Wen. A generic transform from multi-round interactive proof to NIZK. In *PKC 2023, Part II, LNCS 13941*, pages 461–481, May 2023.
- Fis01. M. Fischlin. Trapdoor commitment schemes and their applications, 01 2001.
- FJR22. T. Feneuil, A. Joux, and M. Rivain. Syndrome decoding in the head: Shorter signatures from zero-knowledge proofs. In *CRYPTO 2022, Part II, LNCS 13508*, pages 541–572, August 2022.
- FS87. A. Fiat and A. Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *CRYPTO’86, LNCS 263*, pages 186–194, August 1987.
- GGHAK22. A. Goel, M. Green, M. Hall-Andersen, and G. Kaptchuk. Stacking sigmas: A framework to compose Σ -protocols for disjunctions. In *EUROCRYPT 2022, Part II, LNCS 13276*, pages 458–487, May / June 2022.
- GHAKS23. A. Goel, M. Hall-Andersen, G. Kaptchuk, and N. Spooner. Speed-stacking: Fast sublinear zero-knowledge proofs for disjunctions. In *EUROCRYPT 2023, Part II, LNCS 14005*, pages 347–378, April 2023.
- GKK⁺22. C. Ganesh, H. Khoshakhlagh, M. Kohlweiss, A. Nitulescu, and M. Zajac. What makes fiat-shamir zkSNARKs (updatable SRS) simulation extractable? LNCS, pages 735–760, 2022.
- GMW91. O. Goldreich, S. Micali, and A. Wigderson. Proofs that yield nothing but their validity for all languages in NP have zero-knowledge proof systems. *J. ACM*, 38(3):691–729, 1991.
- GOP⁺22. C. Ganesh, C. Orlandi, M. Pancholi, A. Takahashi, and D. Tschudi. Fiat-shamir bulletproofs are non-malleable (in the algebraic group model). In *EUROCRYPT 2022, Part II, LNCS 13276*, pages 397–426, May / June 2022.
- GOP⁺25. C. Ganesh, C. Orlandi, M. Pancholi, A. Takahashi, and D. Tschudi. Fiat-shamir bulletproofs are non-malleable (in the random oracle model). *J. Cryptol.*, 38(1):11, 2025.
- GW11. C. Gentry and D. Wichs. Separating succinct non-interactive arguments from all falsifiable assumptions. In *43rd ACM STOC*, pages 99–108. ACM Press, June 2011.
- GWC19. A. Gabizon, Z. J. Williamson, and O. Ciobotaru. PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Report 2019/953, 2019.
- HS20. A. Haque and A. Scafuro. Threshold ring signatures: New definitions and post-quantum security. In *PKC 2020, Part II, LNCS 12111*, pages 423–452, May 2020.
- HV20. C. Hazay and M. Venkatasubramanian. On the power of secure two-party computation. *Journal of Cryptology*, 33(1):271–318, January 2020.
- IKOS07. Y. Ishai, E. Kushilevitz, R. Ostrovsky, and A. Sahai. Zero-knowledge from secure multiparty computation. In *39th ACM STOC*, pages 21–30. ACM Press, June 2007.
- JSI96. M. Jakobsson, K. Sako, and R. Impagliazzo. Designated verifier proofs and their applications. In *EUROCRYPT’96, LNCS 1070*, pages 143–154, May 1996.
- KKW18. J. Katz, V. Kolesnikov, and X. Wang. Improved non-interactive zero knowledge with applications to post-quantum signatures. In *ACM CCS 2018*, pages 525–537. ACM Press, October 2018.
- KLP22. A. Kim, X. Liang, and O. Pandey. A new approach to efficient non-malleable zero-knowledge. In *CRYPTO 2022, Part IV, LNCS 13510*, pages 389–418, August 2022.
- Kol24. D. Kolonelos. *Succinct Cryptographic Commitments with Fine-Grained Openings for Decentralized Environments*. PhD thesis, Universidad Politécnica de Madrid, 2024.
- KZ22. D. Kales and G. Zaverucha. Efficient lifting for shorter zero-knowledge proofs and post-quantum signatures. Cryptology ePrint Archive, Report 2022/588, 2022.
- KZG10. A. Kate, G. M. Zaverucha, and I. Goldberg. Constant-size commitments to polynomials and their applications. In *ASIACRYPT 2010, LNCS 6477*, pages 177–194, December 2010.
- Lip25. H. Lipmaa. Plonk is simulation extractable in ROM under falsifiable assumptions. In *Theory of Cryptography - 23rd International Conference, TCC 2025, Aarhus, Denmark, December 1-5, 2025, Proceedings, Part IV, Lecture Notes in Computer Science 16271*, pages 3–36. Springer, 2025.
- LPS25. H. Lipmaa, R. Parisella, and J. Siim. On knowledge-soundness of plonk in ROM from falsifiable assumptions. In *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part VII, Lecture Notes in Computer Science 16006*, pages 362–395. Springer, 2025.
- RRR16. O. Reingold, G. N. Rothblum, and R. D. Rothblum. Constant-round interactive proofs for delegating computation. In *48th ACM STOC*, pages 49–62. ACM Press, June 2016.

- Set20. S. Setty. Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In *CRYPTO 2020, Part III, LNCS* 12172, pages 704–737, August 2020.
- Sha79. A. Shamir. How to share a secret. *Commun. ACM*, 22(11):612–613, November 1979.
- Ste06. J. Stern. A new paradigm for public key identification. *IEEE Trans. Inf. Theor.*, 42(6-P1):1757–1768, sep 2006.
- ZLH⁺22. G. Zeng, J. Lai, Z. Huang, Y. Wang, and Z. Zheng. DAG- Σ : A DAG-based sigma protocol for relations in CNF. In *ASIACRYPT 2022, Part II, LNCS* 13792, pages 340–370, December 2022.

A Composition of Multi-Round Protocols via CDS

A.1 Monotone Access Structure

First we recall the definition of the monotone access structure from [CDS94].

Definition 8 (Monotone Access Structure [CDS94]). An access structure $\Gamma \subset 2^M$ defined over a set M is called a monotone access structure if for all $A \in \Gamma$ and for all $B \supset A$ it holds that $B \in \Gamma$. Sets in Γ are called authorized sets, and sets not in Γ are called unauthorized sets.

Definition 9 (Dual Structure [CDS94]). Let Γ be an access structure defined over a set M . If $A \subseteq M$, then $\bar{A}$ denotes the complement of A in M . Now Γ^* , the dual access structure is defined as follows:

$$A \in \Gamma^* \Leftrightarrow \bar{A} \notin \Gamma.$$

The dual Γ^* of a monotone access structure is also monotone, and satisfies $(\Gamma^*)^* = \Gamma$.

We also recall the following proposition from [CDS94].

Proposition 5 ([CDS94]). Let Γ be monotone. A set is qualified in Γ if and only if it has a non-empty intersection with every qualified set in Γ^* .

A.2 Secret Sharing Scheme

A semi-smooth perfect secret sharing scheme [CDS94], SSS_Γ , over domain S and access structure Γ over a set M consists of four polynomial-time algorithms, Share, Rec, CheckShares and Complete that:

$\text{Share}_\Gamma(s) \rightarrow \{s^j\}_{j \in M}$: is a probabilistic algorithm that takes secret $s \in S$ and outputs shares $\{s^j\}_{j \in M}$. Let $D(s)$ the distribution of outputs from $\text{Share}_\Gamma(s)$.

$\text{Rec}_\Gamma(\{s^j\}_{j \in A}) \rightarrow s$: is a reconstruction algorithm that takes a qualified set of shares $\{s^j\}_{j \in A}$, $A \in \Gamma$, and recovers secret $s \in S$.

$\text{CheckShares}_\Gamma(s, \{s^j\}_{j \in M})$: is a share verification algorithm that takes secret s and all shares $\{s^j\}_{j \in M}$, returns 1 or 0.

$\text{Complete}_\Gamma(s, \{s^j\}_{j \in A})$: takes shares of a non-qualified set of shares $\{s^j\}_{j \in A}$ for $A \notin \Gamma$ and secret s , and outputs $\{s^j\}_{j \in \bar{A}}$ that $\{s^j\}_{j \in M}$ constitute a complete set of shares of s .

It provides the following properties:

Correctness. For all $s \in S$, $\{s^j\}_{j \in M} \leftarrow \text{Share}_\Gamma(s)$, and $A \in \Gamma$, it holds that $s \leftarrow \text{Rec}(\{s^j\}_{j \in A})$.

Perfect hiding. For any $A \notin \Gamma$, $s \in S$, and $\{s^j\}_{j \in M} \leftarrow \text{Share}_\Gamma(s)$, the distribution of $\{s^j\}_{j \in A}$, denoted by D_A , is independent of s .

Consistency testing. $\text{CheckShares}_\Gamma(s, \{s^j\}_{j \in M})$ returns 1 if and only if, for all $A \in \Gamma$, $\text{Rec}_\Gamma(\{s^j\}_{j \in A}) = s$.

Share completion. For any $A \notin \Gamma$, $s \in S$, $\{s^j\}_{j \in A} \leftarrow D_A$, and $\{s^j\}_{j \in \bar{A}} \leftarrow \text{Complete}_\Gamma(s, \{s^j\}_{j \in A})$, it holds that $1 \leftarrow \text{CheckShares}_\Gamma(s, \{s^j\}_{j \in M})$ and $\{s^j\}_{j \in M}$ are distributed according to $D(s)$.

Efficient SSS_Γ exists for Γ being a threshold structure [Sha79], monotone circuit [BL90], and monotone span program [CDM00]. If, for every non-qualified set $A \notin \Gamma$, D_A equals uniform distribution over the corresponding share domain, then it is called a *smooth* perfect secret sharing scheme. Shamir’s secret sharing scheme for threshold structures is an example [Sha79].

A.3 Insecurity of Naïve Multi-Round CDS

We describe CDS composition [CDS94] in a general form for $2\mu + 1$ -move protocols. It matches the original composition when $\mu = 1$. We warn that the protocol is not sound for $\mu \geq 2$ and $k(> 3)$ -special sound as mentioned earlier, and we only introduce it to establish notation and illustrate the problematic of the naïve composition.

Let Π^j be a $(2\mu+1)$ -move public-coin proof protocol for relation R^j with a tuple of three algorithms (A^j, Z^j, V^j) (Definition 1), and (x^j, w^j) be a pair of instance and witness satisfying $R^j(x^j, w^j) = 1$. Let Γ be a monotone access structure over $[n]$, and $R_\Gamma((x^1, \dots, x^n), (w^1, \dots, w^n))$ be a compound relation that returns 1 if and only if there exists $A \in \Gamma$ that $R^j(x^j, w^j) = 1$ for all $j \in A$. We denote the special honest verifier zero-knowledge simulator of Π^j by Sim^j . Given Π^j for $j \in [n]$, an access structure Γ , and a semi-smooth secret sharing scheme SSS_{i, Γ^*} over a secret domain S_i and a share domain C_i for $i \in [\mu]$, a naïve CDS composition constructs Prover and Verifier as follows where steps $2i$ and $2i + 1$ are repeated for $i \in [\mu]$:

$\Pi_{\mu, \Gamma}^{\text{cds}'}$ (Prover($\{x^j\}_{j \in [n]}, \{w^j\}_{j \in A}$), Verifier($\{x^j\}_{j \in [n]}\)):$

- Step 1. For each $i \in [\mu]$, Prover samples $\{c_i^j\}_{j \in \bar{A}} \leftarrow D_{\bar{A}}$. Then Prover calls the special honest verifier zero-knowledge simulator $(a^j, \{z_i^j\}_{i \in [\mu]}) \leftarrow \text{Sim}^j(x^j, \{c_i^j\}_{i \in [\mu]})$. For those $j \in A$, Prover commits to a^j by running $a^j \leftarrow A^j(x^j; r^j)$. Prover sends $\{a^j\}_{j \in [n]}$ to Verifier.
- Step $2i$. Verifier samples $c_i \xleftarrow{\$} S_i$, and sends it to Prover.
- Step $2i + 1$. Prover completes shares by $\{c_i^j\}_{j \in [n]} \leftarrow \text{Complete}_{i, \Gamma^*}(c_i, \{c_i^j\}_{j \in \bar{A}})$, and computes $z_i^j \leftarrow Z^j(x^j, w^j, \{c_m^j\}_{m \in [i]}; r^j)$ for all $j \in A$. It then sends $\{c_i^j, z_i^j\}_{j \in [n]}$ to Verifier.
- Step final. Verifier runs $V^j(x^j, a^j, \{c_i^j\}_{i \in [\mu]}, \{z_i^j\}_{i \in [\mu]})$ for $j \in [n]$ and $\text{CheckShares}_{i, \Gamma^*}(c_i, \{c_i^j\}_{j \in [n]})$ for $i \in [\mu]$, and outputs 1 if all outputs are 1, outputs 0, otherwise.

It is shown in [CDS94] that, if every Π^j is a 3-move public-coin proof protocol that is 2-special sound and honest verifier zero-knowledge, and Γ admits a smooth perfect secret sharing scheme, then the above protocol $\Pi_{\mu, \Gamma}^{\text{cds}'}$ is a Σ -protocol for relation R_Γ . If every Π^j is honest verifier zero-knowledge, the above can be augmented to accept an access structure Γ that admits a smooth secret sharing scheme. It is done by modifying the prover's first step in a way that it first runs an HVZK simulator to directly obtain challenge c_i^j for $j \in \bar{A}$.

When $\mu \geq 2$, however, $\Pi_{\mu, \Gamma}^{\text{cds}'}$ is not sound anymore. Consider a simple disjunctive access structure $\Gamma = \{\{A\}, \{B\}, \{A, B\}\}$. A cheating prover who knows witnesses for *both* A and B could strategically decide, round by round, whether to fix the challenge for the A -part or the B -part. For example, it might fix c_1^A and c_2^B . When an extractor tries to rewind this prover to get a full tree of transcripts (required for multi-round special soundness), it immediately fails. Rewinding on challenge c_1 would never change c_1^A (it was fixed by the prover), and rewinding on c_2 would never change c_2^B . Consequently, a complete extraction tree for either A or B cannot be formed, breaking the soundness of the protocol.

B Composition of Multi-Round Protocols via Share-then-Hash

To support the composition of an arbitrary polynomial number of instances, we generalize the Share-then-Hash method proposed in [AAB⁺20] with the help of a collision resistant hash (CRH) $H : \{0, 1\}^* \mapsto \mathcal{C}_{i^*}$. Intuitively, rather than straightly use sampled values s^j as challenges $c_{i^*}^j = s^j$ for simulated transcripts, the prover is now required to obtain challenges $c_{i^*}^j \leftarrow H(\Gamma, \vec{x}, j, s^j)$ from the hash function.

The incorporation of CRH into protocol design introduces an inherent risk of extraction failure due to the adversary's potential successful guess and the hash collisions. Conversely, the security loss associated with perfect composition - previously exponential in nature - can be substantially reduced to a polynomial factor. To accommodate this probabilistic limitation while maintaining security guarantees, we adapt the *predicate special soundness* framework proposed in [AAB⁺24] and recall its relation to Fiat-Shamir knowledge soundness. Our composition with Share-then-Hash is presented in Figure 2. The security of the protocol is stated in Theorem 3.

B.1 Extending Predicate Special Soundness

We first recall the notations from [AAB⁺24].

Definition 10 (Set of Partial Accepting Trees). Let $\mu, k_1, \dots, k_\mu \in \mathbb{N}$ and let $\Pi = (P, V)$ be a $(2\mu + 1)$ -move public-coin proof for a relation R . Additionally, let $m \in [\mu]$ and $\ell \in [k_m]$. Let $\mathcal{C}_m$ be the m -th challenge set.

- We define $\mathbb{T}_{\mu+1}$ be the set of possible accepting transcripts for Π .
- We define $\mathbb{T}_{m+1}^{(\ell)}$ be the set of possible accepting $(1, \dots, 1, \ell, k_{m+1}, \dots, k_\mu)$ -trees of transcripts for Π , and denote $\mathbb{T}_m = \mathbb{T}_{m+1}^{(k_m)}$, i.e., the set of possible accepting $(1, \dots, 1, k_m, \dots, k_\mu)$ -trees. For each tree $t \in \mathbb{T}_{m+1}^{(\ell)}$, t can be parsed as a tuple of ℓ sub-trees $t = (t_1, \dots, t_\ell) \in \mathbb{T}_{m+1}^\ell$.
- For $t \in \mathbb{T}_{m+1}$, we define $\text{trunk}(t)$ to be the prefix $(a, c_1, z_1, \dots, c_m, z_m)$ shared by all the transcripts in t , and $\text{chal}_i(t)$ for $i \in [m]$ to be the i -th challenge $c_i \in \mathcal{C}_i$ shared by the transcripts.
- Let $\mathcal{C}_m^{(\ell)}$ be the set of tuples $(c_1, \dots, c_\ell) \in \mathcal{C}_m$ with $c_{i_1} \neq c_{i_2}$ for all $i_1 \neq i_2$. These are all the combinations of m -th challenges that may occur in $\mathbb{T}_{m+1}^{(\ell)}$.

To extend the notion of special soundness, [AAB⁺24] introduced *challenge predicates* for a set of challenges. Intuitively, their challenge predicate checks whether a subset of challenges defining branches in the same level satisfies desired properties for extracting witness. In slightly more detail, the ℓ -th challenge predicate on each level takes the first $\ell - 1$ challenges $c_1, \dots, c_{\ell-1}$, and checks that the current challenge c_ℓ falls in a “good” domain of challenges determined by the first $\ell - 1$ challenges. While this definition was sufficient for analyzing lattice-based proof systems in [AAB⁺24], it turns out that the analysis of Share-then-Hash demands challenge predicates with a different format of inputs. Looking ahead, a set of good challenges for Share-then-Hash is determined by the first $\ell - 1$ responses $z_1, \dots, z_{\ell-1}$ following the challenges $c_1, \dots, c_{\ell-1}$. We therefore generalize the notion of challenge predicates in such a way that they take *subtrees* $t_1, \dots, t_{\ell-1}$ as input, encompassing response-dependent good challenge domains.¹² Although the analysis of Share-then-Hash does not require entire subtrees, we provide a more general form of definition.¹³ Furthermore, while [AAB⁺24] required every predicate to be efficiently computable, their knowledge extractor for Fiat-Shamir-transformed protocols [AAB⁺24, Appendix H, Extractor 1] in fact only needs the ability to check whether a chain of level m predicates $\Phi_{m,1}^{\text{chal}}, \dots, \Phi_{m,k_m}^{\text{chal}}$ simultaneously output 1 on a fixed set of subtrees. Therefore, we limit the computability requirement to a chain of predicates, rather than assuming every predicate is efficiently checkable. The definition below is of independent interest and may enable the analysis of other multi-round protocols with more complex extraction criteria.

Definition 11 (Generalized Challenge Predicates). Let $m \in [\mu]$ and $\ell \in [k_m]$. A generalized challenge predicate on level m for the ℓ -th challenge is a deterministic function $\Phi_{m,\ell}^{\text{chal}} : \mathbb{T}_{m+1}^{(\ell-1)} \times \mathcal{C}_m \rightarrow \{0, 1\}$, where the first input is assumed to be empty if $\ell = 1$. We say that a chain of level m challenge predicates is efficiently checkable if there exists a polynomial-time algorithm that, on input $(t_1, \dots, t_{k_m}) \in \mathbb{T}_m$, checks whether there exists $\ell \in [k_m]$ such that $\Phi_{m,\ell}^{\text{chal}}(t_1, \dots, t_{\ell-1}, c_\ell) = 0$.

Definition 12 (Failure Density of Generalized Challenge Predicates). Let $m \in [\mu]$ and $\ell \in [k_m]$. Define a set of good subtrees

$$\text{GoodTree}_{m,\ell-1} = \left\{ (t_1, \dots, t_{\ell-1}) \in \mathbb{T}_{m+1}^{(\ell-1)} \mid \forall i \in [\ell-1] : \Phi_{m,i}^{\text{chal}}(t_1, \dots, t_{i-1}, c_i) = 1 \right\}$$

using the shorthand $c_i = \text{chal}_m(t_i)$. For fixed $\ell - 1$ subtrees in $\mathbb{T}_{m+1}^{(\ell-1)}$, consider the set of bad ℓ -th challenges such that $\Phi_{m,\ell}^{\text{chal}}$ fails,

$$\text{BadChal}_{m,\ell}(t_1, \dots, t_{\ell-1}) = \{c \in \mathcal{C}_m \setminus \{c_1, \dots, c_{\ell-1}\} \mid \Phi_{m,\ell}^{\text{chal}}(t_1, \dots, t_{\ell-1}, c) = 0\}.$$

The challenge predicate $\Phi_{m,\ell}^{\text{chal}}$ has failure density $p_{m,\ell}^{\text{chal}}$ if it always holds that $|\text{BadChal}_{m,\ell}(t_1, \dots, t_{\ell-1})| \leq p_{m,\ell}^{\text{chal}} |\mathcal{C}_m|$ for all $(t_1, \dots, t_{\ell-1}) \in \text{GoodTree}_{m,\ell-1}$.

¹² We remark the idea here is similar in spirit to the *useful challenge structure* of Attema et al. [AKLY24], where a set of useful challenges are determined with respect to a sequence of transcripts that have been found.

¹³ It is more general in the sense that one can always define a predicate that examines part of the subtree relevant to determination of good challenges.

In the following, we adapt the predicate system of [AAB⁺24] with generalized challenge predicates, while omitting commitment predicates.

Definition 13 (System of Predicates). A predicate system Φ for a $(k_1, \dots, k_\mu)$ -tree structure is a collection of predicates for each level in the tree. The m -th level has k_m challenge predicates $\Phi_{m,1}^{\text{chal}}, \dots, \Phi_{m,k_m}^{\text{chal}}$. We recursively define a series of predicate Φ_m for $m \in [\mu + 1]$, describing whether a partial tree of transcripts satisfies the predicate system. For a single accepting transcript $t \in \mathbb{T}_{\mu+1}$ we let $\Phi_{\mu+1}(t) = 1$. For all larger subtrees $t = (t_1, \dots, t_{k_m}) \in \mathbb{T}_m$ for some $m \in [\mu]$, then $\Phi_m(t) = 1$ if and only if

$$\bigwedge_{\ell \in [k_m]} (\Phi_{m+1}(t_\ell) = 1 \wedge \Phi_{m,\ell}^{\text{chal}}(t_1, \dots, t_{\ell-1}, c_\ell) = 1).$$

For notational convenience, we let $\Phi = \Phi_1$.

Definition 14 (Predicate Special Soundness). Let $\Pi = (P, V)$ be a $(2\mu + 1)$ -message public-coin argument for a relation R . We say that Π is $(\vec{k}, \Phi)$ -predicate-special-sound for $\vec{k} = (k_1, \dots, k_\mu)$ and a predicate system Φ if there exists a polynomial time algorithm which given a statement x and a $\vec{k}$ -tree of accepting transcripts t for this statement with $\Phi(t) = 1$ always outputs a witness w such that $R(x, w) = 1$.

Adapting Theorem 5.1 of [AAB⁺24], we immediately obtain the following theorem on Fiat-Shamir knowledge soundness.

Theorem 5 ([AAB⁺24] (adapted)). Let $\Pi = (P, V)$ be a $(\vec{k}, \Phi)$ -predicate-special-sound argument for a relation R with the corresponding failure density $\{p_{m,\ell}^{\text{chal}}\}_{m \in [\mu], \ell \in [k_m]}$. If a chain of level m challenge predicates $\Phi_{m,1}^{\text{chal}}, \dots, \Phi_{m,k_m}^{\text{chal}}$ is efficiently checkable for all $m \in [\mu]$, then the Fiat-Shamir transformation $\text{FS}[\Pi]$ is adaptively knowledge sound for the relation R with knowledge error

$$2(Q + 1) \sum_{i=1}^{\mu} \max \left(\frac{k_i - 1}{|\mathcal{C}_i|}, \sum_{\ell=1}^{k_i} p_{i,\ell}^{\text{chal}} \right),$$

where Q is the number of random oracle queries made by the prover. The number of times that the knowledge extractor invokes the prover is in expectation at most $K + Q(K - 1)$, where $K = \prod_{i=1}^{\mu} k_i$.

Remark 4. While our generalized challenge predicate now takes subtrees as inputs instead of individual challenges, this modification does not affect the knowledge error analysis of [AAB⁺24]. This is because their extractor [AAB⁺24, Appendix H, Extractor 1]—which is itself adapted from [AFK22]—always obtains the first $\ell - 1$ subtrees (each containing one challenge in the current level) in a depth-first manner before sampling ℓ -th challenge. Thus, the extractor’s failure probability can be conditioned on the existing $\ell - 1$ subtrees rather than on $\ell - 1$ individual challenges.

B.2 Analysis of Share-then-Hash in the Predicate Special Soundness Framework

To see why we need generalized challenge predicates, we first sketch the special soundness proof of Share-then-Hash. As in the proof of multi-round CDS, we set $k'_i = k_i$ for $i > i^*$ and $k'_i = n(k_i - 1) + 1$ for $i < i^*$ while parameterizing the number of critical round branches by $k'_{i^*} = \tau \geq n(k_{i^*} - 2) + 2$. Now, consider the set $\mathbb{T}_{i^*}$ of possible $(1, \dots, 1, k'_{i^*}, \dots, k'_\mu)$ -trees for the protocol Π_F^{sth} . Fix a subtree $t = (t_1, \dots, t_\tau) \in \mathbb{T}_{i^*}$, where the i^* -th level challenge-response of each t_ℓ is denoted by $(s_\ell, \{z_\ell^j, s_\ell^j\}_{j \in [n]})$. (Note we omit the subscript i^* here.) Essentially, for witness extraction to succeed, we require that (1) $\text{CheckShares}_{\Gamma^*}(s_\ell, \{s_\ell^j\}_{j \in [n]}) = 1$ for all $\ell = 1, \dots, \tau$, and (2) for all $B \in \Gamma^*$, there exists $j \in B$ such that $|\{s_1^j, \dots, s_\tau^j\}| \geq k_{i^*}$. (1) is already satisfied for any accepting transcript. Once (2) is satisfied, we can then rely on Proposition 5 to show the existence of a qualified set $A \in \Gamma$ by setting $A = \bigcup_{B \in \Gamma^*} \{j \in B : |\{s_1^j, \dots, s_\tau^j\}| \geq k_{i^*}\}$. This implies that for each $j \in A$, one can find at least k_{i^*} distinct shares out of τ branches. Assuming collision-resistance of the hash function H , we can then conclude that there are at least k_{i^*} distinct challenges for each $j \in A$. In what follows, we formally define $\Phi_{i^*,1}^{\text{chal}}, \dots, \Phi_{i^*,\tau}^{\text{chal}}$ such that (2) is satisfied if they all output 1.

Generalized Challenge Predicates and Failure Densities Let us denote a set $S_\ell^j = \{s_1^j, \dots, s_\ell^j\}$ of j -th shares for given subtrees $(t_1, \dots, t_\ell) \in \mathbb{T}_{i^*+1}^{(\ell)}$, and $\text{Rec}_{\Gamma^*}(\{S_\ell^j\}_{j \in B})$ as a shorthand for $\{\text{Rec}_{\Gamma^*}(\{s^j\}_{j \in B})\}_{s^j \in S_\ell^j}$. Intuitively, this denotes a set of secrets reconstructed from every possible combination of $|B|$ shares chosen from $\prod_{j \in B} S_\ell^j$. It is then easy to see that $|\text{Rec}_{\Gamma^*}(\{S_\ell^j\}_{j \in B})| = \prod_{j \in B} |S_\ell^j|$. We now define the following generalized challenge predicates:

- For $\ell = 1$, $\Phi_{i^*, \ell}^{\text{chal}}(s_\ell)$ returns 1 for any $s_\ell \in S$
- For $1 < \ell \leq k'_{i^*}$, where $k'_{i^*} = \tau \geq n(k_{i^*} - 2) + 2$, $\Phi_{i^*, \ell}^{\text{chal}}(t_1, \dots, t_{\ell-1}, s_\ell)$ returns 1 if the following conditions are satisfied and returns 0 otherwise:

$$\forall B \in \Gamma^* : \left(\exists j \in B : |S_{\ell-1}^j| \geq k_{i^*} \right) \vee s_\ell \notin \text{Rec}_{\Gamma^*}(\{S_{\ell-1}^j\}_{j \in B}) \quad (2)$$

Before proving predicate special soundness, we prove the following utility lemma. By Lemma 1, it is also easy to see that a chain of the above predicates is efficiently checkable. That is, given $(t_1, \dots, t_\tau) \in \mathbb{T}_{i^*}$, one can efficiently check if there exists a failing predicate by checking $\{j \in [n] : |S_\tau^j| \geq k_{i^*}\} \in \Gamma$.

Lemma 1. *Let $(t_1, \dots, t_\tau) \in \mathbb{T}_{i^*}$ be a $(1, \dots, 1, \tau, k_{i^*+1}, \dots, k_\mu)$ -tree of accepting transcripts and let $\tau \geq n(k_{i^*} - 2) + 2$. If $\Phi_{m, \ell}^{\text{chal}}(t_1, \dots, t_{\ell-1}, s_\ell) = 1$ for all $\ell \in [\tau]$, where s_ℓ is the i^* -th round challenge of t_ℓ , then there exists a qualified set $A \in \Gamma$ such that for all $j \in A$, $|S_\tau^j| \geq k_{i^*}$.*

Proof. Let $\{s_\ell^j\}_{j \in [n]}$ be a set of shares that reconstructs to the critical-round challenge s_ℓ in t_ℓ . Since each t_ℓ is accepting, we have that $\text{CheckShares}_{\Gamma^*}(s_\ell, \{s_\ell^j\}_{j \in [n]}) = 1$ for every $\ell \in [\tau]$. We show that, for every $B \in \Gamma^*$, there exists $j \in B$ such that $|S_\tau^j| \geq k_{i^*}$. If the predicate $\Phi_{i^*, \ell}^{\text{chal}}$ outputs 1 by satisfying the first condition of (2) for some ℓ , then this statement is trivially true. Suppose the predicate outputs 1 by satisfying the second condition of (2) for all $\ell \in [\tau]$. For any $\ell > 1$ and $B \in \Gamma^*$, we have that $s_\ell = \text{Rec}_{\Gamma^*}(\{s_\ell^j\}_{j \in B})$ by the consistency testing property of SSS_{Γ^*} . If $s_\ell \notin \text{Rec}_{\Gamma^*}(\{S_{\ell-1}^j\}_{j \in B})$, then it must be that $s_\ell^j \notin S_{\ell-1}^j$ for some $j \in B$. Thus we guarantee that the sum $\sum_{j \in B} |S_\ell^j|$ is monotonically increasing with respect to ℓ . Since $\sum_{j \in B} |S_1^j| = |B|$, we have that

$$\sum_{j \in B} |S_\tau^j| \geq |B| + \tau - 1.$$

Thus, by setting $|B| + \tau - 1 \geq |B|(k_{i^*} - 1) + 1$, we can guarantee that for any $B \in \Gamma^*$, there exists some $j \in B$ such that $|S_\tau^j| \geq k_{i^*}$. Since $|B| \leq n$, we can set $\tau \geq n(k_{i^*} - 2) + 2$.

Finally, let $A = \bigcup_{B \in \Gamma^*} J_B$, where J_B denotes a set of $j \in B$ such that $|S_\tau^j| \geq k_{i^*}$. As each J_B is guaranteed to be non-empty, we have that $A \cap B \neq \emptyset$ for every $B \in \Gamma^*$, which implies $A \in \Gamma$ by Proposition 5. $\square$

Next, we find the failure density of the above predicates.

Lemma 2. *The level i^* challenge predicates $\Phi_{i^*, 1}^{\text{chal}}, \dots, \Phi_{i^*, k'_{i^*}}^{\text{chal}}$ have the following failure densities:*

$$p_{i^*, \ell}^{\text{chal}} = \begin{cases} 0 & \text{if } \ell = 1 \\ \frac{2^{n \cdot (k_{i^*} - 1)^n}}{|S|} & \text{if } 1 < \ell \leq k'_{i^*} \end{cases}$$

Proof. The case $\ell = 1$ is trivial. For $1 < \ell \leq k'_{i^*}$, we first fix $\ell - 1$ “good” subtrees $(t_1, \dots, t_{\ell-1}) \in \text{GoodTree}_{i^*, \ell-1}$. If $\Phi_{i^*, \ell-1}^{\text{chal}}$ outputs 1 by meeting the first condition of (2) for some $B \in \Gamma^*$, then the subsequent $\Phi_{i^*, \ell}^{\text{chal}}$ always outputs 1 for the same B regardless of s_ℓ . Thus, to find the upper bound on the size of bad challenges, we assume that for all $B \in \Gamma^*$, $s_{\ell-1} \notin \text{Rec}_{\Gamma^*}(\{S_{\ell-2}^j\}_{j \in B})$. By the definition of $\Phi_{i^*, \ell}^{\text{chal}}$, we have that

$$\begin{aligned} & |\text{BadChal}_{i^*, \ell}(t_1, \dots, t_{\ell-1})| \\ & \leq \sum_{B \in \Gamma^*} \left| \left\{ s \in S : (\forall j \in B : |S_{\ell-1}^j| \leq k_{i^*} - 1) \wedge s \in \text{Rec}_{\Gamma^*}(\{S_{\ell-1}^j\}_{j \in B}) \right\} \right| \\ & \leq \sum_{B \in \Gamma^*} (k_{i^*} - 1)^{|B|} \leq 2^n \cdot (k_{i^*} - 1)^n. \end{aligned}$$

Thus, we obtain the failure density $p_{i^*, \ell}^{\text{chal}} = 2^n \cdot (k_{i^*} - 1)^n / |S|$. $\square$

Analysis of Predicate Special Soundness We are now ready to prove our main result, Theorem 3.

Proof. Completeness. It directly follows from the completeness and critical-round special honest verifier zero-knowledge of Π^j . We note that $\text{Complete}_{\Gamma^*}$ works for the set $\bar{A}$ of shares $\{s^j\}_{j \in \bar{A}}$ since $\bar{A} \notin \Gamma^*$ when $A \in \Gamma$.

Critical-Round Special Honest Verifier Zero-Knowledge. Consider the following simulator for $\Pi_{\Gamma}^{\text{sth}}$ that proceeds as follows, given as input $\{x^j\}_{j \in [n]} \in \mathcal{L}_{R_{\Gamma}}$, $\{c_i\}_{i \neq i^*}$ and $s \in S$:

1. Compute $\{s^j\}_{j \in [n]} \leftarrow \text{Share}_{\Gamma^*}(s)$
2. For all $j \in [n]$: $c_{i^*}^j = H(\Gamma, \vec{x}, j, s^j)$ and $(st_i^j, a^j) \leftarrow \text{SimA}^j(x^j, c_{i^*}^j)$
3. For all $i \in [\mu]$ and $j \in [n]$: $(st_{i+1}^j, z_i^j) \leftarrow \text{SimZ}^j(st_i^j, c_i^j)$, where $c_i^j = c_i$ if $i \neq i^*$
4. Output $(\{a^j\}_{j \in [n]}, \{c_i\}_{i \in [\mu] \setminus \{i^*\}}, \{z_i^j\}_{i \in [\mu], j \in [n]}, \{s^j\}_{j \in [n]}, s)$

The perfect hiding and share completion properties of SSS_{Γ^*} imply that the distribution of $\{s^j\}_{j \in [n]}$ generated as above is identical to that of a real transcript. Since $\{x^j\}_{j \in [n]} \in \mathcal{L}_{R_{\Gamma}}$, for $j \in A$, the simulator generates a^j, z_i^j exactly as a real prover; for $j \in A$, the distribution of a^j, z_i^j output by CRSHVZK simulators for Π^j is indistinguishable with the one generated by a real prover. Grouping Step 1-2 together as SimA and Step 3 as SimZ, respectively, the above algorithm is indeed a valid CRSHVZK simulator.

Predicate Special Soundness. The proof can be viewed as generalization of [CDS94]. However, we must carefully choose the special soundness parameters for $\Pi_{\Gamma}^{\text{sth}}$ in such a way that the corresponding tree of accepting transcripts contains a well-structured tree for every $j \in A \in \Gamma$ from which the extractor Ext^j for Π^j can successfully extract witness for x^j . We construct an efficient extractor Ext^{sth} that determines some $A \in \Gamma$ and then internally runs sub-extractors $\{\text{Ext}^j\}_{j \in A}$ to output a valid set of witnesses $\mathbf{w} = \{w^j\}_{j \in A}$, given as input any $\vec{k}'$ -tree T^{sth} of accepting transcripts.

We first recall the structure of the tree T^{sth} for $\Pi_{\Gamma}^{\text{sth}}$. Each path in T^{sth} defines a transcript

$$(\{a^j\}_{j \in [n]}, \{c_i\}_{i \in [\mu] \setminus \{i^*\}}, \{z_i^j\}_{i \in [\mu], j \in [n]}, s, \{s^j\}_{j \in [n]})$$

which can be decomposed into n individual transcripts:

$$\{(a^j, \{c_i\}_{i \in [\mu] \setminus \{i^*\}}, c_{i^*}^j, \{z_i^j\}_{i \in [\mu]})\}_{j \in [n]}$$

where $c_i^j = c_i$ for $i \neq i^*$ and $c_{i^*}^j = H(\Gamma, \vec{x}, j, s^j)$ for $j \in [n]$. Thus, T^{sth} can also be decomposed into n individual trees, $T^1, \dots, T^n$, where every path of T^j is derived from decomposition of the corresponding path in T^{sth} . We first observe the following properties:

1. In non-critical rounds $i \neq i^*$, the challenges in the decomposed trees T^j for $j \in [n]$ remain the same as the corresponding challenges in T^{sth} .
2. In round i^* , the shares s^j in the decomposed trees T^j for $j \in [n]$ pass the consistency check performed by $\text{CheckShares}_{\Gamma^*}$ with respect to the corresponding secret s in T^{sth} .

The analyses of Case 1 ($i^* < i \leq \mu$) and Case 3 ($i < i^*$) are identical to those of Theorem 2.

To prove Case 2 ($i = i^*$), we show the following statement: if $k_{i^*}' = \tau \geq n(k_{i^*} - 2) + 2$, then for any $(1, \dots, 1, k_{i^*}', k_{i^*+1}, \dots, k_{\mu})$ -subtree $T_{i^*}^{\text{sth}} = (t_1, \dots, t_{k_{i^*}'})$ for $\Pi_{\Gamma}^{\text{sth}}$ such that $\Phi_{i^*, \ell}^{\text{chal}}(t_1, \dots, t_{\ell-1}, s_{\ell}) = 1$ for $\ell = 1, \dots, k_{i^*}'$, there exists $A \in \Gamma$ such that for every $j \in A$, the corresponding decomposed subtree $T_{i^*}^j$ contains a $(1, \dots, 1, k_{i^*}, \dots, k_{\mu})$ -subtree $\tilde{T}_{i^*}^j$ for statement x^j . By Lemma 1, we have that there exists $A \in \Gamma$ such that for all $j \in A$, the corresponding shares $s_1^j, \dots, s_{k_{i^*}'}^j$ are pairwise distinct. Thus, assuming that the outputs $c_{i^*, \ell}^j$ of the hash function $H(\Gamma, \vec{x}, j, s_{\ell}^j)$ never collide for distinct inputs, for each $j \in A \in \Gamma$, one can derive from $T_{i^*}^j$ a $(1, \dots, 1, k_{i^*}, \dots, k_{\mu})$ -subtree $\tilde{T}_{i^*}^j$ of accepting transcripts for statement x^j .

Extractor. Combining Case 1-3 above, there exists $A \in \Gamma$ such that for every $j \in A$, the corresponding decomposed tree T^j contains a $\vec{k}$ -tree $\tilde{T}^j$ for Π^j . The extractor Ext^{sth} then simply runs sub-extractors Ext^j on $\tilde{T}^j$ to extract witnesses for all $j \in A$. As Π^j is $\vec{k}$ -SS, every Ext^j always succeeds in extracting witness.

□

C Other Definitions

Definition 15 (Honest-Verifier Zero-Knowledge). *An interactive proof system is honest-verifier zero-knowledge if there exists a polynomial-time algorithm Sim (simulator) that, for any $(x, w) \in \mathcal{L}_{RW}$, distribution of outputs from $\text{Sim}(x)$ and that of transcripts observed in $\langle P(w), V \rangle(x)$ are indistinguishable.*

For more notation related to the $(k_1, \dots, k_\mu)$ -tree of transcripts T . Outside the notations introduced in Section 3.2, some extra notations are introduced below to support other definitions. By $\text{nonleaves}(T)$, we denote all node indices of T except for the leaves. Let $\text{nodes}(T, i)$ denote the set of node indices in depth i of T , e.g., $\text{nodes}(T, 2) = \{(1, 1), (1, 2), \dots, (1, k_1)\}$. For every $\mathcal{C}_i$ and $k_i > 1$, we denote sets of pairwise distinct k_i challenges by

$$\mathcal{C}_{k_i}^{\text{dis}} := \{(c_1, \dots, c_{k_i}) \mid \forall j, \ell (\ell \neq j) \in [k_i], (c_j, c_\ell) \in \mathcal{C}_i \times \mathcal{C}_i, c_j \neq c_\ell\}.$$

Let $\text{pnun}(T, i)$ denote the number of parent nodes at level i , i.e., $\text{pnun}(T, i) = \prod_{j=1}^i k_{j-1}$ for $k_0 = 1$.

Definition 16 (Knowledge Soundness). *An interactive proof system for relation R is knowledge sound with knowledge error ϵ_{ks} if there exists an expected polynomial-time algorithm Ext_{ks} called an extractor that, for every algorithm P^* , every $x \in \{0, 1\}^\lambda$, and $\text{aux} \in \{0, 1\}^*$,*

$$\Pr[w \leftarrow \text{Ext}_{\text{ks}}^{P^*(\text{aux})}(x) : R(x, w) = 1] \geq \frac{\Pr[\langle P^*(\text{aux}), V \rangle(x) = 1] - \epsilon_{\text{ks}}(\lambda)}{\text{poly}(\lambda)}.$$

Definition 17 (Statistical $(k_1, \dots, k_\mu)$ -Special Soundness [ABO⁺24]). *A $(2\mu+1)$ -round public-coin proof protocol is statistical $(k_1, \dots, k_\mu)$ -special sound with knowledge error ϵ_{stss} if there exists a polynomial-time algorithm Ext such that for any adversary $\mathcal{A}$, it holds that:*

$$\Pr \left[\begin{array}{l} T \text{ is a } (k_1, \dots, k_\mu)\text{-tree} \\ \text{of accepting transcripts} \\ \wedge \left(\forall i \in [\mu-1], \forall id \in \overrightarrow{[k_i]}, \right. \\ \quad \left. \text{chal}(T, id) = \{c_{id||j}\}_{j \in [k_{i+1}]} \right) \\ \wedge R(x, w) = 0 \end{array} : \begin{array}{l} \forall i \in [\mu], \forall id \in \overrightarrow{[k_i]}, c_{id} \leftarrow \mathcal{C}_i \\ (x, T) \leftarrow \mathcal{A}(\{c_{id}\}_{id \in \overrightarrow{[k_\mu]}}) \\ w \leftarrow \text{Ext}(x, T) \end{array} \right] \leq \epsilon_{\text{stss}},$$

where $\overrightarrow{[k_i]} = 1 \times [k_1] \times \dots \times [k_i]$. A tree of transcript is called “bad” if the extractor fails.

It is stressed that error bound ϵ_{stss} is independent of messages from the prover. It is shown in [ABO⁺24] that, a parallel repetition of $(k_1, \dots, k_\mu)$ -special sound protocol results in a statistical $(k_1, \dots, k_\mu)$ -special sound protocol with negligible statistical soundness error. They also showed that it constitutes a proof of knowledge system.

Definition 18 (Round-by-Round Soundness [CCH⁺19, FGQ⁺23]). *Let $\Pi = (\mathcal{A}, \mathcal{Z}, \mathcal{V})$ be $(2\mu+1)$ -move public-coin proof protocol. We say that Π is round-by-round sound if, there exists a “doomed set” $\mathcal{D} \in \{0, 1\}^*$ such that,*

- If $x \notin \mathcal{L}$, then $(x, \emptyset) \in \mathcal{D}$, where $\emptyset$ denotes the empty transcript.
- For all (2ℓ) -move partial transcript $\tau = (a, c_1, z_1, \dots, z_{\ell-1}, c_\ell)$, such that $(x, \tau) \in \mathcal{D}$, for all next message z_ℓ given by the prover, there exists a negligible function $\text{negl}(\cdot)$ such that

$$\Pr_{c_{\ell+1} \leftarrow \mathcal{C}_{\ell+1}} [(x, \tau, z_\ell, c_{\ell+1}) \notin \mathcal{D}] \leq \text{negl}(\lambda).$$

- For any x , any (2μ) -move partial transcript τ and any last prover message z_μ , if $(x, \tau) \in \mathcal{D}$ then $V(x, \tau, z_\mu) = 0$.

A Σ -protocol [Cra96] is a three-move public-coin proof protocol that is 2-special sound and special honest verifier zero-knowledge.

Definition 19 (Predicate Special Soundness [AAB⁺24]). *Let $\Pi = (\mathcal{P}, \mathcal{V})$ be a $2\mu+1$ -message public-coin argument of knowledge for a relation R_{PP} . We say that Π is $(\mathbf{K}, \Phi)$ -predicate-special sound for $\mathbf{K} = (\mathbf{k}_1, \dots, \mathbf{k}_\mu)$ and a predicate system Φ if there exists a polynomial time algorithm which given a statement x and a $\mathbf{K}$ -tree of transcripts for this statement with $\Phi(\mathbf{t}) = \mathbf{1}$ always outputs a witness w such that $w \in R_{\text{PP}}(x)$.*

Definition 20 (Special Unsoundness [AFK23, BGTZ25]). Let $\Pi = (\mathcal{P}, \mathcal{V})$ be a $(2\mu + 1)$ -move public-coin proof protocol, and let $(\ell_1, \dots, \ell_\mu) \in \mathbb{N}^\mu$. We say that Π has $(\ell_1, \dots, \ell_\mu)$ -special unsoundness if there exists a dishonest prover $\mathcal{A}$ of the following form and, so that in the execution with $\mathcal{V}$ and input x the following holds:

- $\mathcal{A}$ starts off in active mode, which is so that in every round i , when $\mathcal{A}$ sends the message m_i , there exists a subset $\mathcal{L}_i \subseteq \mathcal{C}_i$ such that $|\mathcal{L}_i| = \ell_i$ (defined as a function of the state of $\mathcal{A}$ at that point) such that if the subsequent challenge c_i is in $\mathcal{L}_i$, then $\mathcal{A}$ switches into passive mode.
- If $\mathcal{A}$ switches into passive mode, then it remains in passive mode until the end of the protocol, and $\mathcal{V}$ accepts at the end of the protocol.

Definition 21 (k -Zero Knowledge [GKK⁺22, DG23]). Let $\Pi = (P, V)$ be a $(2\mu + 1)$ -move public-coin interactive proof protocol with HVZK simulator Sim , and $k \in [\mu]$. Let Π_{FS} be its associated FS-transformed NIZK. We say Π_{FS} satisfies (perfect) k -zero knowledge (k -ZK) if there exists a zero-knowledge simulator $\text{Sim}_{\text{FS}, k}$ that only needs to program the random oracle in round k , and whose output is identically distributed to that of honestly generated proofs.

Following the literature [AABN02, GOP⁺22, DG23], we exploit the concept of min-entropy to represent the likelihood of the first message a of an interactive protocol taking on some fixed value.

Definition 22 (Min-entropy of first message). Let λ be a security parameter and language $\mathcal{L}_R$ be an NP language associated by R . Consider a pair $(x, w) \in R$ and let $\Pi = (P, V)$ be a multi-round interactive proof. Let $\text{Coin}(\lambda)$ be the set of coins used by the prover and $A(\cdot)$ the algorithm that outputs the first message from P . Consider the set $A(x, w) = \{A(x, w; r) : r \leftarrow \text{Coin}(\lambda)\}$, i.e. the set of all possible first round messages associated to (x, w) . The min-entropy function associated to Π is defined as

$$\zeta(\lambda) = \min_{(x, w)} (-\log_2 \eta(x, w)),$$

where the minimum is taken over all possible (x, w) drawn from R , and $\eta(x, w)$ is the maximum probability that a first message takes on a particular value, i.e.,

$$\eta(x, w) = \max_{a \in A(x, w)} (\Pr[A(x, w; r) = a \mid r \leftarrow \text{Coin}(\lambda)]).$$

We say $\Pi = (P, V)$ has sufficient min-entropy if $\zeta(\lambda)$ is super-logarithmic in λ .

C.1 Relations to Other Notions

In this section, we compare CRZK with related notions, including its reduction to k -zero knowledge and its implication of k -special unsoundness.

CRZK leads to k -zero knowledge. We recap the notion of k -zero-knowledge (Definition 21) that was first introduced in [GKK⁺22] and formalized in [DG23]. Informally, an interactive proof is k -zero-knowledge if there exists a zero-knowledge simulator Sim_{FS} that only needs to program the random oracle in round k , and whose transcript is indistinguishable from honestly generated ones, stated as follows:

Theorem 6. Let Π be a $(2\mu + 1)$ -move public-coin proof protocol which is critical-round special honest verifier zero-knowledge at round i_{zk}^* and has sufficient min-entropy (Definition 22). Let Π_{FS} be the corresponding non-interactive proof protocol via Fiat-Shamir transform. Then Π_{FS} is k -zero knowledge with $k = i_{\text{zk}}^*$.

Proof. (Sketch) We construct a simulator, Sim_{FS} , that fulfills requirements in [DG23], as follows:

1. Given statement x as input, sample $c_{i_{\text{zk}}^*} \leftarrow \mathcal{C}_{i_{\text{zk}}^*}$ and obtain $(st_1, a) \leftarrow \text{SimA}(x, c_{i_{\text{zk}}^*})$.
2. Query the random oracle and get $c_i = H(x, a, z_1, \dots, z_{i-1})$ for all $i \neq i_{\text{zk}}^*$. Specially if $i = 1$, get $c_1 = H(x, a)$.
If $i = i_{\text{zk}}^*$, reprogram $H(x, a, z_1, \dots, z_{i_{\text{zk}}^*-1}) := c_{i_{\text{zk}}^*}$.
 Sim_{FS} aborts if $H(x, a, z_1, \dots, z_{i_{\text{zk}}^*-1})$ has already been defined.
3. Obtain $(st_{i+1}, z_i) \leftarrow \text{SimZ}(st_i, c_i)$.

Since Π has sufficient min-entropy, the probability that Sim_{FS} aborts in Step 2 is negligible. The distribution of $(a, c_1, z_1, \dots, c_\mu, z_\mu)$ is indistinguishable from that of honestly generated transcripts due to the critical-round special honest verifier zero-knowledge property. $\square$

k-CRZK and *k*-special unsoundness. Our *k*-CRZK implies *k*-special unsoundness [AFK23, BGTZ25] at the critical round i_{zk}^* for languages that constitute a hard subset membership problem [GW11]. Special unsoundness (Definition 20) claims the ability of a cheating prover to convince the verifier of a false statement if the challenge sent by the verifier is a bad challenge at certain round. We can build the cheating prover by running the *k*-CRZK simulator on $k - 1$ preselected challenges. If the verifier “unluckily” sends one of these $k - 1$ challenges at the critical round, the cheating prover is able to behave honestly in the remaining rounds. Due to the hardness of the subset membership problem with respect to the concerned language, the CRZK simulator works on false statements as required. The reverse implication does not generally hold since special unsoundness does not concern the output distribution of the cheating prover.

D Critical Rounds in the KKW Framework

D.1 KKW Framework

We succinctly describe the KKW framework that follows the MPCitH paradigm [IKOS07]. In this framework, the prover runs an MPC protocol that evaluates a boolean circuit C on an input w , commits to the views of all parties, and then opens all-but-one of these views to the verifier. The protocol relies on an XOR-based n -out-of- n secret sharing scheme; we denote shares of value a by $\llbracket a \rrbracket$. This MPC protocol, in the preprocessing model, is secure against semi-honest all-but-one corruptions. The protocol can be summarized as follows:

- For each wire α of circuit C , let $z_\alpha \in \{0, 1\}$ be the wire value. Each party holds a share $\llbracket \lambda_\alpha \rrbracket$ of a random value λ_α along with a masked value $\hat{z}_\alpha := z_\alpha + \lambda_\alpha$.
- The shares $\llbracket \lambda_\alpha \rrbracket$ are generated independent with the witness w in the *preprocessing phase*. A gate is defined by an input wire α, β , and an output wire γ . For an XOR gate parties can locally compute the share of output wire $\llbracket \lambda_{\alpha \oplus \beta} \rrbracket = \llbracket \lambda_\alpha \rrbracket \oplus \llbracket \lambda_\beta \rrbracket$, while for a multiplication gate, each party are given $\llbracket \lambda_{\alpha \cdot \beta} \rrbracket$ with the help of an auxiliary value that is given to n -th party and computed by $(\sum_{i=1}^n \llbracket \lambda_{\alpha_i} \rrbracket) \cdot (\sum_{i=1}^n \llbracket \lambda_{\beta_i} \rrbracket) - \sum_{i=1}^{n-1} \llbracket \lambda_{\alpha_i \cdot \beta_i} \rrbracket$.
- The set of shares $\{\llbracket \lambda_\alpha \rrbracket\}_{\alpha \in C}$ is generated by each i -th party in the preprocessing phase using a random seed referred to as state_i . For $1 \leq i \leq n - 1$, i -th party also uses state_i to generate the share of $\llbracket \lambda_{\alpha \cdot \beta} \rrbracket$, while n -th party uses auxiliaries aux_n as its shares.
- The masked shares $\{\hat{z}_\alpha\}$ of *input wire* of circuit and broadcast messages (denoted by msgs_i) of each of the parties allow for a *deterministic* online phase of the MPC protocol.

Next, we describe the resulting MPCitH protocol. To prevent the prover from cheating in the preprocessing phase, the prover must first generate and commit to m executions of the preprocessing stage, and later open all of them except one (corresponding to a verifier’s challenge). The unopened material is used for executing the MPC protocol later on. For clarity, we will omit the random coins of the commitment scheme in the description below. We assume that whenever a commitment is opened, the corresponding random coins are provided to the verifier. This results in a 5-move protocol as described below:

1. First, prover runs m independent executions of the preprocessing phase as follows. Prover samples m random values $\{\text{seed}_j\}_{j \in [m]}$. Then each seed_j is used to generate the set of $\{\text{seed}_{j,i}\}_{i \in [n]}$ together with randomness values (we omit these values for simplicity). For each $j \in [m]$, computing $\text{aux}_{j,n}$ as described above, defining $\text{state}_{j,i} := \text{seed}_{j,i}$ for $i \in [n - 1]$ and $\text{state}_{j,n} := \text{seed}_{j,n} \parallel \text{aux}_{j,n}$. Later, prover commits to each state as $\text{com}_{j,i} = \text{Com}(\text{state}_{j,i})$, each seed_j as $h_j = H(\text{com}_{j,1}, \dots, \text{com}_{j,c})$. Prover then sends $h := H(h_1, \dots, h_m)$ to verifier.
2. Verifier asks prover to open $m - 1$ of the preprocessing material, i.e., all except for the c_1 -th instance, where $c_1 \leftarrow \$[m]$.
3. Prover uses the unopened preprocessing material seed_{c_1} to get $\{\text{state}_{c_1,i}\}_{i \in [n]}$ and uses them to deterministically simulate the execution of MPC protocol. Finally, prover computes the masked values $\{\hat{z}_\alpha\}$ for the input wires (based on w and $\{\text{state}_{c_1,i}\}_{i \in [n]}$) and the broadcast messages from parties during execution $\{\text{msgs}_i\}_{i \in [n]}$. Prover sends $\{\text{seed}\}_{j \neq c_1}$ as the opening of all-but-one c -th processing materials and $\{\hat{z}_\alpha\}, h' := H(\text{msgs}_1, \dots, \text{msgs}_n)$ to verifier.

4. Verifier asks prover to open the views of all parties except for the c_2 -th party in the simulation of the MPC protocol, where $c_2 \leftarrow \$[n]$.
5. Prover opens the views by sending $\{\text{state}_{c_1,i}\}_{i \neq c_2}$ along with $\text{com}_{c_1,c_2}, \text{msgs}_{c_2}$ that can be used to verify the correctness of the MPC execution.

Finally, the verifier:

- For $i \neq c_2$, uses $\text{state}_{c_1,i}$ to compute $\text{com}_{c_1,i}$ and then combines these with com_{c_1,c_2} to compute $h_{c_1} = H(\text{com}_{c_1,1}, \dots, \text{com}_{c_1,n})$.
- For $j \neq c_1$, uses $\{\text{seed}\}_{j \neq c_1}$ to compute $\{h_j\}_{j \neq c_1}$. Then, uses h_{c_1} to check whether $h = H(h_1, \dots, h_m)$, otherwise outputting reject.
- From $\{\hat{z}_\alpha\}, \{\text{state}_{c_1,i}\}_{i \neq c_2}, \text{msgs}_{c_2}$, simulates the MPC protocol to get $\{\text{msgs}_i\}_{i \neq c_2}$ and the output bit b . If $b = 0$ then it outputs reject.
- Checks whether $h' \neq H(\text{msgs}_1, \dots, \text{msgs}_n)$ and outputs reject, otherwise accepts.

D.2 Critical Round in KKW

Theorem 7. *Given the 5-move interactive honest-verifier zero-knowledge proof in [KKW18] (denoted as KKW), assuming that the hash function used is collision-resistant and the commitment scheme used is computationally binding and hiding, then KKW is flexible and can be:*

- Critical-round special honest-verifier zero-knowledge at round $i_{zk}^* = 1$ or at round $i_{zk}^* = 2$.
- Extended $\vec{k}$ -special sound where $\vec{k} = (2, 2)$.

Proof. We prove the flexibility property of KKW with respect to critical-round HVZK. We first prove that KKW satisfies ExSS by constructing an extractor ExExt to show that KKW is extended $(2, 2)$ -special sound. For critical-round HVZK, we construct a pair of simulators (SimA, SimZ) that simulate the real execution in both cases, thereby satisfying the required indistinguishability conditions. We denote a transcript of KKW protocol by (a, c_1, z_1, c_2, z_2) .

ExSS.

Given an extended $(2, 2)$ -tree of transcripts T with four accepting transcripts that belong to T , our goal is to show that there exists an extended extractor ExExt that can either extract an actual witness, or find a collision on the commitment scheme or the hash function. We construct ExExt as below.

Given an accepted transcript (a, c_1, z_1, c_2, z_2) , the extractor ExExt works as follows:

- From z_1 , ExExt learns $\{\text{seed}_j\}_{j \neq c_1}$, h' and $\{\hat{z}_\alpha\}$.
- Consider another transcript $(a, c_1, z_1, c'_2, z'_2)$ with the same prefix (a, c_1, z_1) and $c_2 \neq c'_2$ since $c_2 \neq c'_2$ then from z'_2 , ExExt learns all $\{\text{state}_{c_1,i}\}_{i \in [n]}$. We note that, ExExt also learns $(\text{com}_{c_1,c_2}, \text{msgs}_{c_2})$ because they are included inside of z_2 .
- From $\{\text{state}_{c_1,i}\}_{i \in [n]}$, ExExt learns the value λ_α as the masks of each input wire, therefore from $\{\hat{z}_\alpha\}$, ExExt can effectively compute a witness w .

As in the first case, we argue that $C(w) = 1$. To prove this, we need to show that each $\{\text{state}_{c_1,i}\}_{i \in [n]}$ are generated honestly from a seed seed_{c_1} obtained during the preprocessing phase. ExExt does the following:

- Consider another accepting transcript $(a, c'_1, z'_1, *, *)$ of T where $c_1 \neq c'_1$. As $c_1 \neq c'_1$, the extractor ExExt can recover seed_{c_1} from z'_1 , as this value was generated honestly in the preprocessing phase.
- Given seed_{c_1} , ExExt computes $\{\text{state}_{c_1,i}\}_{i \in [n]}$, which must be consistent with the values outputted in the fifth move, z_2 . If this consistency fails, it would imply either a break in the binding property of the commitment scheme or a collision in the hash function H . In such a case, as a solution to R_{hd} , the extractor would output either a hash collision or the inputs required to break the commitment scheme, since all states are committed using a fixed value $a := h$.

Critical-round HVZK.

Case 1: $i_{zk}^* = 1$. The critical-round special HVZK at $i_{zk}^* = 1$ is proven by constructing two simulators $(\text{SimA}(x, c_1), \text{SimZ})$ as follows,

- $\text{SimA}(x, c_1) \rightarrow (st_1, a)$: For a known first-round challenge c_1 , the SimA does as follows:
 For all $j \in [m]$, SimA generates honestly as an honest prover to get all m preprocessing materials including $\text{seed}_j, \{\text{seed}_{j,i}\}_{i \in [n]}, \text{aux}_{j,n}$, defines $\text{state}_{j,i} := \text{seed}_{j,i}$ for $i \in [n-1]$ and $\text{state}_{j,n} := \text{seed}_{j,n} \parallel \text{aux}_{j,n}$.
 For $j = c_1$, SimA samples uniformly $\{\hat{z}_\alpha\}$ as masked value of input wire of circuit C . Together with $\{\text{state}_{c_1,i}\}_{i \in [n]}$, SimA executes the MPC protocol until the reconstruction output step. At this step Sim learns the shares of output wire $\llbracket b \rrbracket_i$ of all i -th party ($i \in [n]$), Sim then defines an auxiliary value aux_b such that $\llbracket b \rrbracket_i \oplus \text{aux}_b = 1$.
 SimA redefines the $\text{aux}_{c_1,n}$ as before, except that the auxiliary value corresponding to the last multiplication gate of C is now defined by XORing with aux_b and $\text{state}_{c_1,n}$ is redefined as $\text{seed}_{c_1,n} \parallel \text{aux}_{c_1,n}$ using new value of $\text{aux}_{c_1,n}$.
 Next, the SimA computes each value h_j for $j \in [m]$ as the honest prover would. Finally, the SimA computes $h = H(h_1, \dots, h_m)$ honestly. Output $st_1 := (\{\text{state}_{j,i}\}_{j \in [m], i \in [n]}, \{\hat{z}_\alpha\})$ and $a := h$.
- $\text{SimZ}(st_1, c_1) \rightarrow (z_1, st_2)$. To simulate the third message, SimZ starts by retrieving the states from st_1 . Then, it executes *honestly* online phase of MPC protocol to get the broadcast messages $\{\text{msgs}_i\}_{i \in [n]}$ using $\{\text{state}_{c_1,i}\}_{i \in [n]}$. Finally, it computes $h' = H(\text{msgs}_1, \dots, \text{msgs}_n)$ from the state. Output $st_2 := (st_1, \{\text{msgs}_i\}_{i \in [n]})$ and $z_1 := (\{\text{seed}_j\}_{j \neq c_1}, h', \{\hat{z}_\alpha\})$.
- $\text{SimZ}(st_2, c_2) \rightarrow z_2$: To simulate the fifth message, from st_2 , simply output, as the honest prover would do.
 Output $z_2 := (\{\text{state}_{c_1,i}\}_{i \neq c_2}, \text{com}_{c_1,c_2}, \text{msgs}_{c_2})$.

Correctness. We first show that the transcript $\{\text{seed}_{c_1,i}\}_{i \in [n]}, \{\hat{z}_\alpha\}, \text{aux}_{c_1,n}$ produced by the simulator allows the MPC protocol to output 1. We focus on the last multiplication gate of circuit C , where random shares are assigned to this gate. Denote the shares of its input wires as $\llbracket \lambda_\alpha \rrbracket$ and $\llbracket \lambda_\beta \rrbracket$, while the shares of the output wire are $\llbracket \lambda_{\alpha,\beta} \rrbracket$. For $i \in [n-1]$, the i -th party generates $\llbracket \lambda_{\alpha,\beta} \rrbracket_i$ from the random seed $\{\text{seed}_{c_1,i}\}$, and the n -th party uses $\text{aux}_{c_1,n}$ to define its share. By the way the auxiliary value is defined for this gate in SimA , the n -th party gets $\llbracket \lambda_{\alpha,\beta} \rrbracket_n \oplus \text{aux}_b$. Therefore, after all parties broadcast their shares and reconstruct them, they obtain a value $\hat{z}_\gamma = z_\gamma \oplus \lambda_\gamma \oplus \text{aux}_b$ (λ_γ is the masked value for this wire).

The online phase continues until all parties reconstruct the circuit's output. We note that after executing the last multiplication gate to obtain $\hat{z}_\gamma$, $\text{aux}_{c_1,n}$ is no longer used, and there are no broadcast messages from the parties until the final step of reconstructing the mask of the output value b , since the $\{\hat{z}_\alpha\}$ of addition gates can be computed locally. Therefore, aux_b is xored into all $\{\hat{z}_\alpha\}$ of the gates that is after the last multiplication gate.

For the step of reconstructing the output of the circuit, every party outputs $\llbracket b \rrbracket_i \oplus \text{aux}_b = 1$.

Indistinguishability. We then prove that the transcripts produced by the simulators (a, c_1, z_1, c_2, z_2) are computationally indistinguishable from those generated during actual protocol executions with an honest verifier.

We observe that $z_1 := (\{\text{seed}_j\}_{j \neq c_1}, h', \{\hat{z}_\alpha\})$ is indistinguishable from the real protocol since $\{\text{seed}_j\}_{j \neq c_1}, \hat{z}_\alpha$ are sampled uniformly as in the real protocol, and h' is computed by executing the MPC protocol *honestly* from seed_{c_1} and $\text{aux}_{c_1,n}$. We note that although SimA emulates $\text{aux}_{c_1,n}$, in this round, the verifier does not have any information about seed_{c_1} or $\text{aux}_{c_1,n}$.

Turning to $z_2 := (\{\text{state}_{c_1,i}\}_{i \neq c_2}, \text{com}_{c_1,c_2}, \text{msgs}_{c_2})$, we first note that com_{c_1,c_2} is computed exactly according to the protocol description, ensuring consistency with the initial message $a := h$. We now argue that even though SimA emulates $\text{aux}_{c_1,n}$, the values $(\{\text{state}_{c_1,i}\}_{i \neq c_2}, \text{msgs}_{c_2})$ remain indistinguishable from their real values in the actual protocol.

- If $c_2 = n$, then $\{\text{state}_{c_1,i}\}_{i \neq n}$ are indeed sampled randomly as in the honest execution.
- If $c_2 \neq n$, the verifier knows $\text{state}_{c_1,n} = \text{seed}_{c_1,n} \parallel \text{aux}_{c_1,n}$ but does not know seed_{c_1,c_2} . Consequently, the verifier cannot verify whether $\text{aux}_{c_1,n}$ was honestly generated by checking the correctness of the masked value shares for the final multiplication gates.

Moreover, the simulated values ensure that the MPC output is 1, and all broadcast messages $\{\text{msgs}_i\}_{i \in [n]}$ are computed honestly following MPC protocol's description. Therefore, $(\{\text{state}_{c_1, i}\}_{i \neq c_2}, \text{msgs}_{c_2})$ remain indistinguishable from the real values.

Case 2: $i_{zk}^* = 2$.

The CRSHVZK property at $i_{zk}^* = 2$ is proven by constructing two simulators ($\text{SimA}(x, c_2), \text{SimZ}$). We build these simulators based on the semi-honest security of the MPC protocol, for which there exists a simulator Sim that outputs simulated consistent views of $n - 1$ parties, i.e., all except for the c_2 -th party, as follows. Sim starts by sampling $\{\text{state}_i\}_{i \neq c_2}, \{\hat{z}_\alpha\}, \text{msgs}_{c_2}$ and uses these sampled values to simulate the online phase of the MPC protocol until the reconstruction output step. At this step Sim learns the shares of output $\llbracket b \rrbracket_i$ of i -th party ($i \neq c_2$), Sim then defines a $\llbracket b \rrbracket_p$ such that $\oplus \llbracket b \rrbracket_i = 1$, appends $\llbracket b \rrbracket_p$ into msgs_{c_2} and gets a new broadcast message msgs_{c_2} . Therefore, we can modify Sim syntactically by redefining its input to be a set of $\{\text{state}_i\}_{i \neq c_2}, \{\hat{z}_\alpha\}$ that are sampled randomly in advance, such that $\text{Sim}(\{\text{state}_i\}_{i \neq c_2}, \{\hat{z}_\alpha\}) \rightarrow \text{msgs}_{c_2}$.

Now, we define the simulators (SimA, SimZ) as follows, for challenges $c_1 \leftarrow \$[m], c_2 \leftarrow \$[n]$.

- $\text{SimA}(x, c_2) \rightarrow (st_1, a)$: For a known second-round challenge c_2 , the simulator does as follows. First, SimA chooses uniform seeds as the honest prover would do, obtains seed_j for $j \in [m]$ and from seed_j computes honestly $\{\text{seed}_{j,i}\}_{i \in [n]}$. Then, for each $j \in [m]$, SimA computes $\text{aux}_{j,n}$ from $\{\text{seed}_{j,i}\}_{i \in [n-1]}$. SimA defines $\text{state}_{j,i} = \text{seed}_{j,i}$ for all $i \in [n-1]$ and $\text{state}_{j,n} := \text{seed}_{j,n} \parallel \text{aux}_{j,n}$, and samples $\{\hat{z}_{j,\alpha}\}_{j \in [m]}$ honestly. Then, SimA runs the MPC simulator $\text{Sim}(\{\text{state}_{j,i}\}_{j \in [m], i \neq c_2}, \{\hat{z}_{j,\alpha}\}_{j \in [m]}) \rightarrow \{\text{msgs}_{j,c_2}\}_{j \in [m]}$. This is done m times to simulate m independent preprocessing phases. Next, the SimA computes each value h_j for $j \in [m]$ as the honest prover would. Finally, the SimA computes $h = H(h_1, \dots, h_m)$ honestly. Output $st_1 := (\{\text{state}_{j,i}\}_{j \in [m], i \neq c_2}, \{\hat{z}_{j,\alpha}\}_{j \in [m]}, \{\text{msgs}_{j,c_2}\}_{j \in [m]})$ and $a := h$.
- $\text{SimZ}(st_1, c_1) \rightarrow (z_1, st_2)$: To simulate the third message, SimZ starts by retrieving the states from st_1 . Then, it computes the remaining $\text{msgs}_{c_1, i}$ honestly for every $i \neq c_2$. Finally, it computes $h' = H(\text{msgs}_{c_1, 1}, \dots, \text{msgs}_{c_1, n})$ from the state. Output $st_2 := \{st_1, \text{msgs}_{c_1, c_2}\}$ and $z_2 := (\{\text{seed}_j\}_{j \neq c_1}, h', \{\hat{z}_{c_1, \alpha}\})$.
- $\text{SimZ}(st_2, c_2) \rightarrow z_2$: To simulate the fifth message, simply output, as the honest prover would do, the MPC simulation corresponding to the c_1 -th preprocessing phase except for the view of c_2 -th party. Output $z_2 := (\{\text{state}_{c_1, i}\}_{i \neq c_2}, \text{com}_{c_1, c_2}, \text{msgs}_{c_1, c_2})$.

Due to the security of the MPC protocol against all-but-one corruption, it is easy to see that by a standard hybrid argument, the transcripts produced by the simulator (a, c_1, z_1, c_2, z_2) are computationally indistinguishable from those generated during actual protocol executions with an honest verifier. $\square$

Cut-and-choose KKW. In the following lemma, we generalize KKW using $(m - \tau)$ -out-of- m cut-and-choose in preprocessing phase, i.e., the prover first generates and commits to m executions of the preprocessing stage, and later opens $m - \tau$ of them except τ (corresponding to a verifier's challenge), for $1 \leq \tau \leq m - 1$. The τ unopened materials are used for executing τ instances of MPC protocol later on. To be clear, for a transcript (a, c_1, z_1, c_2, z_2) , the challenges c_1, c_2 are the set of τ elements. Each hidden preprocessing material is used for one MPC instance, where it opens all-but-one views of parties. Denote $c_1 = (c_1^1, c_1^2, \dots, c_1^\tau) \in [m]^\tau$ and $c_2 = (c_2^1, c_2^2, \dots, c_2^\tau) \in [n]^\tau$.

Lemma 3. *Given the 5-move interactive honest-verifier zero-knowledge proof Π in [KKW18] using $(m - \tau)$ -out-of- m cut-and-choose in preprocessing phase ($\tau = O(1)$), assuming that the hash function used is collision-resistant and that the commitment scheme used is computationally binding and hiding, then the proof system Π is flexible and can be:*

- *Critical-round special honest-verifier zero-knowledge at round $i_{zk}^* = 1$ or at round $i_{zk}^* = 2$.*
- *Extended $\vec{k}$ -special sound where $\vec{k} = ((\binom{m-1}{\tau-1} + 1), 2)$.*

Proof. The proof of this lemma follows the same steps as Theorem 7. There is, however, one caveat, which is that the construction is now $((\binom{m-1}{\tau-1} + 1), 2)$ -special sound. Therefore, the extended extractor

ExExt needs an extended $((\binom{m-1}{\tau-1} + 1, 2)$ -tree of transcripts (with a total of $2\tau \cdot (\binom{m-1}{\tau-1} + 1)$ transcripts) to extract an actual witness with probability 1.

In the following, we reason about the number of branches required by ExExt. For two distinct challenges (c_2, c'_2) in the second round, we claim that there must exist at least one execution among the τ MPC executions where the indices of the hidden party differ. In particular, for $(c_2^1, c_2^2, \dots, c_2^\tau) \neq (c_2'^1, c_2'^2, \dots, c_2'^\tau)$, there must exist at least one index, denoted as $i^* \in \tau$ such that $c_2^{i^*} \neq c_2'^{i^*}$. Since $c_2^{i^*}, c_2'^{i^*}$ are corresponding to indices of hidden parties for the same i^* -th MPC execution. Now in the i^* -th MPC, given the transcripts z_2, z'_2 sent in response to c_2, c'_2 , the extractor ExExt can see the view of all parties and extract a witness. However, the witness is not guaranteed to be valid. As in Theorem 7, to ensure the ExExt succeeds in extracting a valid witness (or in breaking the collision-resistance of the hash function), ExExt must also recover the preprocessing material used for the i^* -th MPC execution. This means that the number of branches (or challenges) in the first round needs to be $\binom{m-1}{\tau-1} + 1$.

To see that the latter is true, note that there can be up to $\binom{m-1}{\tau-1}$ challenges under which the i^* -th preprocessing remains hidden: the other $\tau - 1$ hidden preprocessing materials can be picked arbitrarily from the remaining $m - 1$ indices. Therefore, ExExt requires $\binom{m-1}{\tau-1} + 1$ to guarantee that the i^* -th preprocessing can be learned with probability 1.

E More Applications

E.1 Accumulators

An accumulator, also known as a set commitment, is a succinct primitive that allows one to commit to a set of elements $S = \{x_1, \dots, x_t\}$ and then produce a short proof of membership for any $x_i \in S$ [Bd94, BP97]. We introduce a definition below, following the syntax in [Kol24].

Definition 23 (Accumulator). A (static) accumulator with domain $\mathcal{M}$ is a tuple of algorithms $(\text{Gen}, \text{Accum}, \text{WitGen}, \text{Ver})$ with the following syntax:

$\text{Gen}(1^\lambda) \rightarrow ck$: On input the security parameter λ and an upper bound for the set size n , $\text{Gen}(1^\lambda)$ returns a commitment key ck .

$\text{Accum}(ck, S) \rightarrow (com, st)$: On input a commitment key ck and a set $S \subset \mathcal{M}$, $\text{Accum}(ck, S)$ outputs a commitment com and a state st .

$\text{WitGen}(ck, st, m) \rightarrow \pi$: On input a commitment key ck , a state st and a value m , $\text{WitGen}(ck, st, m)$ outputs a membership proof π .

$\text{Ver}(ck, com, m, \pi) \rightarrow 0/1$: On input a commitment key ck , a commitment com , a message m , and an opening proof π , $\text{Ver}(ck, com, m, \pi)$ outputs 1 (accept) or 0 (reject).

Moreover, these algorithms should satisfy the following properties:

Completeness. For any $S \subset \mathcal{M}$ such that $|S| \leq n$ and for any $m \in S$,

$$\Pr \left[\text{Ver}(ck, com, m, \pi) = 1 \mid \begin{array}{l} ck \leftarrow \text{Gen}(1^\lambda, n) \\ (com, st) \leftarrow \text{Com}(ck, S) \\ \pi \leftarrow \text{WitGen}(ck, st, m) \end{array} \right] = 1.$$

Soundness. For any PPT adversary $\mathcal{A}$,

$$\Pr \left[\begin{array}{l} \text{Ver}(ck, com, m^*, \pi^*) = 1 \\ \wedge m^* \notin S^* \end{array} \mid \begin{array}{l} ck \leftarrow \text{Gen}(1^\lambda, n) \\ S^* \leftarrow \mathcal{A}(ck) \\ (com, st) \leftarrow \text{Com}(ck, S^*) \\ (m^*, \pi^*) \leftarrow \mathcal{A}(ck, com, st) \end{array} \right] \leq \text{negl}(\lambda).$$

Succinctness. Both com and π have size bounded by $\mathcal{O}(\lambda, \text{polylog}(n))$. The running time of Ver is also bounded by $\mathcal{O}(\lambda, \text{polylog}(n))$.

To construct our accumulators, we describe a slightly stronger variation of CRZK which requires a simulator that takes $k - 1$ possible challenges for the critical round. We introduce it below.

Definition 24 (Critical-Round k -Special Honest Verifier Zero-Knowledge). A $(2\mu+1)$ -move public-coin proof protocol is critical-round k -special honest verifier zero-knowledge (k -CRSHVZK or simply k -CRZK for short) at round i_{zk}^* if there exists a set of polynomial-time algorithm (SimAX, SimZX) that:

- SimAX takes x and $(c_{i_{zk}^*}^{(1)}, \dots, c_{i_{zk}^*}^{(k-1)}) \in \mathcal{C}_{i_{zk}^*}^{k-1}$ as input, and outputs (st_1, a) where st_1 is a state information and a is a prover's message at step 1.
- SimZX takes st_i and $c_i \in \mathcal{C}_i$, and outputs (st_{i+1}, z_i) where st_{i+1} is an updated state, and z_i is a prover's response for step $2i + 1$.
- For any $(x, w) \in L_{RW}$, $c_i \in \mathcal{C}_i$ for $i \in [\mu] \setminus \{i_{zk}^*\}$, $(c_{i_{zk}^*}^{(1)}, \dots, c_{i_{zk}^*}^{(k-1)}) \in \mathcal{C}_{i_{zk}^*}^{k-1}$, and any $c_{i_{zk}^*} \in (c_{i_{zk}^*}^{(1)}, \dots, c_{i_{zk}^*}^{(k-1)})$, distribution of $(a, c_1, z_1, \dots, c_\mu, z_\mu)$ generated as $(st_1, a) \leftarrow \text{SimAX}(x, c_{i_{zk}^*}^{(1)}, \dots, c_{i_{zk}^*}^{(k-1)})$, $(st_{i+1}, z_i) \leftarrow \text{SimZX}(st_i, c_i)$ for i from 1 to μ , and that of $(a', c_1, z'_1, \dots, c_\mu, z'_\mu)$ generated as $(st_1, a') \leftarrow A(x, w)$ and $(st_{i+1}, z'_i) \leftarrow Z(st_i, c_i)$ for i from 1 to μ are indistinguishable.

Construction of an Accumulator. We generalize our construction of a trapdoor commitment scheme (Figure 3) to construct an accumulator $\text{ACC}_{\Pi_{i^*}}$ from a $(2\mu + 1)$ -move public-coin proof protocol Π_{i^*} that is critical-round k_{i^*} -special honest verifier zero knowledge for $k_{i^*} \geq 3$. Our accumulator scheme requires that the maximum size of n that is supported by $\text{Gen}(1^\lambda, n)$ is bounded by the parameter $B = k_{i^*} - 1$ that depends on Π_{i^*} . We present our construction in Figure 5.

Recall that critical-round k_{i^*} -special HVZK (Definition 24) requires that there exists a zero-knowledge simulator $(st_1, a) \leftarrow \text{SimZX}(x, c_{i^*}^{(1)}, \dots, c_{i^*}^{(k-1)})$ where each $c_{i^*}^{(j)} \in \mathcal{C}_{i^*}$, i.e., that takes $k_{i^*} - 1$ challenges to simulate the first message of the protocol a .¹⁴ A peculiar consequence is that the message space $\mathcal{M} \subset \mathcal{C}_{i^*}$ must be a proper subset such that $|\mathcal{M}| \leq |\mathcal{C}_{i^*}| - B$. This is necessary for the proof of soundness to go through. The reason is that a commitment to S always needs to be simulated based on B challenges, and if $|S| < B$, the challenges not in S cannot be part of the message space to guarantee that the extraction of a complete tree of transcripts is successful. Hence, we need to “taint” B values from $\mathcal{C}_{i^*}$, which will be used in the simulation but cannot be part of the message space.

Theorem 8. The construction $\text{ACC}_{\Pi_{i^*}}$ in Figure 5 constitutes an accumulator if Π_{i^*} is a perfectly complete, extended- $\vec{k}$ -special sound, and k_{i^*} -special honest verifier zero-knowledge interactive proof at round $i_{zk}^* = i^*$. It is sound if G is a programmable random oracle, relation R_{hd} is one-way, and for all $i \in [\mu] \setminus i_{zk}^*$, $\frac{k_i}{|\mathcal{C}_i|}$ is negligible.

Proof. One can see by inspection that the construction satisfies correctness if $n \leq B$. The construction also satisfies succinctness as both com and π are independent of n and therefore of size $\mathcal{O}(\lambda)$.

For soundness, let $\mathcal{A}$ be an adversary against accumulator soundness with non-negligible success probability. We will show how to use $\mathcal{A}$ as a black box to construct an adversary $\mathcal{B}$ that breaks the one-wayness of the relations R or R_{hd} . For this, it suffices to show that $\mathcal{B}$ can use $\mathcal{A}$ to obtain a valid $\vec{k}$ -tree of transcripts. The proof follows a similar strategy as the proof of Theorem 4, so we skip some technical details.

The reduction $\mathcal{B}$ is given a challenge instance x and access to a programmable random oracle G . As in the previous proof, $\mathcal{B}$ follows a lazy (uniformly random) simulation strategy for G . $\mathcal{B}$ sets up the soundness game for $\mathcal{A}$ by setting $ck = x$. Then, $\mathcal{A}(ck)$ outputs a set $S = \{m_1, \dots, m_t\} \subset \mathcal{M}$ such that $t \leq n$.

Next, $\mathcal{B}$ runs $(com, st) \leftarrow \text{Com}(ck, S)$ where $com = (a, c_1, z_1, \dots, c_{i^*-1}, z_{i^*-1})$. Then, it runs $(m^*, \pi^*) \leftarrow \mathcal{A}(ck, com, st)$ which returns a valid proof π^* for a message $m^* \notin S$, which $\mathcal{B}$ parses as $\pi^* = (m^*, z_j^*, c_{j+1}^*, \dots, c_\mu^*, z_\mu^*)$. Note that $m^* \in \{m_{t+1}, \dots, m_{k_{i^*}-1}\}$ is not valid since $m_{t+1}, \dots, m_{k_{i^*}-1}$ are out of the message space. Recall that the goal of $\mathcal{B}$ is to obtain a $\vec{k}$ tree of transcripts. For every round $i \leq i^*$, $\mathcal{B}$ acts as in the proof of Theorem 4 and follows the tree builder in Figure 4. For rounds $i > i^*$, note that $\mathcal{B}$ can simulate $k_{i^*} - 1$ valid sub-trees that start at level i^* by simply running $(st_{i+1}, z_{i+1}) \leftarrow \text{SimZX}(st_i, c_{i+1})$ for all the required branches and their corresponding challenges, where at level i^* it sets $c_{i^*}^{(j)} = m_j$ for every $j = 1, \dots, k_{i^*} - 1$ (recall that the accumulator st contains messages $m_1, \dots, m_t, m_{t+1}, \dots, m_{k_{i^*}-1}$, where $S = \{m_1, \dots, m_t\}$ and $m_{t+1}, \dots, m_{k_{i^*}-1} \notin \mathcal{M}$).

¹⁴ One example of such protocol is the three-move Stern Σ -protocol [Ste06], which is trivially 3-critical at its only round.

Set up.

- Let $\Pi_{i^*} = (A, Z, V, \text{SimAX}, \text{SimZX})$ be a $(2\mu + 1)$ -move public-coin proof for relation R_{hd} that is extended- $\vec{k}$ -special sound, and k_{i^*} -CRSHVZK at round $i_{zk}^* = i^*$ where $k_{i^*} \geq 3$.
- Let $B = k_{i^*} - 1$ and $\mathcal{C}_{i^*}$ be the challenge space of Π_{i^*} at the i^* -th round.
- Let the message space $\mathcal{M} \subset \mathcal{C}_{i^*}$ be a proper subset such that $|\mathcal{M}| \leq |\mathcal{C}_{i^*}| - B$.

Construction.

Gen: Given 1^λ and a size bound n as input, check whether $n \leq B$ and otherwise abort. Sample an instance $(x, w) \leftarrow \mathcal{L}_{RW}(\lambda)$. The commitment key is $ck := x$.

Accum: Given (ck, S) as input, parse $ck = x$ and $S = (m_1, \dots, m_t)$. Abort if $t > n$, otherwise do:

- If $t < B$, choose $B - t$ arbitrary elements $m_{t+1}, \dots, m_B \in \mathcal{C}_{i^*} \setminus \mathcal{M}$.
- Compute $(st_1, a) \leftarrow \text{SimAX}(x, m_1, \dots, m_B)$.
- For $i = 1, \dots, i^* - 1$, do

$$c_i := G(x, a, c_1, z_1, \dots, c_{i-1}, z_{i-1}) \quad (3)$$

and $(st_{i+1}, z_i) \leftarrow \text{SimZ}(st_i, c_i)$.

- For $i = i^*$ and for $j = 1, \dots, t$, do $c_i^{(j)} := m_j$ and $(st_{i+1}^{(j)}, z_i^{(j)}) \leftarrow \text{SimZX}(st_i, c_i^{(j)})$.
- For $i = i^* + 1, \dots, \mu$ and $j = 1, \dots, t$, do

$$c_i^{(j)} := G(x, a, c_1, z_1, \dots, c_{i-1}^{(j)}, z_{i-1}^{(j)}) \quad (4)$$

and $(st_{i+1}^{(j)}, z_i^{(j)}) \leftarrow \text{SimZX}(st_i^{(j)}, c_i^{(j)})$.

- Output $com := (a, c_1, z_1, \dots, c_{i^*-1}, z_{i^*-1})$ and $st := \{m_j, z_{i^*}^{(j)}, c_{i^*+1}^{(j)}, z_{i^*+1}^{(j)}, \dots, c_\mu^{(j)}, z_\mu^{(j)}\}_{j=1, \dots, t}$.

WitGen: Given (ck, m, st) as input, check whether $m = m_j$ for some $m_j \in st$, and otherwise abort.

Then, parse st to output $\pi_j = (m_j, z_{i^*}^{(j)}, c_{i^*+1}^{(j)}, z_{i^*+1}^{(j)}, \dots, c_\mu^{(j)}, z_\mu^{(j)})$.

Ver: Given (ck, com, m, π) as input, parse them into $(x, a, c_1, z_1, \dots, c_\mu, z_\mu)$ with $c_{i^*} := m$.

Check if every c_i satisfies the relations in (3) and (4), and output $V(x, a, c_1, z_1, \dots, c_\mu, z_\mu)$.

Fig. 5. Accumulator $\text{ACC}_{\Pi_{i^*}}$.

Therefore, to get a valid $\vec{k}$ tree of transcripts T^* , $\mathcal{B}$ only needs one additional sub-tree that starts at level i^* . The missing sub-tree can be obtained by setting $c_{i^*}^{(k_{i^*})} = m^*$ and by rewinding $\mathcal{A}$ while reprogramming the random oracle G to provide distinct (uniformly random) challenges at every iteration, following the same steps as in the last step of Figure 4.

The remaining part of the analysis is identical to that of Theorem 4. $\square$

Trapdoor Commitment from $k_{i^} \geq 2$.* Inspired by our accumulator, we present a feasibility result to show how to build a trapdoor commitment scheme if Π_{i^*} is CRZK with $k_{i^*} \geq 2$ at the critical round, generalizing the $k_{i^*} \leq 2$ result from earlier sections. The idea is to extend the openings of the trapdoor commitment scheme to include $t = \lceil k_{i^*}/2 \rceil$ transcripts (branching at level i^*) in the opening information. Then, any adversary who breaks commitment binding provides k_{i^*} transcripts branching at level i^* , and security follows via a rewinding argument as in the previous constructions.

We also need that all challenges at level i^* are distinct. For this, we use an encoding $\gamma : \mathcal{M} \rightarrow \mathcal{C}_{i^*}^t$ such that for any element $c \in \mathcal{C}_{i^*}$, there exists at most one element $m \in \mathcal{M}$ such that $c \in \gamma(m)$. Intuitively, this condition ensures that any challenge $c \in \text{Im}(\gamma)$ is associated to a unique $m \in \mathcal{M}$. We describe the commit and verify algorithms in more detail:

Com (ck, m) : Start by computing $(st_1, a) \leftarrow \text{SimAX}(x, \gamma(m))$. Without loss of generality, we assume the simulator takes t challenges; it can just use elements of $\mathcal{C}_{i^*} \setminus \text{Im}(\gamma)$ if more are required.

- For $i = 1, \dots, i^* - 1$, compute c_i and z_i as in Figure 5.
- For $i = i^*$ and for $j = 1, \dots, t$, do $c_i^{(j)} := \gamma(m)_j$ and $(st_{i+1}^{(j)}, z_i^{(j)}) \leftarrow \text{SimZX}(st_i, c_i^{(j)})$.
- For all remaining $i = i^* + 1, \dots, \mu$ and $j = 1, \dots, t$, compute the challenges $c_i^{(j)}$ and messages $z_i^{(j)}$ as in Figure 5.

Output:

$$com := (a, c_1, z_1, \dots, c_{i^*-1}, z_{i^*-1}),$$

$$open := \{\gamma(m)_j, z_{i^*}^{(j)}, c_{i^*+1}^{(j)}, z_{i^*+1}^{(j)}, \dots, c_\mu^{(j)}, z_\mu^{(j)}\}_{j=1, \dots, t}.$$

$\text{Ver}(ck, com, m, open)$: Check the validity of every c_i until round $i^* - 1$ as in Figure 5. Then, compute $c_{i^*}^{(j)} := \gamma(m)_j$ for $j = 1, \dots, t$ and check the validity of every $c_i^{(j)}$ for $j = i^* + 1, \dots, \mu$. Finally, check that $V(x, a, c_1, z_1, \dots, c_i^{(j)}, z_i^{(j)}, \dots, c_\mu^{(j)}, z_\mu^{(j)}) = 1$ for every $j = 1, \dots, t$.

E.2 Trapdoor Commitments with Flexible Trapdoor Allocation

Observe that the MR-CDS composition in Section 5.2 is closed for some class of protocols, i.e., the resulting proof is also critical-round zero-knowledge. Thus, it can be seamlessly combined with our construction of the trapdoor commitment scheme in Section 6.

This modular construction allows for flexible trapdoor setup regarding a monotone access structure of one's interest. For instance, by combining proofs for statements A and B into $A \vee B$ using the composition, and using the composed critical-round proof system to build the commitment scheme, we obtain a trapdoor commitment scheme that is equivocal only if the committer knows a witness for either A or B . One could alternatively base the construction on a critical-round proof system for NP. However, this modular approach is particularly advantageous when the underlying relation can be decomposed into sub-relations that each admit efficient critical-round proofs, such as KKW protocols or standard sigma-protocols. This enables more efficient and flexible instantiations of the trapdoor commitment scheme, tailored to the structure of the specific relations involved.